Elaborato di Architettura dei Sistemi Integrati

Indice

Indice 1

Prima Esercitazione: Adder a 16bit 3

Seconda Esercitazione: ALU del processore 7

- Modulo aritmetico 7
- Modulo logico 11
- ALU – Unità aritmetico-logica 14

Terza Esercitazione: Sintesi della ALU del processore 20

- Sintesi in componenti generici 20
- Sintesi con celle standard 21
- Timing Analysis 21
- Sintesi con vincoli temporali e scripts di sintesi 22

Quarta Esercitazione: Sintesi del datapath del processore 25

- Il datapath 25
- Sintesi 28
- Modello di timing per il vincolo sul tempo di setup 29
- Modello di timing per il vincolo sul tempo di hold 30
- Effetto delle non-idealità del clock 31
- Simulazione post-layout con annotazione dei ritardi 32
- Quinta Esercitazione: Placement e Routing del datapath 33
- Configurazione 34
- Floorplan 35
- Placement 36
- In-place Optimization 37
- Albero di clock 38
  - Clock Skew Summary 38
- Routing 39
- Controllo 39

Sesta Esercitazione: Simulazione a livello transistor 41

Alessandro Arrichiello M63/43
Flavio Battimo M63/38

Analisi delle caratteristiche in DC dei MOSFET 41
Leakage 43
Analisi dinamica 44
Netlist gerarchiche 46
Tempi di setup e di hold di un registro 47
Simulazione con ultrasim 49
Settima Esercitazione: Dal livello layout alla simulazione transistor 50
Maschere per la tecnologia CMOS 50
Struttura delle celle standard 53
Controllo delle regole di progetto 54
Estrazione circuitale 57
Simulazione del circuito estratto dal layout 63
Estrazione del datapath di pico 66
Controllo delle regole di progetto 66
Estrazione circuitale 67

Alessandro Arrichiello M63/43
Flavio Battimo M63/38

Prima Esercitazione: Adder a 16bit

Durante la prima esercitazione ci è stato introdotto l'uso della suite di sviluppo Cadence Design Systems, in particolare abbiamo preso familiarità con il tool nc-sim e con la scrittura iniziale di un primo listato VHDL di prova.

Come primo listato VHDL abbiamo sviluppato un adder a 16bit utilizzando parte del codice fornito all'esercitazione,

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

ENTITY adder16 IS
  PORT (
      a,b : in std_logic_vector(15 downto 0);
      s : out std_logic_vector(15 downto 0);
      c : in std_logic
  );
END adder16;

architecture dataflow of adder16 is

-- port di tipo std_logic ed std_logic_vector
-- segnali di tipo signed per effettuare l'addizione

SIGNAL as, bs, cs, ss: signed(a'range);

BEGIN
cs <= (0 => c, others => '0');
ss <= as + bs + cs;
as <= signed (a);
bs <= signed (b);

s <= std_logic_vector (ss);

end dataflow;
```

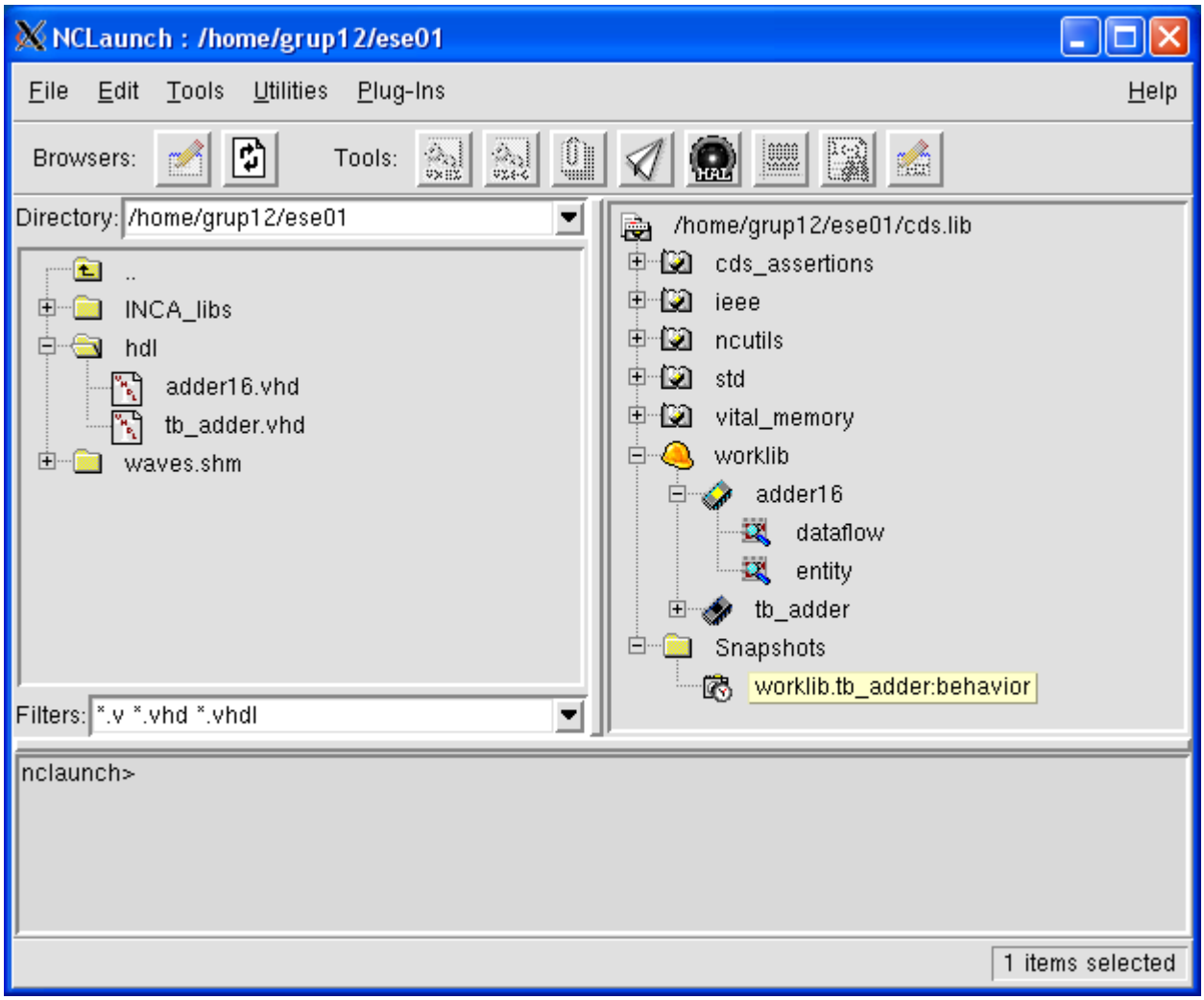
La parte mancante del codice consisteva principalmente nella definizione delle porte, in questo caso cioè degli ingressi **a**, **b** e il carry-in **c** più l'uscita **s**.

In questo caso l'esempio mostrato è stato realizzato con una rappresentazione di tipo dataflow, tale rappresentazione descrive il modo con il quale i dati fluiscono dai terminali di ingresso, attraverso i segnali interni, fino ai terminali di uscita dell'entità, utilizzando operatori predefiniti.

Dopo aver definito dei segnali interni andiamo ad effettuare dopo il BEGIN la connessione dei terminali, andando a collegare gli ingressi **a**, **b** e **c** ai segnali interni **as**, **bs** e **cs** e quindi andando a specificare l'operazione da effettuare, cioè la somma tra **as**, **bs** e **cs** nel segnale interno **ss**, collegato a sua volta all'uscita **s**.

Alessandro Arrichiello M63/43
Flavio Battimo M63/38

Successivamente abbiamo esplorato il tool di simulazione nclaunch appartenente sempre alla suite Cadence. Dopo aver configurato opportunamente il workspace e le cartelle contenenti i file vhdl da noi prodotti, abbiamo testato la compilazione e la correttezza della sintassi del codice VHDL mandando il listato in compilazione attraverso l'apposito pulsante.



Quindi dopo aver verificato la correttezza del codice VHDL abbiamo prodotto un file di **TestBench** per testare l'effettivo funzionamento dell'elemento **Adder** prodotto.

Alessandro Arrichiello M63/43
Flavio Battimo M63/38

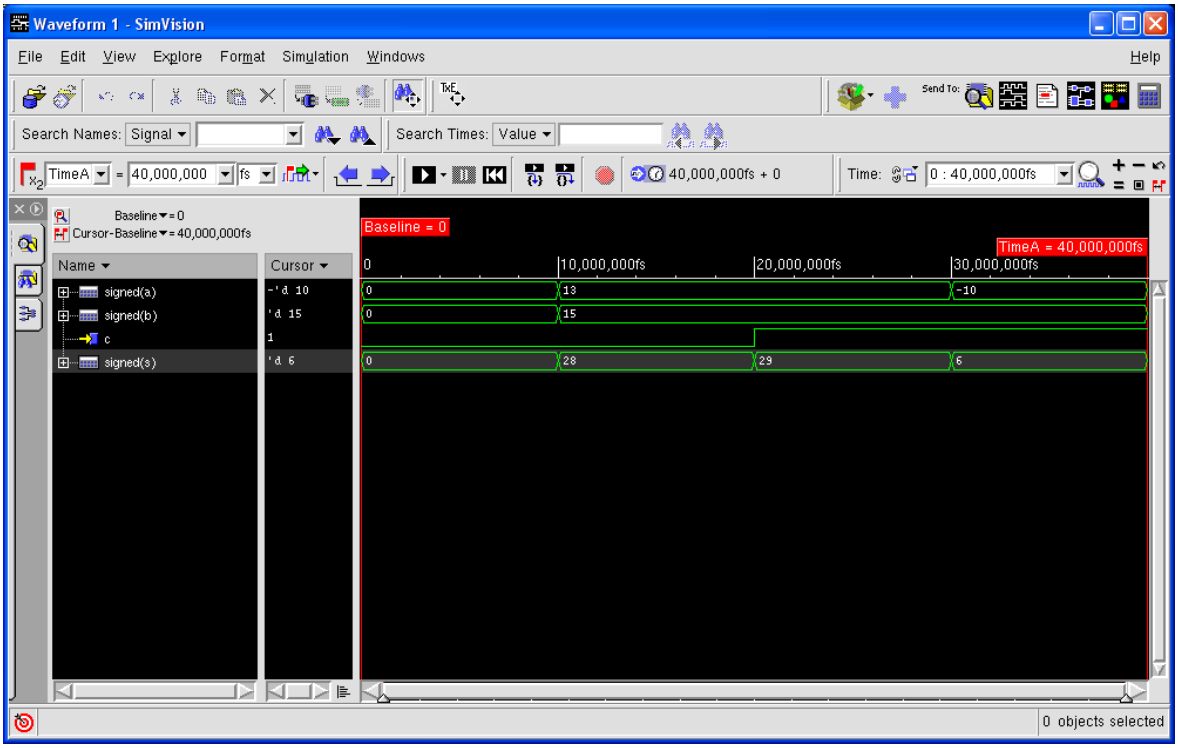
Abbiamo quindi completato il codice del testbench fornito nell'esercitazione aggiungendo la definizione delle porte come si può notare dal codice seguente:

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;
use ieee.numeric_std.all;

ENTITY tb_adder IS
END tb_adder;

ARCHITECTURE behavior OF tb_adder IS

COMPONENT adder16
  PORT (
  a,b : in std_logic_vector(15 downto 0);
  s : out std_logic_vector(15 downto 0);
  c : in std_logic
  );
END COMPONENT;

SIGNAL mya,myb,mys:  std_logic_vector(15 downto 0);
SIGNAL myc:  std_logic;

BEGIN
      uut: adder16 PORT MAP(
            a => mya,
            b => myb,
            c => myc,
            s => mys
      );

tb : PROCESS
   BEGIN
      mya <= B"0000_0000_0000_0000"; myb <= B"0000_0000_0000_0000";
      myc <= '0';
      wait for 10 ns;
      mya <= B"0000_0000_0000_1101"; myb <= B"0000_0000_0000_1111";
      wait for 10 ns;
      myc <= '1';
      wait for 10 ns;
      mya <= std_logic_vector ( to_signed(-10, 16) );
      wait for 10 ns;
      wait; -- will wait forever
END PROCESS;

END behavior;
```

Successivamente abbiamo compilato il listato prodotto attraverso il software nclaunch e quindi abbiamo simulato il testbench lanciando il simulatore **SimVision** con l'apposito pulsante in nclaunch.

Alessandro Arrichiello M63/43
Flavio Battimo M63/38

A questo punto abbiamo iniziato ad esplorare il simulatore scegliendo le variabili da monitorare e di cui tracciare il grafico rappresentativo. Dopo aver selezionato le variabili abbiamo lanciato il tool **WaveForm Viewer**, il quale una volta avviata la simulazione ci ha mostrato l'andamento dei segnali selezionati (in questo caso **a**,**b**,**c**,**s**) in modo da verificare l'effettivo funzionamento dell'elemento **Adder** prodotto in precedenza. Per garantire una corretta comprensione dei grafici abbiamo anche modificato la tipologia di rappresentazione dei valori nel tool WaveForm Viewer da esadecimale a **decimale signed**.



Al fine di velocizzare ed automatizzare quelle che sono le procedure di compilazione e testing dei listati VHDL prodotti ci è stato suggerita la creazione di un file di script che prevedesse le operazioni in sequenza da eseguire al fine di testare l'elemento **Adder** proposto. A tal scopo abbiamo creato seguendo le indicazioni un file sim.script nel quale abbiamo aggiunto:

- Il comando per la compilazione e l'aggiunta al workspace del file adder16.vhd
- Il comando per la compilazione e l'aggiunta al workspace del file di testbench tb_adder.vhd
- Il comando per l'esecuzione dell'ambiente di testing WaveForm Viewer con argomento il file restore.tcl generato a runtime in una precedente esecuzione del testbench.

Tutto ciò è mostrato di seguito:

ncvhdl -work worklib -cdslib /home/grup12/ese01/cds.lib -logfile ncvhdl.log -errormax 15 -update -v93 -linedebug -status /home/grup12/ese01/hdl/**adder16.vhd**
ncvhdl -work worklib -cdslib /home/grup12/ese01/cds.lib -logfile ncvhdl.log -errormax 15 -update -v93 -linedebug -status /home/grup12/ese01/hdl/**tb_adder.vhd**
ncsim -gui -cdslib /home/grup12/ese01/cds.lib -logfile ncsim.log -errormax 15 -status worklib.tb_adder:behavior -input **restore.tcl**

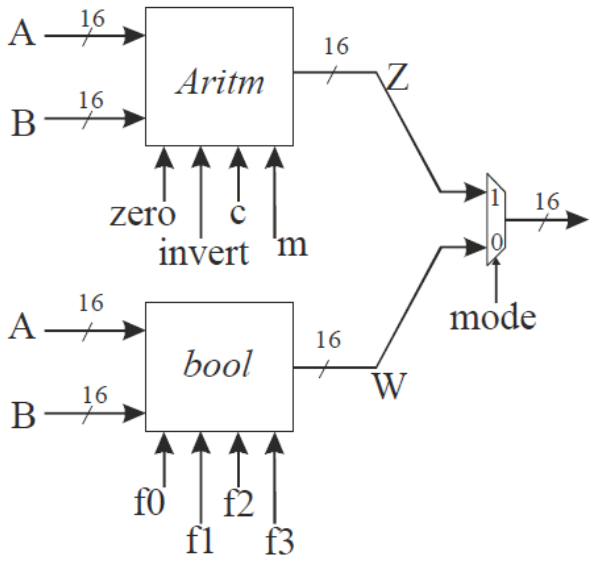
Abbiamo infine testato il corretto funzionamento dello script prodotto attraverso l'esecuzione del comando: **source sim.script &**

Alessandro Arrichiello M63/43
Flavio Battimo M63/38

Seconda Esercitazione: ALU del processore

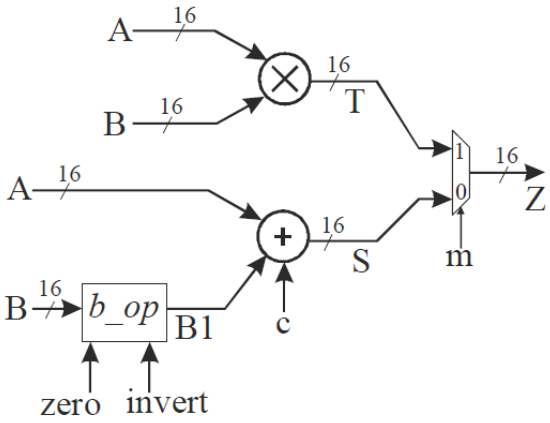
Nella seconda esercitazione ci è stato proposto di iniziare lo sviluppo dell'ALU del processore PICO. Ricordiamo che la **ALU (Unità aritmetico-logica)** di un processore è quell'unità preposta all'esecuzione dei calcoli aritmetici e logici all'interno di un processore.

L'unità logico-aritmetica che ci è stata proposta di sviluppare è suddivisa in due sottosezioni: una parte aritmetica ed un blocco booleano, con un multiplexer che seleziona l'uscita dei due blocchi, come mostrato nella figura di seguito:



Modulo aritmetico

Analizzando la **parte aritmetica** si può notare che essa è formata da un **moltiplicatore** con due ingressi a 16 bit ed un'uscita a 16bit. Questa limitazione è stata introdotta per rendere il formato di uscita del moltiplicatore congruente con il resto dei segnali utilizzati all'interno dell'ALU.



Sempre all'interno della parte aritmetica abbiamo quindi un **addizionatore** con due ingressi a 16bit, uno dei quali fornito in uscita da un blocco zero-inverter, necessario al fine di calcolare un eventuale sottrazione (corrispondente in binario alla somma del numero che si vuole sottrarre invertito e aggiungendo 1, rappresentazione in complementi a 2).

Entrambe le uscite, del moltiplicatore e dell'addizionatore, entrano in un **multiplexer** pilotato da un segnale di selezione.

Una volta stabilito il completo funzionamento dell'addizionatore abbiamo iniziato lo sviluppo della parte aritmetica, sviluppando l'**addizionatore** comprensivo di modulo zero-invertente **b_op** e di **moltiplicatore** sviluppando il relativo listato in VHDL.

Alessandro Arrichiello M63/43
Flavio Battimo M63/38

Per ogni componente ci è stata fornita una tabella rappresentativa attraverso la quale è stato implementato il rispettivo componente descritto.

Tabella descrittiva del componente zero-invertente **b_op**:

zero	Invert	B1
0	0	B
0	1	not B
1	0	0
1	1	111…11

Tabella descrittiva del componente **Addizionatore**:

invert	zero	c	B1	S
0	0	0	B	A+B
0	0	1	B	A+B+1
0	1	0	0	A
0	1	1	0	A+1
1	0	0	not B	A-B-1
1	0	1	not B	A-B
1	1	0	111…11	A-1
1	1	1	111…11	A

Di seguito sarà mostrato il listato VHDL prodotto, effettuiamo però innanzitutto delle considerazioni riguardo all'implementazione adottata. Abbiamo deciso di utilizzare una rappresentazione VHDL dataflow per la parte aritmetica pertanto vista la semplicità delle operazioni, abbiamo deciso al fine anche di migliorare la leggibilità del codice da noi prodotto di accorpare in un unico listato VHDL l'intera parte aritmetica.

Dopo aver definito le connessioni entranti/uscenti della parte aritmetica abbiamo quindi dichiarato dei segnali interni necessari al calcolo delle operazioni richieste (**BZ,BZ1**,**as**,**bs**,**zs**,**b1**,**cs**,**zss**), abbiamo inoltre dichiarato un segnale da 32bit (**zms**) per ospitare il risultato del moltiplicatore dal quale prendiamo solo i 16bit più significativi.

Quindi abbiamo collegato i segnali di ingresso **A**, **B** e **c** rispettivamente ad **as**, **bs** e **cs**. A questo punto abbiamo mappato gli ingressi **as**, **bs** e **cs** e l'uscita **zss** all'addizionatore e gli ingressi **as** e **bs** e l'uscita **zms** al moltiplicatore. In questo modo i due componenti (addizionatore e moltiplicatore) funzionano in parallelo, sarà poi il multiplexer successivamente definito a selezionare l'uscita desiderata.

Come si può notare il moltiplicatore e l'addizionatore sono stati creati usando le librerie VHDL interne, utilizzando semplicemente i rispetti operatori di somma (**+**) e prodotto (*****).

Successivamente abbiamo mappato il blocco (zero-invertente) dell'ingresso **B**, necessario per l'implementazione sia dell'operazione di differenza sia per la generazione di ingresso nullo, con il segnale **B1**.

Infine nell'ultima parte del codice abbiamo generato il multiplexer necessario alla selezione dell'uscita (**Z**) del moltiplicatore o dell'addizionatore a seconda del bit di ingresso **m**.

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
entity aritm is
port(
A : in std_logic_vector(15 downto 0);
B : in std_logic_vector(15 downto 0);
zero, invert, c, m : in std_logic;
Z : out std_logic_vector(15 downto 0)
);
end entity;
architecture dataflow of aritm is

signal BZ,BZ1 : std_logic_vector(a'range);
signal as,bs,zs,b1,cs,zss : signed(a'range);
signal zms : signed((2*a'left)+1 downto 0);

BEGIN
as<=signed(A);
bs<=signed(B);
cs <= (0 => c , others => '0');

zms <= as*bs;
zss <= as+b1+cs;

BZ <= B when zero='0' else (others=>'0');
BZ1 <= BZ when invert='0' else not BZ;
B1 <= signed(BZ1);

zs <= zms(zms'left downto a'left+1) when m='1'

Z<=std_logic_vector(zs);

end dataflow;
```

Come suggeritoci dal testo dell'esercitazione abbiamo quindi provveduto a sviluppare un testbench che testasse l'effettivo funzionamento del modulo aritmetico sviluppato. Per semplicità abbiamo definito un vettore di 4 bit per pilotare i segnali **zero**, **invert**, **c**, **m**.

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

ENTITY tb_aritm IS
END tb_aritm;

architecture test of tb_aritm is
component aritm is port (
A : in std_logic_vector(15 downto 0);
B : in std_logic_vector(15 downto 0);
zero, invert, c, m : in std_logic;
Z : out std_logic_vector(15 downto 0)
);
end component;
signal myA, myB, myZ:std_logic_vector(15 downto 0);
signal codice: std_logic_vector (3 downto 0);
-- e' costituito, nell'ordine, da:
-- m, invert, zero, c
begin
dut: aritm port map(
```


Alessandro Arrichiello M63/43
Flavio Battimo M63/38

```
A => myA,
B => myB,
m => codice(3),
invert => codice(2),
zero => codice(1),
c => codice(0),
Z => myZ);
stimulus: process
begin
myA <= std_logic_vector(to_signed(99, 16));
myB <= std_logic_vector(to_signed(43, 16));
codice <= "0000";
wait for 10 ns;
assert myZ=std_logic_vector(to_signed(142, 16))
report "errore in Z=A+B"
severity warning;

myA <= std_logic_vector(to_signed(99, 16));
myB <= std_logic_vector(to_signed(43, 16));
codice <= "0010";
wait for 10 ns;
assert myZ=std_logic_vector(to_signed(99, 16))
report "errore in Z=A"
severity warning;

myA <= std_logic_vector(to_signed(99, 16));
myB <= std_logic_vector(to_signed(43, 16));
codice <= "0011";
wait for 10 ns;
assert myZ=std_logic_vector(to_signed(100, 16))
report "errore in Z=A+1"
severity warning;

myA <= std_logic_vector(to_signed(99, 16));
myB <= std_logic_vector(to_signed(43, 16));
codice <= "0101";
wait for 10 ns;
assert myZ=std_logic_vector(to_signed(56, 16))
report "errore in Z=A-B"
severity warning;

myA <= std_logic_vector(to_signed(99, 16));
myB <= std_logic_vector(to_signed(111, 16));
codice <= "0101";
wait for 10 ns;
assert myZ=std_logic_vector(to_signed(-12, 16))
report "errore in Z=A-B"
severity warning;

myA <= std_logic_vector(to_signed(99, 16));
myB <= std_logic_vector(to_signed(111, 16));
codice <= "0110";
wait for 10 ns;
assert myZ=std_logic_vector(to_signed(98, 16))
report "errore in Z=A-1"
severity warning;

myA <= std_logic_vector(to_signed(4352, 16));
myB <= std_logic_vector(to_signed(2304, 16));
codice <= "1000";
wait for 10 ns;
assert myZ=std_logic_vector(to_signed(153, 16))
```

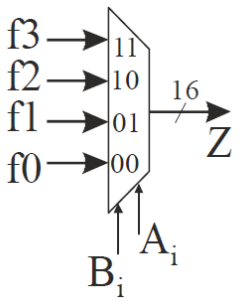
```
report "errore in Z=A*B"
severity warning;

myA <= std_logic_vector(to_signed(4352, 16));
myB <= std_logic_vector(to_signed(2304, 16));
codice <= "1101";
wait for 10 ns;
assert myZ=std_logic_vector(to_signed(153, 16))
report "errore in Z=A*B"
severity warning;

myA <= std_logic_vector(to_signed(7549, 16));
myB <= std_logic_vector(to_signed(13773, 16));
codice <= "1000";
wait for 10 ns;
assert myZ=std_logic_vector(to_signed(1586, 16))
report "errore in Z=A*B"
severity warning;

myA <= std_logic_vector(to_signed(-5888, 16));
myB <= std_logic_vector(to_signed(2560, 16));
codice <= "1000";
wait for 10 ns;
assert myZ=std_logic_vector(to_signed(-230, 16))
report "errore in Z=A*B"
severity warning;

myA <= std_logic_vector(to_signed(-5888, 16));
myB <= std_logic_vector(to_signed(2560, 16));
codice <= "1000";
wait for 10 ns;
assert myZ=std_logic_vector(to_signed(-20, 16))
report "errore in Z=A*B"
severity warning;

wait for 10 ns;
wait;
end process;

end test;
```

Come si può notare dopo l'applicazione di ogni ingresso il testbench prevede il controllo dell'uscita prodotta, in questo modo si può controllare la correttezza della simulazione senza dover verificare ogni singola forma d'onda prodotta dal simulatore.

Dopo aver verificato il corretto funzionamento del modulo aritmetico sviluppato abbiamo iniziato l'implementazione del modulo logico.

Modulo logico

L'unità logica realizza tutte le 16 possibili funzioni booleane di due variabili. Essa, per ciascun bit di A e B, utilizza un multiplexer 4/1, come mostrato in figura:



I valori dei segnali f3-f0 determinano quale delle possibili funzioni booleane viene implementata. Abbiamo quindi implementato tutte le possibili operazioni associate ai 16 possibili valori assumibili dal vettore f0-f3.

Alessandro Arrichiello M63/43
Flavio Battimo M63/38

Tabella riepilogativa delle operazioni implementate nella parte logica:

f3	f2	f1	f0	W
0	0	0	0	0
0	0	0	1	A NOR B
0	0	1	0	A AND NOT B
0	0	1	1	NOT B
0	1	0	0	B AND NOT A
0	1	0	1	NOT A
0	1	1	0	A XOR B
0	1	1	1	A NAND B
1	0	0	0	A AND B
1	0	0	1	A XNOR B
1	0	1	0	A
1	0	1	1	A OR NOT B
1	1	0	0	B
1	1	0	1	B OR NOT A
1	1	1	0	A OR B
1	1	1	1	1

Abbiamo quindi implementato in un file VHDL il modulo logico in modo che rispettasse la tabella di riferimento. Per semplicità di scrittura e lettura abbiamo implementato il tutto con il costrutto **WHEN-ELSE**.

```
library ieee;
use ieee.std_logic_1164.all;
entity bool is
port(
A,B : in std_logic_vector(15 downto 0);
f3, f2, f1, f0 : in std_logic;
W : out std_logic_vector(15 downto 0)
);
end entity bool;
architecture behavioural of bool is
signal s: std_logic_vector(3 downto 0);
BEGIN
s<=(f3,f2,f1,f0);
W <= (others => '0')     when s="0000"
      else A nor B       when s="0001"
      else A and not B   when s="0010"
      else not B         when s="0011"
      else B and not A   when s="0100"
      else not A         when s="0101"
      else A xor B       when s="0110"
      else A nand B      when s="0111"
      else A and B       when s="1000"
      else A xnor B      when s="1001"
      else A             when s="1010"
      else A or not B    when s="1011"
      else B             when s="1100"
      else B or not A    when s="1101"
      else A or B        when s="1110"
      else (others => '1');

end behavioural;
```


Alessandro Arrichiello M63/43
Flavio Battimo M63/38

Infine come suggeritoci nell'esercitazione abbiamo provveduto a testare il listato VHDL prodotto usando la tabella di riferimento proposta. Per fare ciò abbiamo scritto un Testbench che ne verificasse il corretto funzionamento.

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

ENTITY tb_bool IS
END tb_bool;

architecture test of tb_bool is
component bool is port (
A,B : in std_logic_vector(15 downto 0);
f3, f2, f1, f0 : in std_logic;
W : out std_logic_vector(15 downto 0)
);
end component;
signal myA, myB, myW:std_logic_vector(15 downto 0);
signal codice: std_logic_vector (3 downto 0);
-- e' costituito, nell'ordine, da:
-- m, invert, zero, c
begin
dut: bool port map(
A => myA,
B => myB,
f3 => codice(3),
f2 => codice(2),
f1 => codice(1),
f0 => codice(0),
W => myW);
stimulus: process
begin
myA <= B"0000_0011_0000_1111";
myB <= B"1010_1101_1111_0000";


codice <= "0000";
wait for 10 ns;
assert myW=B"0000_0000_0000_0000"
report "errore in W=0"
severity warning;

codice <= "0001";
wait for 10 ns;
assert myW=B"0101_0000_0000_0000"
report "errore in W=A nor B"
severity warning;

codice <= "0011";
wait for 10 ns;
assert myW=B"0101_0010_0000_1111"
report "errore in W=not B"
severity warning;

codice <= "0101";
wait for 10 ns;
assert myW=B"1111_1100_1111_0000"
report "errore in W=not A"
severity warning;
```

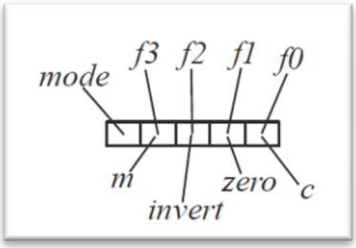

Alessandro Arrichiello M63/43
Flavio Battimo M63/38

```
codice <= "0110";
wait for 10 ns;
assert myW=B"1010_1110_1111_1111"
report "errore in W=A xor B"
severity warning;

codice <= "1010";
wait for 10 ns;
assert myW=B"0000_0011_0000_1111"
report "errore in W=A"
severity warning;

codice <= "1100";
wait for 10 ns;
assert myW=B"1010_1101_1111_0000"
report "errore in W=B"
severity warning;

codice <= "1110";
wait for 10 ns;
assert myW=B"1010_1111_1111_1111"
report "errore in W=A or B"
severity warning;

codice <= "1111";
wait for 10 ns;
assert myW=B"1111_1111_1111_1111"
report "errore in W=1"
severity warning;



wait for 10 ns;
wait;
end process;

end test;
```

Come si può notare ad ogni applicazione di nuovi ingressi, la correttezza della relative uscita è verificata attraverso l'uso del costrutto **ASSERT-REPORT**.

ALU – Unità aritmetico-logica

Abbiamo quindi provveduto al collegamento dei due moduli precedentemente sviluppati al fine di creare l'unità aritmetico-logica vera e propria. I segnali di controllo dell'ALU, infatti, sono composti dal solo codice operativo di cinque bit (opcode) che utilizzeremo per controllare contemporaneamente l'unità aritmetica, quella logica ed il multiplexer. I bit del codice operativo codificano i segnali di controllo delle due unità secondo il seguente schema:



Il bit più significativo del codice operativo corrisponde al segnale mode del multiplexer dell'ALU, e determina se il risultato dell'ALU è calcolato dall'unità aritmetica o da quella logica. I bit da quattro a zero sono condivisi fra le linee m, invert, zero, c ed f3 f2 f1 f0.

Alessandro Arrichiello M63/43
Flavio Battimo M63/38

Quindi l'unità artimetico-logica si comporterà diversamente a seconda dei bit di controllo impostati:

Codice operativo	Operazione
da 0_0000 a 0_1111	Modalità logica
da 1_0000 a 1_0111	Modalità aritmetica(somma/sottr/decr/incr)
da 1_1000 a 1_1111	Modalità aritmetica(moltipl)

Il listato VHDL prodotto che collega i due moduli (aritmetico e logico) ai terminali dell'ALU è riportato di seguito:

```
library ieee;
use ieee.std_logic_1164.all;
entity alu is
port(
      A,B : in std_logic_vector(15 downto 0);
      opcode : in std_logic_vector(4 downto 0);
      W : out std_logic_vector(15 downto 0)
);
end entity alu;

architecture behavioural of alu is
component bool is port (
A,B : in std_logic_vector(15 downto 0);
f3, f2, f1, f0 : in std_logic;
W : out std_logic_vector(15 downto 0)
);
end component;

component aritm is port (
A : in std_logic_vector(15 downto 0);
B : in std_logic_vector(15 downto 0);
zero, invert, c, m : in std_logic;
Z : out std_logic_vector(15 downto 0)
);
end component;

signal Z1,Z2 : std_logic_vector(W'range);
begin
bool_impl: bool port map(
A => A,
B => B,
f3 => opcode(3),
f2 => opcode(2),
f1 => opcode(1),
f0 => opcode(0),
W => Z1);

aritm_impl: aritm port map(
A => A,
B => B,
m => opcode(3),
invert => opcode(2),
zero => opcode(1),
c => opcode(0),
Z => Z2);
W <= Z1 when opcode(4)='0' else Z2;

end behavioural;
```


Alessandro Arrichiello M63/43
Flavio Battimo M63/38

Come si può notare dal listato VHDL entrambi i moduli sono collegati agli ingressi ma la scelta dell'uscita è effettuata da un multiplexer pilotato dal 4° bit dell'opcode. L'implementazione quindi, multiplexer a parte, è costituita da un semplice port-mapping dei due moduli già sviluppati.

Abbiamo quindi infine creato un testbench a partire da quelli già precedentemente implementati al fine di testare il corretto funzionamento del modulo ALU sviluppato.

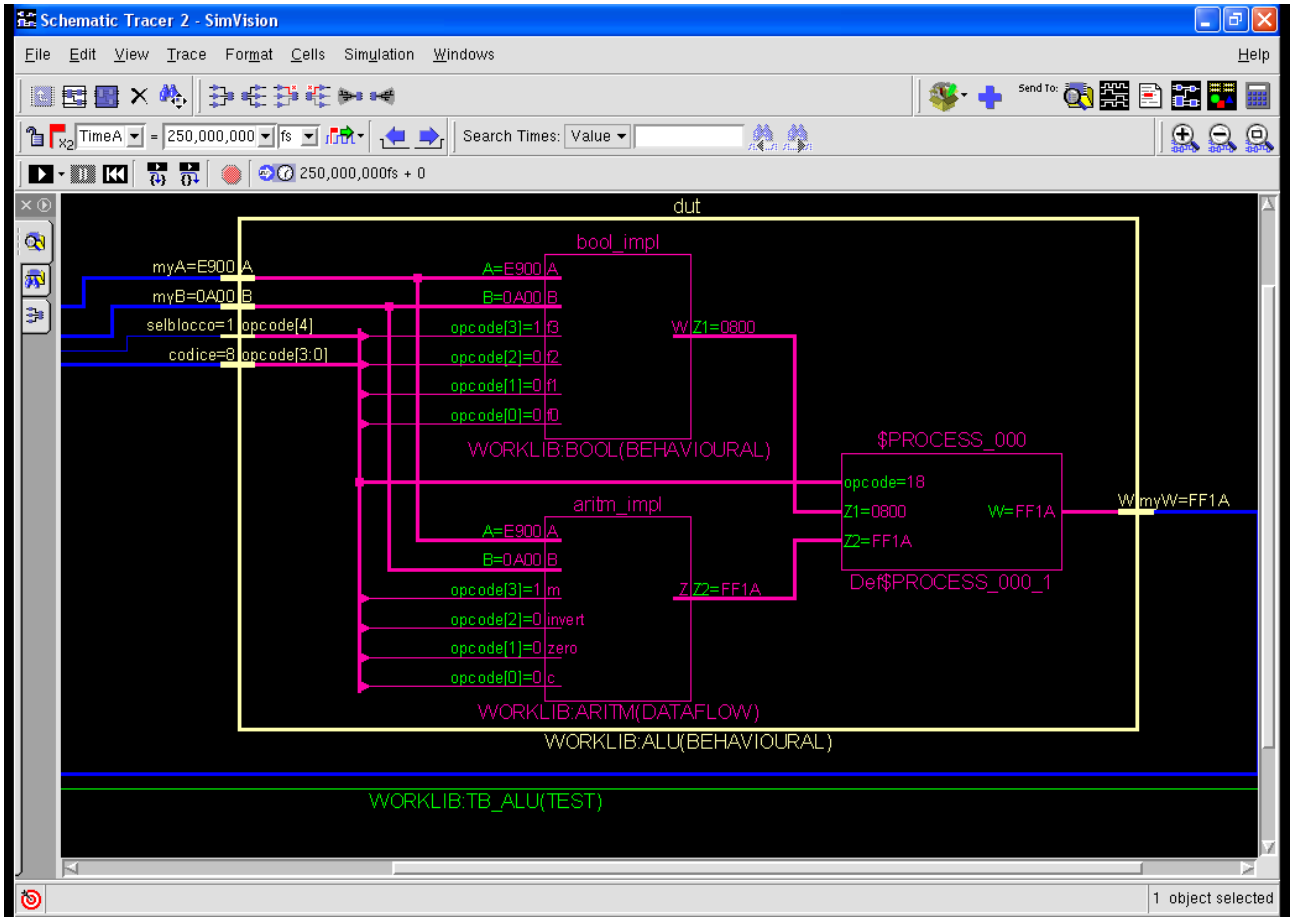
```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

ENTITY tb_alu IS
END tb_alu;

architecture test of tb_alu is
component alu is port(
      A,B : in std_logic_vector(15 downto 0);
      opcode : in std_logic_vector(4 downto 0);
      W : out std_logic_vector(15 downto 0)
);
end component;
signal myA, myB, myW:std_logic_vector(15 downto 0);
signal codice: std_logic_vector (3 downto 0);
signal selblocco: std_logic;

-- e' costituito, nell'ordine, da:
-- m, invert, zero, c
begin
dut: alu port map(
A => myA,
B => myB,
opcode(4) => selblocco,
opcode(3 downto 0) => codice,
W => myW);

stimulus: process
begin

selblocco <= '0';

myA <= B"0000_0011_0000_1111";
myB <= B"1010_1101_1111_0000";


codice <= "0000";
wait for 10 ns;
assert myW=B"0000_0000_0000_0000"
report "errore in W=0"
severity warning;

codice <= "0001";
wait for 10 ns;
assert myW=B"0101_0000_0000_0000"
report "errore in W=A nor B"
severity warning;

codice <= "0011";
wait for 10 ns;
assert myW=B"0101_0010_0000_1111"
report "errore in W=not B"
severity warning;
```

```
codice <= "0101";
wait for 10 ns;
assert myW=B"1111_1100_1111_0000"
report "errore in W=not A"
severity warning;

codice <= "0110";
wait for 10 ns;
assert myW=B"1010_1110_1111_1111"
report "errore in W=A xor B"
severity warning;

codice <= "1010";
wait for 10 ns;
assert myW=B"0000_0011_0000_1111"
report "errore in W=A"
severity warning;

codice <= "1100";
wait for 10 ns;
assert myW=B"1010_1101_1111_0000"
report "errore in W=B"
severity warning;

codice <= "1110";
wait for 10 ns;
assert myW=B"1010_1111_1111_1111"
report "errore in W=A or B"
severity warning;

codice <= "1111";
wait for 10 ns;
assert myW=B"1111_1111_1111_1111"
report "errore in W=1"
severity warning;

wait for 50 ns;


selblocco <= '1';

myA <= std_logic_vector(to_signed(99, 16));
myB <= std_logic_vector(to_signed(43, 16));
codice <= "0000";
wait for 10 ns;
assert myW=std_logic_vector(to_signed(142, 16))
report "errore in Z=A+B"
severity warning;

myA <= std_logic_vector(to_signed(99, 16));
myB <= std_logic_vector(to_signed(43, 16));
codice <= "0010";
wait for 10 ns;
assert myW=std_logic_vector(to_signed(99, 16))
report "errore in Z=A"
severity warning;

myA <= std_logic_vector(to_signed(99, 16));
myB <= std_logic_vector(to_signed(43, 16));
codice <= "0011";
wait for 10 ns;
```

```
assert myW=std_logic_vector(to_signed(100, 16))
report "errore in Z=A+1"
severity warning;

myA <= std_logic_vector(to_signed(99, 16));
myB <= std_logic_vector(to_signed(43, 16));
codice <= "0101";
wait for 10 ns;
assert myW=std_logic_vector(to_signed(56, 16))
report "errore in Z=A-B"
severity warning;

myA <= std_logic_vector(to_signed(99, 16));
myB <= std_logic_vector(to_signed(111, 16));
codice <= "0101";
wait for 10 ns;
assert myW=std_logic_vector(to_signed(-12, 16))
report "errore in Z=A-B"
severity warning;

myA <= std_logic_vector(to_signed(99, 16));
myB <= std_logic_vector(to_signed(111, 16));
codice <= "0110";
wait for 10 ns;
assert myW=std_logic_vector(to_signed(98, 16))
report "errore in Z=A-1"
severity warning;

myA <= std_logic_vector(to_signed(4352, 16));
myB <= std_logic_vector(to_signed(2304, 16));
codice <= "1000";
wait for 10 ns;
assert myW=std_logic_vector(to_signed(153, 16))
report "errore in Z=A*B"
severity warning;

myA <= std_logic_vector(to_signed(4352, 16));
myB <= std_logic_vector(to_signed(2304, 16));
codice <= "1101";
wait for 10 ns;
assert myW=std_logic_vector(to_signed(153, 16))
report "errore in Z=A*B"
severity warning;

myA <= std_logic_vector(to_signed(7549, 16));
myB <= std_logic_vector(to_signed(13773, 16));
codice <= "1000";
wait for 10 ns;
assert myW=std_logic_vector(to_signed(1586, 16))
report "errore in Z=A*B"
severity warning;

myA <= std_logic_vector(to_signed(-5888, 16));
myB <= std_logic_vector(to_signed(2560, 16));
codice <= "1000";
wait for 10 ns;
assert myW=std_logic_vector(to_signed(-230, 16))
report "errore in Z=A*B"
severity warning;

myA <= std_logic_vector(to_signed(-5888, 16));
myB <= std_logic_vector(to_signed(2560, 16));
```

```
codice <= "1000";
wait for 10 ns;
assert myW=std_logic_vector(to_signed(-20, 16))
report "errore in Z=A*B"
severity warning;


wait;
end process;

end test;
```

Infine come suggeritoci dall'esercitazione abbiamo provveduto a testare lo strumento integrato della suite Cadence per la visualizzazione grafica dei componenti sviluppati, esaminando i moduli prodotti graficamente a partire dai nostri listati VHDL.



Alessandro Arrichiello M63/43
Flavio Battimo M63/38

Terza Esercitazione: Sintesi della ALU del processore

Una volta progettato un circuito in VHDL, è necessario sintetizzarlo utilizzando una libreria di celle standard. Le celle standard sono sottocircuiti che realizzano operazioni molto semplici. La fase di sintesi permette di passare da una generica descrizione VHDL del circuito ad una sua descrizione strutturale che utilizzi come componenti le sole celle standard di libreria. La sintetizzazione è un passo necessario poiché così il circuito può facilmente essere implementato, piazzando su silicio le celle necessarie e collegandole tra di loro.

Utilizzeremo per la sintesi una libreria di celle standard in tecnologia CMOS a 45nm con tensione di alimentazione ad 1.1V.

Il software per effettuare la sintesi necessita di un file con le specifiche delle celle di libreria che può utilizzare, uno dei formati più utilizzati è LIBERTY creato da una nota società che sviluppa software CAD di circuiti integrati.

I file in formato LIBERTY sono divisi in due parti: una parte dichiarativa dove vengono fissate le unità di misura, i valori di tensione, temperatura e processo con i quali sono state ottenute le prestazioni delle celle, ed una parte descrittiva contenente la descrizione delle celle standard.

Nella descrizione della cella viene riportata l'occupazione d'area, e la dissipazione di potenza statica. Segue poi la lista dei pin della cella. Per i pin di ingresso sono indicati i valori di capacità di carico ed il massimo tempo di transizione consentito. Per i pin di uscita viene riportata la funzione booleana calcolata dalla cella, la massima capacità di carico che il pin può pilotare e il massimo tempo di transizione consentito sul pin. Inoltre, per ciascun pin di uscita viene riportato il corrispondente ritardo e la dissipazione di potenza dinamica.

Grazie al file Liberty il sintetizzatore, non solo dispone di tutte le informazioni necessarie a mappare un circuito utilizzando la libreria di celle standard, ma può anche ottimizzare il circuito portando in conto l'occupazione d'area, la velocità e la dissipazione di potenza di ciascuna cella.

Sintesi in componenti generici

Abbiamo quindi iniziato questa esercitazione avviando il tool di automatizzazione bgx_shell necessario alla sintetizzazione del modulo precedentemente sviluppato.

Abbiamo caricato i tre file VHDL nel sintetizzatore con i comandi seguenti:

```
read_vhdl hdl/bool.vhd
read_vhdl hdl/aritm.vhd
read_vhdl hdl/alu.vhd
```

Il primo passo per la sintesi consiste nel trasformare il circuito descritto in VHDL in termini di alcuni componenti elementari, definiti "generici", non appartenenti ad un particolare libreria di celle standard: flip-flop, operatori booleani, operatori aritmetici e altre macro di libreria, attraverso il comando:

```
do_build_generic
```

Quindi abbiamo ritrasformato il risultato della sintesi nuovamente in un file VHDL, con il comando:

```
write_vhdl -hier hdl/sintesi1.vhd
```

Aprendo il file “sintesi1.vhd” generato si può notare che sono contenute le descrizioni di entità ed architettura dei vari blocchi sintetizzati. Alcuni segnali sono denominati **UNCONNECTED_000**, **UNCONNECTED_001**, etc. e corrispondono proprio ai 16 segnali del moltiplicatore che non abbiamo collegato, ricordiamo infatti che abbiamo sviluppato un moltiplicatore a 16bit con 32bit di uscita ma di tale uscita ne abbiamo preso solo i 16bit più significativi.

Sintesi con celle standard

Terminata la sintesi in componenti generici dobbiamo caricare nel sintetizzatore la libreria di celle standard. A tal fine utilizzeremo un semplice script automatizzato:

```
# config.tcl
# flusso compatibile con Synopsys
set_flow_compatible_mode on

puts "Reading Library ..."
set                    nangate_dir                    /home/caduser/NCSU-FreePDK45-
1.3/nangate/NangateOpenCellLibrary_PDKv1_3_v2009_07/liberty
read_dotlib   -min   $nangate_dir/NangateOpenCellLibrary_fast_conditional_ccs.lib   -max
$nangate_dir/NangateOpenCellLibrary_typical_conditional_ccs.lib

#Impostazione analisi Timing
set_global pvt_early_path min
set_global pvt_late_path max
set_global timing_analysis_type bc_wc

set_cell_property dont_utilize true [find -cell SDFF_*]
```

Lo script legge due file liberty contenenti la descrizione delle celle standard negli angoli di processo **fast** e **tipico**.

Le ultime istruzioni servono per definire il modo in cui il sintetizzatore deve effettuare l’analisi della velocità del circuito. In particolare, il sintetizzatore è istruito ad utilizzare l’angolo di processo fast per caratterizzare i ritardi del circuito quando siamo interessati a verificare violazioni nel tempo di hold, mentre utilizza l’angolo di processo tipico per determinare la massima frequenza di funzionamento del circuito.

Abbiamo quindi lanciato tale script proposto per la configurazione con il comando:

```
source script/config.tcl
```

Come suggeritoci abbiamo lanciato l’ottimizzazione del circuito con il comando:

```
do_optimize -effort high
```

Lanciando quindi il comando: “**`report_area -hier -cells`**” abbiamo verificato che il blocco che occupa più area è proprio il **moltiplicatore** nel modulo aritmetico mentre le celle standard più utilizzate sono “**INV_X4**” invertitori a 4bit.

Timing Analysis

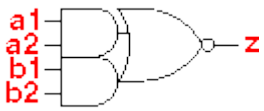
Passiamo ora all’ottimizzazione dei tempi attraverso un’analisi dei tempi con e senza vincoli.

Per ottenere il ritardo massimo abbiamo usato il comando:

```
report_timing -late -unconstrained
```

Abbiamo quindi provveduto ad analizzare l'output di tale comando al fine di verificare dove passa il cammino corrispondente al ritardo massimo e quali celle sono coinvolte.

Dopo una rapida analisi si è potuto constatare che il cammino critico passa per il blocco aritmetico e in particolare per il moltiplicatore, proprio come ci aspettavamo, le celle coinvolte sono per lo più blocchi di porte **2×2-AND into 2-NOR** usate nel blocco moltiplicatore.



Sintesi con vincoli temporali e scripts di sintesi

E' necessario a questo punto definire dei vincoli temporali per indicare la velocità desiderata per il circuito sviluppato, a parità di funzionalità, infatti, il circuito può essere implementato con celle più o meno veloci. Realizzare un circuito più veloce del necessario richiede l'utilizzo di celle più grandi le quali comportano una maggiore dissipazione di potenza.

Pur essendo il nostro sistema puramente combinatorio, il sintetizzatore richiede comunque la definizione di un riferimento temporale (clock) i cui fronti sono necessari per definire i tempi di arrivo degli ingressi ed i tempi richiesti per le uscite.

A tale scopo abbiamo utilizzato come suggerito lo script di automazione "script/constraints.tcl":

```
# constraints.tcl

# Rimuovo eventuali vincoli precedenti
remove_assertions [find -ports]

# Definisco un clock ideale, di periodo ..
create_clock -period $periodo -name my_clk

# Definisco il ritardo della logica di ingresso e di uscita
set_input_delay  -clock my_clk -max 0.100 [all_inputs]
set_output_delay -clock my_clk -max 0.150 [all_outputs]

# Tempo di salita dei segnali di ingresso: 50ps
set_input_transition 0.050 [all_inputs]

# Carico in uscita: 10fF
set_load 0.010 [all_outputs]
```

Con il comando **create_clock** si definisce un segnale di riferimento, di periodo 10ns, denominato "**my_clk**". Il comando successivo, **set_input_delay**, definisce il ritardo (rispetto al fronte del clock) con cui si presentano i segnali di ingresso. Il comando **set_output_delay** definisce invece un ritardo aggiuntivo fra l'uscita del sistema combinatorio ed il nodo in cui verificare la temporizzazione.

Le ultime due linee del file sono inserite al fine di modellare accuratamente ritardo del sistema combinatorio. Il comando **set_input_transition** definisce il tempo di salita dei segnali di ingresso, mentre il comando **set_load** stabilisce la capacità di carico presente sulle uscite.

Abbiamo poi eseguito tale script nel sintetizzatore al fine di applicare tali vincoli e ripetere la timing analysis:

```
source script/constraints.tcl
report_timing -late
```

Ecco di seguito un estratto della timing analysis effettuata:

```
Other End Arrival Time          0.00
- External Delay                0.15
+ Phase Shift                  10.00
= Required Time                 9.85
- Arrival Time                  2.82
= Slack Time                    7.03
```

Provando a modificare il parametro **set_input_transition** (0,150), triplicandolo ed eseguendo nuovamente la timing analysis dopo aver applicato i vincoli, si può facilmente verificare che il ritardo che si modifica è l'ArrivalTime come mostrato di seguito:

```
Other End Arrival Time          0.00
- External Delay                0.15
+ Phase Shift                  10.00
= Required Time                 9.85
- Arrival Time                  2.86
= Slack Time                    6.99
```

Dopo aver riportato il parametro al suo valore originario, modificando il **set_load** (0,030)ed eseguendo nuovamente la timing analysis dopo aver applicato i vincoli, in questo caso si può notare che anche in questo caso cresce l'ArrivalTime come mostrato di seguito:

```
Other End Arrival Time          0.00
- External Delay                0.15
+ Phase Shift                  10.00
= Required Time                 9.85
- Arrival Time                  2.96
= Slack Time                    6.89
```

Dopo aver riportato i parametri ai valori originari, abbiamo modificato lo script dei vincoli riducendo il periodo a 2.5ns. Dalla timing analysis come si può notare si ottiene uno slack negativo: i vincoli di timing non sono più verificati.

```
Other End Arrival Time          0.00
- External Delay                0.15
+ Phase Shift                   2.50
= Required Time                 2.35
- Arrival Time                  2.82
= Slack Time                   -0.47
```

Chiediamo quindi al sintetizzatore di ottimizzare nuovamente il circuito, in modo da soddisfare il vincolo sul ritardo massimo, minimizzando al contempo l'area:

```
do optimize -reclaim maximum area
```

Al fine di ripetere più volte la sintesi, variando il massimo ritardo richiesto al circuito, abbiamo effettuato la modifica del file **constraints.tcl** aggiungendo la variabile **$periodo**, come mostrato di seguito:

```
# constraints.tcl
...
create_clock -period $periodo -name my_clk
...
```

Inoltre abbiamo provveduto alla modifica del file **flow.tcl** aggiungendo un ciclo for che automatizzasse la generazione dei test al variare del periodo, come mostrato di seguito:

```
# Leggo la libreria di celle standard
source /home/grup12/ese03/script/config.tcl
# Leggo i file vhdl
read_vhdl /home/grup12/ese03/hdl/bool.vhd
read_vhdl /home/grup12/ese03/hdl/aritm.vhd
read_vhdl /home/grup12/ese03/hdl/alu.vhd
# Sintesi generica
do_build_generic
# Definisco il periodo di funzionamento

foreach p {5.0 2.5 2.2 2.0 1.8 1.6} {
    set periodo $p
      # Definisco i vincoli per la sintesi
      source /home/grup12/ese03/script/constraints.tcl
      # Ottimizzazione e reports dei risultati ottenuti
      do_optimize -reclaim_maximum_area
      report_resources -hier
      report_timing -late
      report_area
      # Salvataggio su file dei reports
      report_resources -hier > /home/grup12/ese03/reports/resources$periodo.txt
      report_timing -late > /home/grup12/ese03/reports/timing_late$periodo.txt
      report_area > /home/grup12/ese03/reports/area$periodo.txt
}
```

L'abbiamo quindi eseguito con il comando:

```
bgx_shell -f script/flow.tcl
```

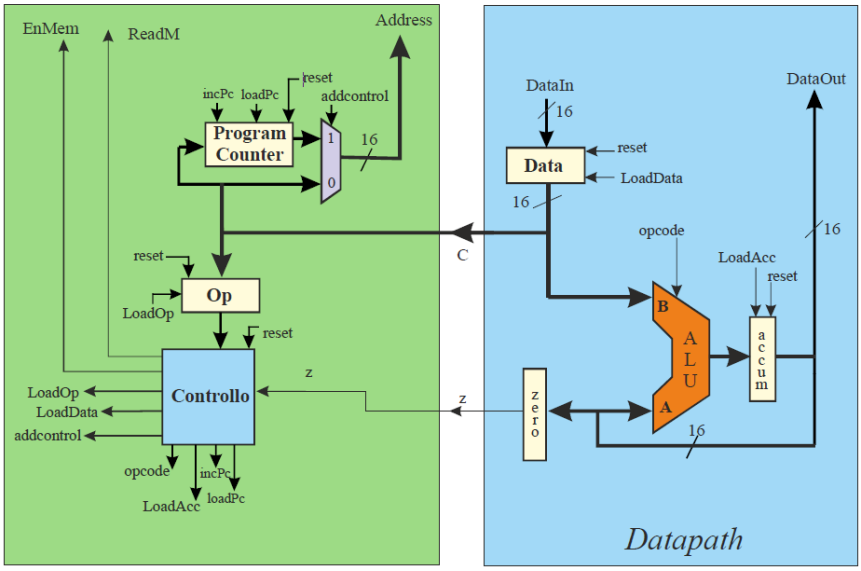
Ed abbiamo analizzato I risultati generati nei file di log nella cartella reports.

In particolare abbiamo potuto analizzare come cambiano le prestazioni del circuito al variare del constraint sulla velocità, per facilitare la lettura abbiamo generato questa tabella riepilogativa:

Variabile periodo [ns]	Slack [ns]	Minimo periodo di funzionamento	Area [um^2]	Architettura moltiplicatore ed addizionatore
5	2,03	2,97	1348,35	ripple/booth - ripple
2,5	0,04	2,46	1421,77	ripple/booth - ripple
2,2	0,01	2,19	1384,80	cla/booth - ripple
2	0,01	1,99	1385,06	cla/booth - ripple
1,8	0	1,8	1579,77	fcla/booth - ripple
1,6	0	1,6	1644,15	fcla/booth - ripple

Quarta Esercitazione: Sintesi del datapath del processore

In questa esercitazione abbiamo provveduto alla creazione e sintesi del datapath del nostro microprocessore andando a riprodurre la struttura proposta:



Il datapath

Il datapath, oggetto di questa esercitazione, include la ALU sviluppata in precedenza, il registro che contiene il dato in ingresso dalla memoria, l'accumulatore che immagazzina il risultato prodotto dall'ALU ed un semplice blocco combinatorio che fornisce un'uscita alta qualora il valore dell'accumulatore sia zero.

Abbiamo quindi provveduto alla scrittura del file VHDL che implementasse tale datapath, come si può notare di seguito:

```
-- datapath.vhd
library ieee;
use ieee.std_logic_1164.all;

entity datapath is
        port(
                DataIn : in std_logic_vector(15 downto 0);
                opcode : in std_logic_vector(4 downto 0);
                reset, LoadData, LoadAcc, clk : in std_logic;
                z      : out std_logic;
                C, DataOut : out std_logic_vector(15 downto 0)
        );
end entity datapath;

architecture behavioural of datapath is

component alu is port(
       A,B : in std_logic_vector(15 downto 0);
       opcode : in std_logic_vector(4 downto 0);
       W : out std_logic_vector(15 downto 0)
);
end component;
```


Alessandro Arrichiello M63/43
Flavio Battimo M63/38

```vhdl
signal Data,Accum,A,B,W ,zeros: std_logic_vector(DataOut'range);

begin
zeros<=(others => '0');
myalu: alu port map(A=>A,B=>B,opcode=>opcode,W=>W);

dataBuffer: process(clk,reset)
begin
	if (clk='1' and clk'event) then
		if (reset = '1') then
			Data <= zeros;
		elsif (LoadData = '1') then
			Data <= DataIn;
		end if;
	end if;

end process;

accumBuffer: process(clk,reset)
begin
	if (clk='1' and clk'event) then
		if (reset = '1') then
			Accum <= zeros;
		elsif (LoadAcc = '1') then
			Accum <= W;
		end if;
	end if;

end process;

DataOut <= Accum;
A <= Accum;
z <= '1' when A = zeros else '0';
C <= Data;
B <= Data;

end behavioural;
```

Per semplicità dopo aver mappato l'alu sviluppata in precedenza abbiamo provveduto alla definizione dei tre moduli: **Data**, **Accumulatore**, **Zero**.

Il modulo **Zero** è stato implementato semplicemente con il costrutto **WHEN-ELSE** all'interno dello stesso file VHDL di definizione del **datapath**.

I due moduli **Accumulatore** e **Data** sono stati anch'essi sviluppati all'interno dello stesso listato VHDL di definizione del **datapath** per ragioni di semplicità attraverso l'uso del costrutto **PROCESS**.

E' bene notare che i ports **DataOut** e **C** sono definiti di modo **out**, è stato quindi necessario introdurre dei segnali ausiliari, visto che i valori su questi port devono essere letti all'interno dell'architettura.

Come suggeritoci nell'esercitazione per i registri abbiamo utilizzato un **reset sincrono**. Il segnale **LoadData**, quando alto, permette di aggiornare al fronte del **clock** il valore del registro **data**. Se **LoadData** è zero, il valore del registro **data** non verrà modificato. Il registro **accum** opera allo stesso modo.

Dopo aver terminato la definizione del listato VHDL per il datapath, abbiamo provveduto a testare l'unità prodotto attraverso il file di testbench proposto nell'esercitazione, verificando l'assenza di errori:

```vhdl
-- datapath.vhd
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
```

```
entity tb_datapath is
end entity tb_datapath;

architecture test of tb_datapath is

component datapath is
        port(
                DataIn : in std_logic_vector(15 downto 0);
                opcode : in std_logic_vector(4 downto 0);
                reset, LoadData, LoadAcc, clk : in std_logic;
                z      : out std_logic;
                C, DataOut : out std_logic_vector(15 downto 0)
        );
end component;

signal mDataIn, mC, mDataOut : std_logic_vector(15 downto 0);
signal mopcode : std_logic_vector(4 downto 0);
signal mreset, mLoadData, mLoadAcc, mz : std_logic;
signal mclk : std_logic := '0';

constant period : time := 10 ns;
constant maxSimTime : time := 25*period;

begin

dut : datapath port map(
DataIn => mDataIn,
opcode => mopcode,
reset => mreset,
LoadData => mLoadData,
LoadAcc => mLoadAcc,
clk => mclk,
z => mz,
C => mC,
DataOut => mDataOut);

clocking : process
begin
 wait for period/2;
 mclk <= not mclk;
 if now > maxSimTime then wait ; -- forever
 end if;
end process clocking;

stimoli : process
begin
 mreset <= '1';
 mLoadData <= '0';
 mLoadAcc <= '0';
 mopcode <= (others => '0');
 mDataIn <= (others => '0');
 wait for period;

 -- reset effettuato; carico il registro Data
 mreset <= '0';
 mLoadData <= '1';
 mDataIn <= std_logic_vector ( to_signed(9,16) );
 wait for period;

 -- attivo il registro accum e definisco il codice operativo
 -- per trasferire C nell'accumulatore
 mLoadData <= '0';
 mLoadAcc <= '1';
 mopCode <= "01100";
 wait for period;

 -- definisco il codice operativo per il decremento
 mopCode <= "10110";
```


Alessandro Arrichiello M63/43
Flavio Battimo M63/38

```
 wait for 12*period;

-- carico il registro Data con un nuovo valore
 mLoadData <= '1';
 mLoadAcc <=  '0';
 mDataIn <= std_logic_vector ( to_signed(7549,16) );
 wait for period;

-- trasferisco il valore nell'accumulatore
-- contemporaneamente inserisco un nuovo valore nel registro data
 mLoadAcc <=  '1';
 mLoadData <= '1';
 mopCode <= "01100";
 mDataIn <= std_logic_vector ( to_signed(13773,16) );
 wait for period;

-- blocco il registro data ed effettuo la moltiplicazione
 mLoadData <= '0';
 mopCode <= "11000";
-- il primo risultato atteso è: 1586 in decimale
 wait; -- forever

end process stimoli;

end architecture test;
```

Sintesi

Per completare l'esercitazione abbiamo quindi realizzato la sintesi dell'intero blocco datapath.

A differenza della ALU, il circuito datapath è sequenziale ed ha un pin di **clock**. Questa informazione va passata al sintetizzatore nel file **constraint.tcl**:

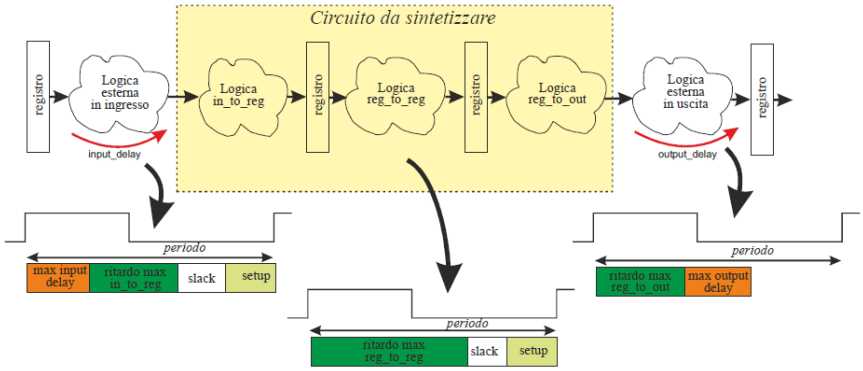
```
# constraints.tcl

# Rimuovo eventuali vincoli precedenti
remove_assertions [find -ports]

# Definisco il clock:
create_clock clk -period $periodo -name my_clk
#Tempi di salita e discesa del clock
set_clock_transition 0.05 my_clk

# Incertezza sui tempi di arrivo dei fronti del clock 100ps
set_clock_uncertainty 0.1 my_clk

# Definisco il ritardo della logica di ingresso e di uscita
set_input_delay  -clock my_clk -max 0.250 [all_inputs]
set_output_delay -clock my_clk -max 0.250 [all_outputs]

#Ritardo della logica di ingresso e di uscita,
#per il controllo sul tempo di hold:
set_input_delay -clock my_clk -min 0.2 [all_inputs]
set_output_delay -clock my_clk -min 0.2 [all_outputs]

# Tempo di salita dei segnali di ingresso: 50ps
set_input_transition 0.050 [all_inputs]

# Carico in uscita: 10fF
set_load 0.010 [all_outputs]
```

Nel comando "create_clock" si fa riferimento al segnale clk (il nome che abbiamo dato al pin di clock del datapath). Inoltre è stato aggiunto il comando "set_input_delay" con l'opzione "–min". In questo modo

verranno considerati due ritardi diversi per la logica in ingresso nella verifica dei tempi di setup (250ps) e di hold (200ps).

Modello di timing per il vincolo sul tempo di setup

Il modello di timing considerato dal sintetizzatore (per la valutazione del ritardo massimo) è riportato nella figura seguente:



Nel nostro caso, non c'è alcuna logica **in_to_reg**, per cui il valore dell'input delay non è critico. La logica **reg_to_out** si limita al circuito che verifica quando l'accumulatore è zero ed è quindi anch'essa non critica. Possiamo in definitiva aspettarci che il limite per il periodo del clock sia dettato dalla logica **reg_to_reg**.

Abbiamo quindi modificato lo script **flow.tcl** in modo da caricare tutti i file vhdl necessari alla sintesi, e successivamente effettuato la sintesi del circuito, dopo aver posto il periodo del clock a 2.5ns.

```
# Leggo la libreria di celle standard
source /home/grup12/ese04/script/config.tcl
# Leggo i file vhdl
read_vhdl /home/grup12/ese04/hdl/bool.vhd
read_vhdl /home/grup12/ese04/hdl/aritm.vhd
read_vhdl /home/grup12/ese04/hdl/alu.vhd
read_vhdl /home/grup12/ese04/hdl/datapath.vhd
# Sintesi generica
do_build_generic
# Definisco il periodo di funzionamento

set periodo 2.5
# Definisco i vincoli per la sintesi
source /home/grup12/ese04/script/constraints.tcl
# Ottimizzazione e reports dei risultati ottenuti
do_optimize -reclaim_maximum_area
do_fix_hold -buffer
#report_resources -hier
#report_timing -late
#report_area
# Salvataggio su file dei reports
report_resources -hier > /home/grup12/ese04/reports/resources${periodo}_uncert.txt
report_timing -late > /home/grup12/ese04/reports/timing_late${periodo}_uncert.txt
report_timing -early > /home/grup12/ese04/reports/timing_early${periodo}_uncert.txt
report_area > /home/grup12/ese04/reports/area${periodo}_uncert.txt
source /home/grup12/ese04/script/write_output.tcl
```

Dopo la fase di elaborazione (comando **do_build_generic**) adesso compare quello relativo alla presenza di flip-flop nel circuito.

Come si può notare analizzando i ritardi con il comando:

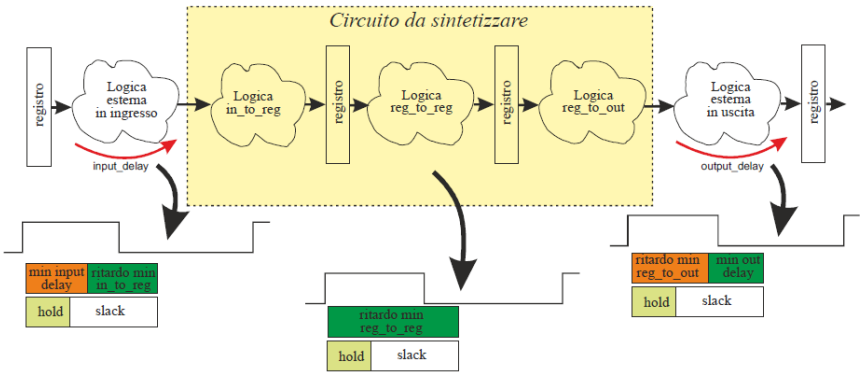
```
"report_timing -late"
```

Abbiamo notato che il cammino critico appunto parte dal registro Data_reg e termina nel registro Accum_reg come si può notare di seguito:

```
      +-------------------------------------------------------------------------------------+
      |          Instance            |     Arc       |    Cell      | Delay | Arrival | Required |
      |                              |               |              |       |  Time   |   Time   |
      |------------------------------+---------------+--------------+-------+---------+----------|
      |                              | clk ^         |              |       |    0.00 |     0.02 |
      | Data_reg_11                  | CK ^ -> Q v   | DFF_X1       |  0.31 |    0.31 |     0.33 |
      | myalu                        | B[11] v       | alu          |       |    0.31 |     0.33 |
      | myalu/aritm_impl             | B[11] v       | aritm        |       |    0.31 |     0.33 |
      | myalu/aritm_impl/i_4         | bs[11] v      | AWDP_MULT_3  |       |    0.31 |     0.33 |
      | myalu/aritm_impl/i_4/i_961   | A v -> ZN ^   | XNOR2_X2     |  0.31 |    0.62 |     0.64 |

…

      | myalu/aritm_impl/i_27        | B v -> Z v    | MUX2_X1      |  0.12 |    2.20 |     2.21 |
      | myalu/aritm_impl             | Z[11] v       | aritm        |       |    2.20 |     2.21 |
      | myalu/i_11                   | B v -> Z v    | MUX2_X1      |  0.11 |    2.31 |     2.33 |
      | myalu                        | W[11] v       | alu          |       |    2.31 |     2.33 |
      | i_21                         | A1 v -> ZN v  | AND2_X1      |  0.05 |    2.36 |     2.38 |
      | i_435                        | A v -> Z v    | MUX2_X1      |  0.09 |    2.45 |     2.47 |
      | Accum_reg_11                 | D v           | DFF_X1       |  0.00 |    2.45 |     2.47 |
      +-------------------------------------------------------------------------------------+
```

Modello di timing per il vincolo sul tempo di hold

La figura seguente riporta il modello di timing, per la valutazione del ritardo minimo:



Abbiamo quindi controllato il vincolo sul tempo di **hold**, digitando il comando "**report_timing –early**".

Come mostrato di seguito il vincolo sul tempo di hold è stato correttamente rispettato e non abbiamo avuto bisogno di utilizzare celle con dispositivi più piccoli (e quindi più lente) oppure introdurre dei buffer fittizi che aumentano il ritardo minimo del sistema, (comandi: "do_fix_hold –resize" e "do_fix_hold –buffer").

```
Other End Arrival Time          0.00
+ Hold                          0.03
+ Phase Shift                   0.00
= Required Time                 0.03
  Arrival Time                  0.19
  Slack Time                    0.17
```


Alessandro Arrichiello M63/43
Flavio Battimo M63/38

Effetto delle non-idealità del clock

Come suggeritoci dall'esercitazione abbiamo quindi tenuto in conto anche delle eventuali non-idealità del clock andando a modificare opportunamente il file **constraints.tcl**:

```
# Incertezza sui tempi di arrivo dei fronti del clock: 100ps
set_clock_uncertainty 0.1 my_clk
```

Abbiamo quindi ripetuto entrambe le analisi dei tempi, ed ecco di seguito I risultati a confronto:

Vincolo sul tempo di setup		Vincolo sul tempo di setup (**UNCERTAIN**)	
Other End Arrival Time	0.00	Other End Arrival Time	0.00
- Setup	0.03	- Setup	0.03
+ Phase Shift	2.50	+ Phase Shift	2.50
		- Uncertainty	0.10
= Required Time	2.47	= Required Time	2.37
- Arrival Time	2.45	- Arrival Time	2.37
= Slack Time	0.02	= Slack Time	0.00

Vincolo sul tempo di hold		Vincolo sul tempo di hold (**UNCERTAIN**)	
Other End Arrival Time	0.00	Other End Arrival Time	0.00
+ Hold	0.03	+ Hold	0.03
+ Phase Shift	0.00	+ Phase Shift	0.00
		+ Uncertainty	0.10
= Required Time	0.03	= Required Time	0.13
Arrival Time	0.19	Arrival Time	0.16
Slack Time	0.17	Slack Time	0.03

Come si può notare inserendo un'incertezza di soli 100ps lo slack si attesta a zero nel modello di timing per il vincolo sul tempo di setup, mentre scende di ben 140ps (0,14ns) nel modello di timing per il vincolo sul tempo di hold.

Abbiamo quindi aggiunto allo script **flow.tcl** lo script **write_output.tcl**. Esso serve per preparare i file necessari per la successiva fase di **place & route** del circuito. Lo script inoltre crea due file nella cartella **sim**: il file **circuit.vhd** che rappresenta una **netlist** vhdl del circuito sintetizzato ed un file **circuit.sdf** che contiene le informazioni necessarie per annotare i ritardi del circuito sintetizzato.

Simulazione post-layout con annotazione dei ritardi

Andando ad analizzare il file **circuit.sdf** appena generato:

Le righe IOPATH riportano i ritardi (tempi di salita e tempi di discesa) per i due archi temporali: ingresso A1 => uscita ZN ed A2 => uscita ZN.

Si hanno due valori per ognuno dei tempi, che corrispondono ai due casi “fast” e “slow” per i quali è stata richiesta la valutazione dei ritardi; il caso “typical” non è stato considerato.

Come si può notare nella tabella comparativa seguente componenti dello stesso tipo hanno ritardi diversi e quindi sono specifici di ogni istanza.

(CELL (CELLTYPE "NOR2_X1") (INSTANCE myalu/aritm_impl/i_4/i_967) (DELAY (ABSOLUTE (IOPATH A1 ZN (0.0498: :0.1069) (0.0298: :0.0520)) (IOPATH A2 ZN (0.0486: :0.0945) (0.0321: :0.0503)))))	CELL (CELLTYPE "NOR2_X1") (INSTANCE i_32) (DELAY (ABSOLUTE (IOPATH A1 ZN (0.1542: :0.2857) (0.0921: :0.1160)) (IOPATH A2 ZN (0.1565: :0.2953) (0.0955: :0.1201)))))

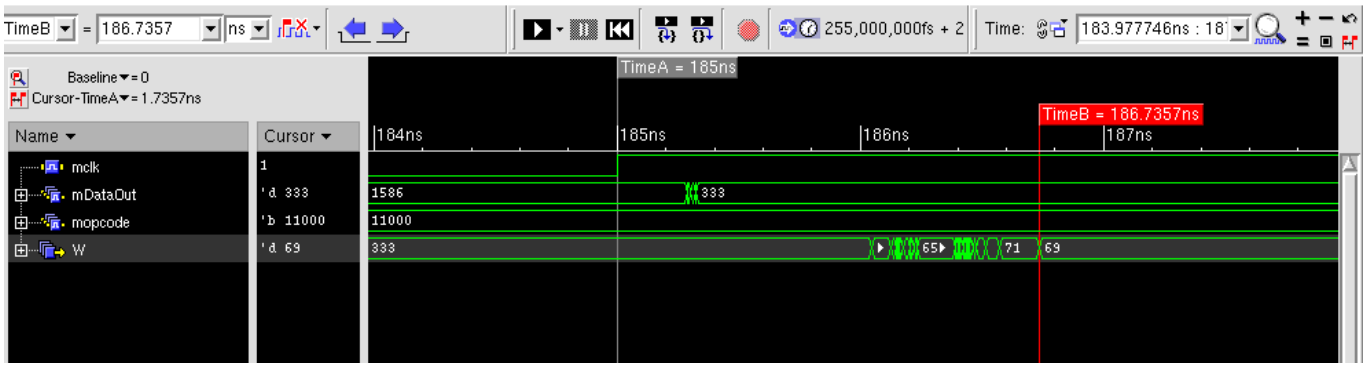
Successivamente abbiamo effettuato la simulazione con l’annotazione dei ritardi attraverso **nclaunch**.

Abbiamo compilato in **nclaunch** i seguenti listati VHDL e relative librerie:

- NangateOpenCellLibrary_typical_conditional_mod.v
- circuit.vhd
- tb_datapath.vhd
- circuit.sfd

Abbiamo quindi generato lo snapshot eseguibile richiamando uno script digitando il seguente comando nells finestra di **nclaunc**: **input link.cmd**

Dopo aver verificato che la fase di annotazione sia completata senza errori abbiamo effettuato la simulazione del circuito. Come si può notare di seguito abbiamo individuato i segnali che rappresentano l’uscita della ALU e ne abbiamo salvato un screenshot per visualizzarne l’andamento:



Come si può verificare il ritardo tra il fronte di salita del clock e l’effettiva uscita dall’alu è di circa: 1,73ns.

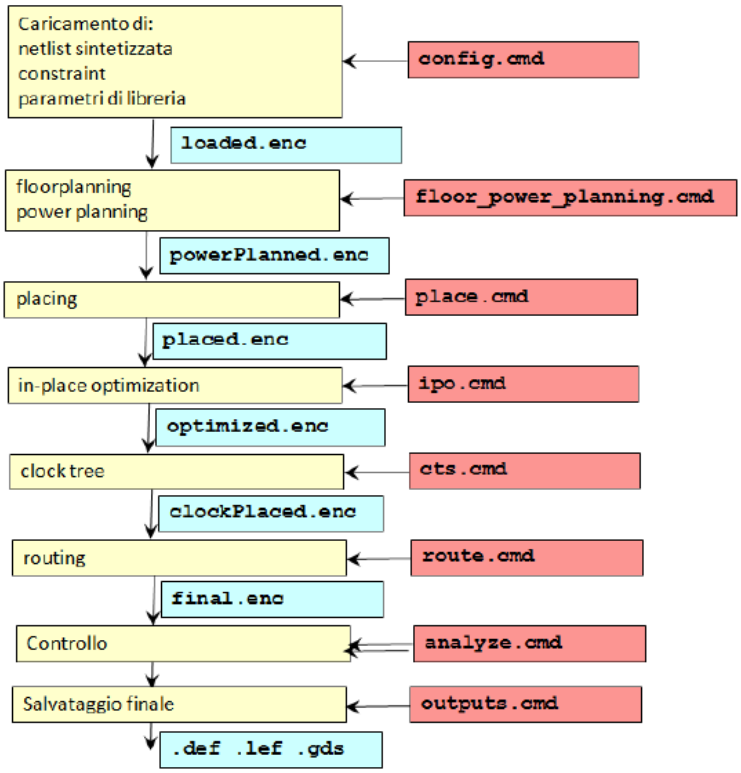
Alessandro Arrichiello M63/43
Flavio Battimo M63/38

Quinta Esercitazione: Placement e Routing del datapath

Il datapath di pico è stato sintetizzato in una netlist contenente una descrizione strutturale del circuito. Per completare il progetto bisogna piazzare le celle che compongono la netlist del circuito sul silicio e collegarle tra di loro attraverso il tool SoC Encounter della Cadence. Il routing prevede il collegamento di moltissimi pin e non è praticamente realizzabile senza l'ausilio di un software per il CAD.

Oltre al piazzamento delle celle su silicio ed al loro collegamento, la fase di P&R di un circuito integrato permette di effettuare due operazioni fondamentali: la realizzazione dell'albero del clock (che consente di inviare il segnale di clock a tutti i registri con uno skew controllato) e l'ottimizzazione del circuito tenendo in conto gli effetti parassiti dovuti ai collegamenti tra le celle (in-place optimization). Durante l'ottimizzazione, le celle del path critico possono essere sostituite con altre che implementano la stessa funzione ma sono realizzate con transistor più grandi. Inoltre durante la fase di ottimizzazione è possibile bufferizzare celle aventi un elevato fanout con l'ausilio di invertitori e buffer.

Ecco di seguito una breve schematizzazione delle fasi che abbiamo attraversato in questa esercitazione:



Alessandro Arrichiello M63/43
Flavio Battimo M63/38

Configurazione

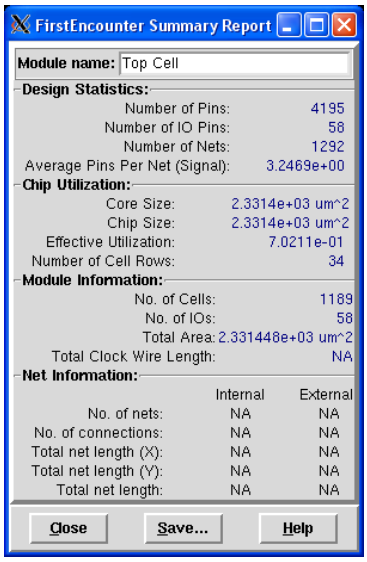
Per effettuare il P&R abbiamo utilizzato i due file prodotti in fase di sintesi: la netlist verilog del circuito (final_synt.v) ed il file dei constraints (synthesized_const.sdc). Inoltre abbiamo definito nel file denominato "pin_locations.io" la posizione dei pin di ingresso/uscita lungo la periferia del blocco, come mostrato brevemente di seguito:

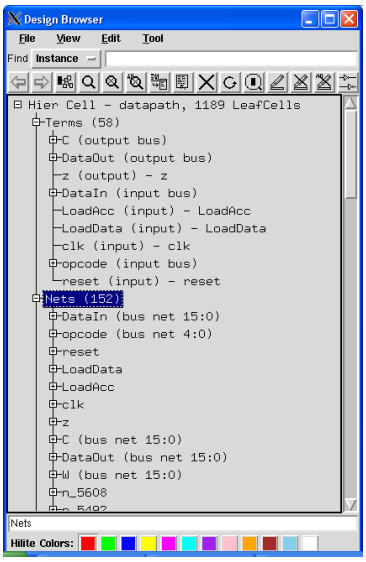
Pin: DataOut[15] S 3	Pin: DataIn[15] W 4	Pin: C[15] W 4	Pin: opcode[4] N 3
Pin: DataOut[14] S 3	Pin: DataIn[14] W 4	Pin: C[14] W 4	Pin: opcode[3] N 3
Pin: DataOut[13] S 3	Pin: DataIn[13] W 4	Pin: C[13] W 4	Pin: opcode[2] N 3
Pin: DataOut[12] S 3	Pin: DataIn[12] W 4	Pin: C[12] W 4	Pin: opcode[1] N 3
Pin: DataOut[11] S 3	Pin: DataIn[11] W 4	Pin: C[11] W 4	Pin: opcode[0] N 3
Pin: DataOut[10] S 3	Pin: DataIn[10] W 4	Pin: C[10] W 4	
Pin: DataOut[9] S 3	Pin: DataIn[9] W 4	Pin: C[9] W 4	Pin: reset N 3
Pin: DataOut[8] S 3	Pin: DataIn[8] W 4	Pin: C[8] W 4	Pin: LoadData N 3
Pin: DataOut[7] S 3			Pin: LoadAcc N 3
Pin: DataOut[6] S 3	Pin: DataIn[7] E 4	Pin: C[7] E 4	Pin: clk N 3
Pin: DataOut[5] S 3	Pin: DataIn[6] E 4	Pin: C[6] E 4	Pin: z N 3
Pin: DataOut[4] S 3	Pin: DataIn[5] E 4	Pin: C[5] E 4	
Pin: DataOut[3] S 3	Pin: DataIn[4] E 4	Pin: C[4] E 4	
Pin: DataOut[2] S 3	Pin: DataIn[3] E 4	Pin: C[3] E 4	
Pin: DataOut[1] S 3	Pin: DataIn[2] E 4	Pin: C[2] E 4	
Pin: DataOut[0] S 3	Pin: DataIn[1] E 4	Pin: C[1] E 4	
	Pin: DataIn[0] E 4	Pin: C[0] E 4	

Come si può notare abbiamo mappato il bus DataOut sul lato Sud in metal 3, il bus DataIn metà sul lato Ovest e metà sul lato Est in metal 4, il bus C metà sul lato Ovest e metà sul lato Est in metal 4, infine l'opcode il reset e gli altri pin sul lato nord in metal 3.

Abbiamo quindi avviato il tool di place&route SoC Encounter, dapprima abbiamo caricato il file di configurazione proposto nell'esercitazione: fp45_nangate_gui_2006.conf, contenente il layout delle celle standard in formato LEF (Library Exchange Format), tale file rappresenta una descrizione ridotta del layout delle celle standard, in cui viene riportato solamente il posizionamento dei metalli all'interno di ciascuna cella standard.

Si può visualizzare il "summary report" cliccando sul terzultimo widget, tale pulsante fornisce indicazioni sul numero di pin, di net, sulle dimensioni del circuito ecc, mentre Il "design browser" (sesto da destra) consente di vedere la gerarchia del progetto; espandendo la voce "terms" si può verificare il posizionamento dei pin di ingresso-uscita.





Dopo aver eseguito il primo comando:

```
source scripts/config.cmd
```

In questa prima fase lo script si limita a fissare il nome del progetto, il nome dei terminali di alimentazione e gli identificativi che permettono al tool di riconoscere i buffer e gli invertitori all'interno della libreria. Inoltre viene anche fatta uno stima del timing e dello slack:

```
|Setup mode |   all    | reg2reg | in2reg   | reg2out | in2out   | clkgate |
+-----------+----------+---------+----------+---------+----------+---------+
|  WNS (ns):|  0.268   |  0.268  |  0.513   |  1.575  |   N/A    |   N/A   |
```

Lo slack ottenuto è migliore rispetto a quanto previsto in fase di sintesi; il programma di P&R infatti non adotta alcun wireload model ed, al momento, non include nessuna stima per il ritardo delle interconnessioni.

Floorplan

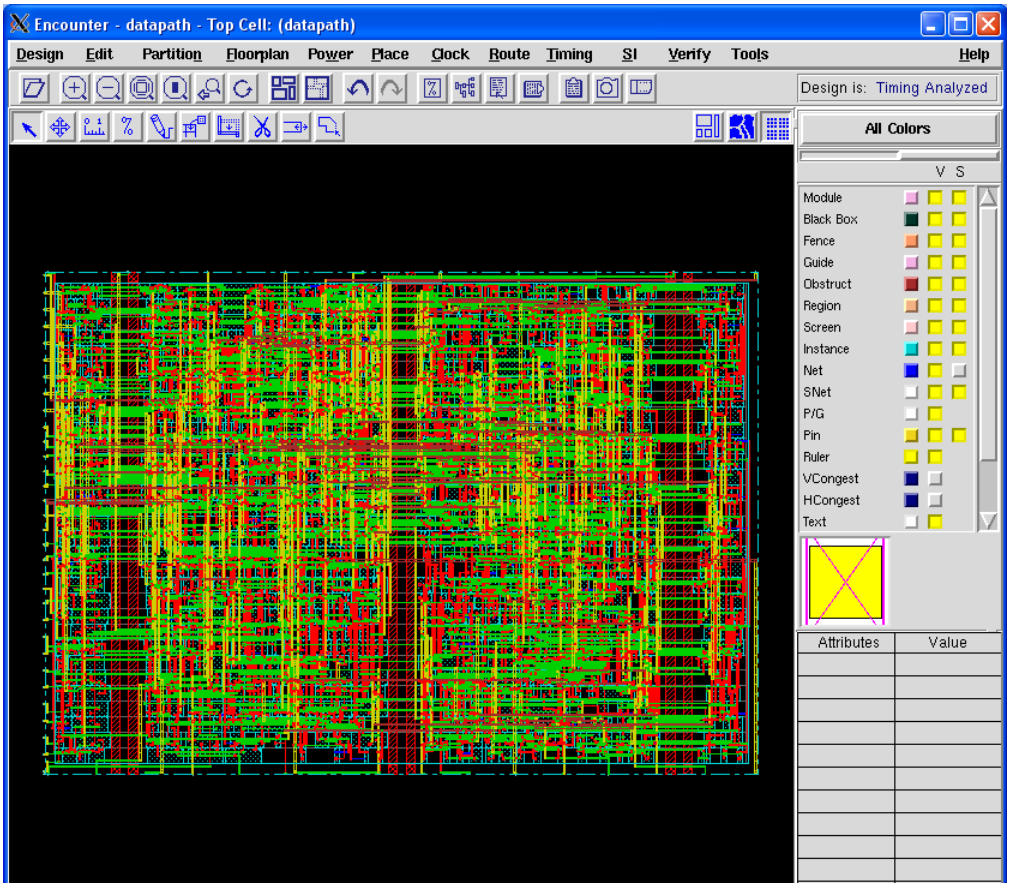
Il prossimo passo da eseguire consiste nel fissare la pianta (in inglese floorplan) del circuito che vogliamo realizzare. Nella determinazione del floorplan si decide la forma che deve avere il circuito, le dimensioni geometriche complessive, il posizionamento delle linee di alimentazione.Abbiamo quindi impostato i parametri di configurazione del floorplan come suggerito nell'esercitazione, in particolare diciamo ad encounter di voler realizzare un floorplan rettangolare (ratio Height/Width=0.7) di cui le celle standard occupino il 60% dello spazio (campo core utilization). Inoltre prevediamo di lasciare lateralmente al core del circuito uno spazio vuoto delle dimensioni di 0,95um.Inoltre decidiamo di non lasciare alcuno spazio fra una riga di celle e la successiva (Row Spacing). Il 40% di spazio "in più" sarà utilizzato in fase di ottimizzazione o riempito da encounter con celle fittizie (filler); lo spazio sopra le celle (incluse le celle filler) verrà utilizzato per il routing.

Attraverso l'apposito menu "Power >Power Planning >Add stripes" abbiamo quindi definito le linee di alimentazione, in particolare delle strisce verticali di metal2 per le alimentazioni. Le linee di alimentazione e massa avranno ampiezza 0.8um e saranno distanti fra loro di 0.8um. Utilizziamo tre coppie di linee; le due più esterne saranno piazzate a 5um dai bordi est ed ovest del blocco.

Placement

Una volta realizzato il floorplan siamo passati alla fase di placement. A tal proposito abbiamo usato lo script contenente i comandi per questa fase: `scripts/place.cmd`.

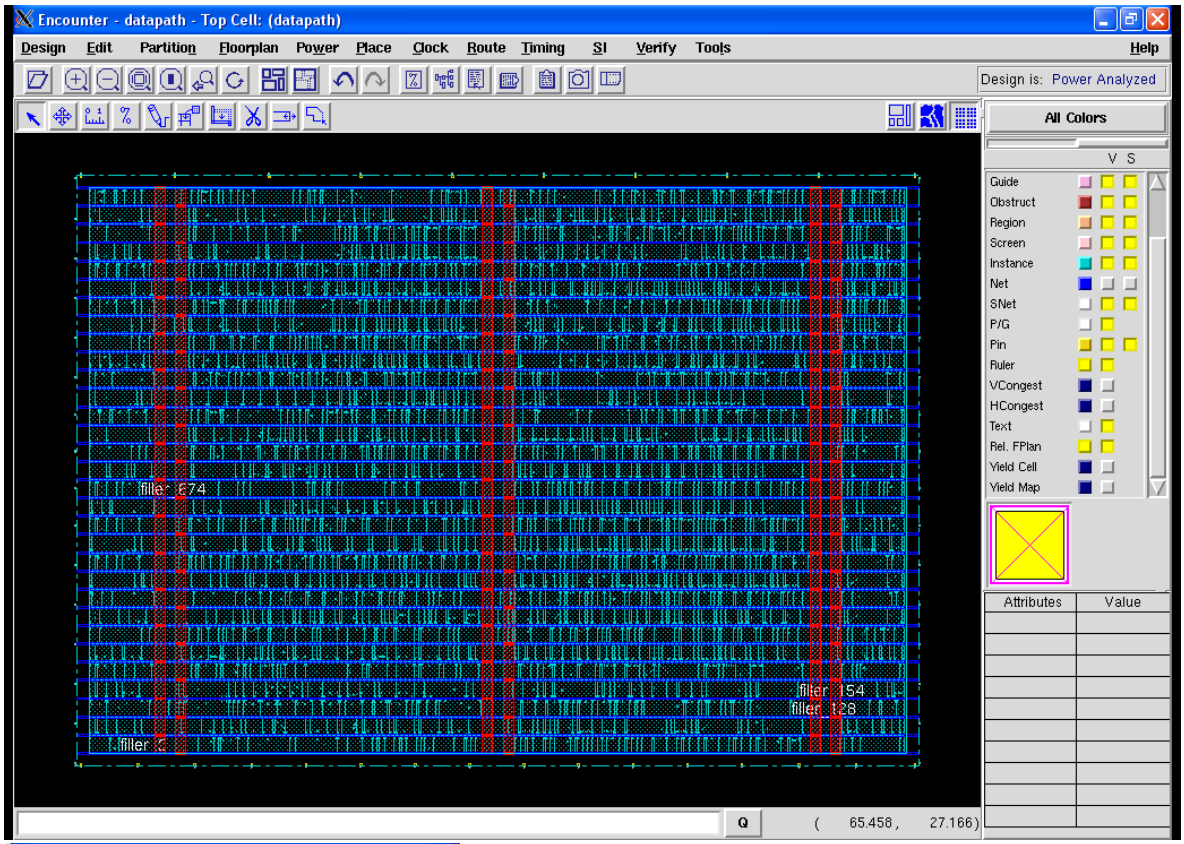
Il tool di placing in questa fase effettua quello che viene detto un trial route (un collegamento di tentativo) che serve esclusivamente per stimare le capacità delle interconnessioni.

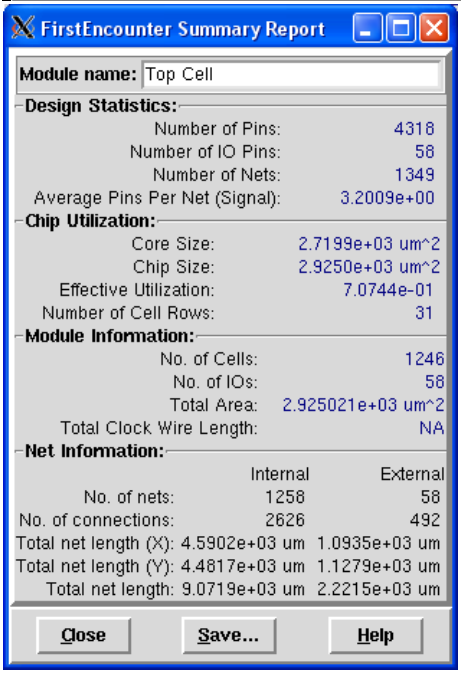


Alessandro Arrichiello M63/43
Flavio Battimo M63/38

In-place Optimization

L'ottimizzazione del circuito viene realizzata con lo script `scripts/ipo.cmd`. Si richiede al tool di ottimizzare il circuito in modo da ottenere uno slack di 100ps su tutti i cammini (parametro targetSlack). Lo script, inoltre, esegue una nuova analisi dei ritardi del circuito.





Premendo il tasto "summary report" abbiamo valutato l'occupazione di area (Effective Utilization) del circuito che risulta aumentata rispetto alle analisi precedenti:

```
Effective Utilization: 6.6507e-01
```

```
Effective Utilization: 7.0744e-01
```

Consultando le analisi dei ritardi del circuito ottenuti nelle varie fasi del progetto prima e dopo l'ottimizzazione (preCTS e preCTS_opt) abbiamo verificato inoltre che in tutti i casi in cui era presente uno slack negativo con relativa SETUP VIOLATION, dopo l'ottimizzazione tali cammini critici sono stati tutti riportati a valori di slack positivi.

Alessandro Arrichiello M63/43
Flavio Battimo M63/38

Albero di clock

Il passo successivo è stato la realizzazione dell'albero di clock.

Per effettuare questa operazione abbiamo utilizzato gli script cts.cmd e il file di configurazione clock.cts.

Quest'ultimo file viene caricato in fase di sintesi dell'albero di clock e contiene i constraints riguardanti la sua realizzazione. In particolare riporta: il nome del pin di clock, il valore massimo e minimo del ritardo sull'albero di clock, il valore massimo di skew accettabile, il tempo di transizione massimo consentito sui segnali di clock, il nome delle celle da utilizzare come buffer per la realizzazione dell'albero e la cella che si ipotizza pilotare l'albero e che determina la driving strength della logica a monte dell'albero di clock.

Lo script cts.cmd esegue la sintesi dell'albero del clock, l'analisi dei ritardi del circuito e salva due file di report: un file verilog con la netlist completa anche dei buffer dell'albero di clock, ed una serie di file di report.

Dopo aver eseguito lo script abbiamo aperto il file html di report con il web browser:

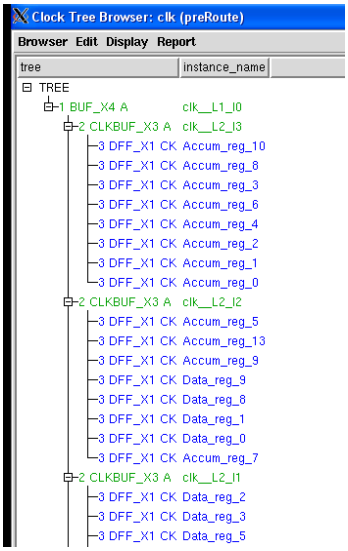
Clock Skew Summary

Clock Name	Nr. Sink	Nr. Buffer	Rise Phase Delay	Fall Phase Delay	Trig. Edge Skew	Rise Skew	Fall Skew	Max. Local Skew
clk	32	5	154.9(ps) ~ 161.5(ps)	154.4(ps) ~ 157.7(ps)	6.6(ps)	6.6(ps)	3.3(ps)	36.4(ps)

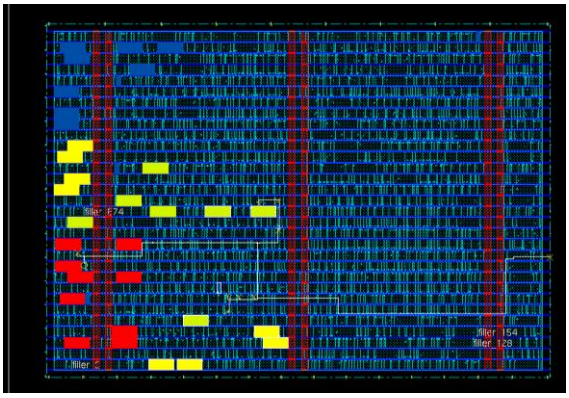
Come si può notare, poiché il circuito datapath include pochi flip-flop, l'albero di clock consente di ottenere uno skew limitatissimo.

Analizzando il timing del circuito dopo la realizzazione dell'albero di clock si può notare come lo slack sia leggermente decrementato:

```
|Setup mode   | all    | reg2reg | in2reg   | reg2out | in2out   | clkgate |
+------------+-------+---------+---------+---------+---------+---------+
|WNS (ns):    |0.157   |  0.157  |  0.369  |  1.287  |   N/A   |   N/A   |
```



Al fine di evidenziare le operazioni effettuate, abbiamo provveduto attraverso l'apposito menu di encounter ad evidenziare il clock a partire dalla selezione di alcuni registri. Abbiamo in questo modo evidenziato l'albero di clock, con diversi colori per i registri, a seconda dello skew.



Alessandro Arrichiello M63/43
Flavio Battimo M63/38

Routing

Abbiamo eseguito quindi il routing finale del circuito con lo script `scripts/route.cmd`.

```
#################
#### Routing ####
#################

#Add filler cells
addFiller   -cell   FILLCELL_X1   FILLCELL_X2   FILLCELL_X4   FILLCELL_X8
FILLCELL_X16 FILLCELL_X32 -prefix filler -noDRC

#Routing Supply nets

#Followpins
sroute     -deleteExistingRoutes          -noPadPins     -stopStripeSCPin
boundaryWithPin -noStripes -noPadRings \
       -noBlockPins -jogControl { preferWithChanges differentLayer }

# Final Routing
source script/nanoroute.cmd

saveDesign final.enc
```

Il primo comando serve per aggiungere al layout le celle filler. Queste particolari celle contengono solo le due linee di alimentazione e non contengono transistors. Sono però necessarie a riempire gli spazi vuoti del core assicurando quindi il collegamento delle linee orizzontali in metal 1 che collegano i segnali VDD e GND delle celle.

Il secondo comando (sroute) serve per collegare tutte le net di VDD e GND orizzontali e per prolungarle fino al bordo della cella.

Il comando "`source scripts/nanoroute.cmd`" richiama un altro script contenente le opzioni ed il comando vero e proprio di routing.

Infine l'ultimo comando contenuto nello script salva il progetto.

Controllo

A questo punto abbiamo analizzato il circuito e verificato che esso rispetti le regole di processo. Grazie al menù "Verify > verify connectivity" per controllare che tutti i collegamenti siano stati effettuati correttamente. E analogamente, selezionando "Verify > verify geometry" abbiamo controllato che il circuito rispetti le regole di processo.

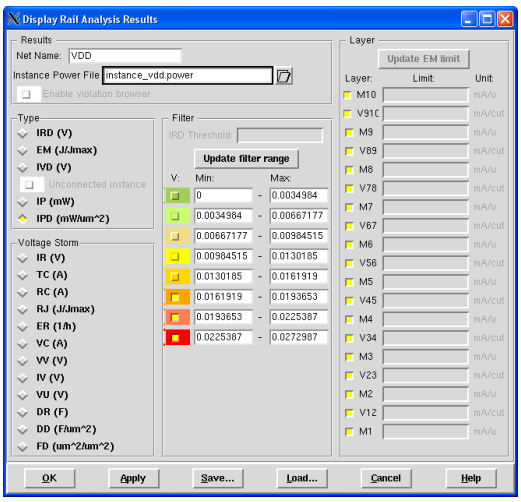
L'ultimo script che abbiamo eseguito come suggerito nell'esercitazione è stato "outputs.cmd".

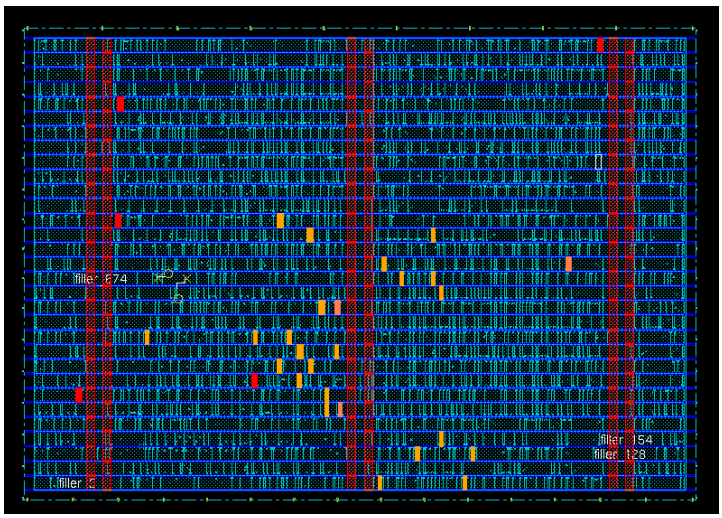
All'interno di questo script vengono effettuate varie operazioni. Innanzitutto viene effettuata una finale analisi dei ritardi del circuito i cui risultati vengono salvati nella cartella tim_final. Viene poi creato il file final.ctsrpt con l'analisi dell'albero del clock. Vengono inoltre generati diversi file contenenti descrizioni alternative del layout del datapath. Tra queste c'e' la descrizione di tipo LEF (che permette di utilizzare il circuito realizzato all'interno di un circuito più grande come se fosse una cella standard o, più precisamente, una macro), la descrizione liberty e la descrizione gds. Quest'ultimo formato è molto

importante poiché contiene tutti i layer del layout ed è il formato principale di rappresentazione dei layout dei circuiti integrati.

La parte finale dello script analizza con approccio statistico la dissipazione di potenza del circuito.

Abbiamo visualizzato il risultato di tale analisi utilizzando la voce di menù apposita del tool Encounter, analizzando ad esempio l' IPD (dissipazione di potenza per unità di area) della net VDD:





L'interfaccia grafica di encounter colora diversamente le celle del layout per mettere in evidenza le celle che dissipano più potenza.

Andiamo quindi ad aumentare la percentuale di occupazione al fine di minimizzare l'area complessiva del circuito. Ecco di seguito una tabella riepilogativa che riporta i risultati delle ripetute analisi:

%occupazione	Area (um^2)	Potenza dissipata (mW)		Slack (ns)	Minimo periodo di clock (ns)
		Totale	Albero di clock		
0.6	2925.021	2.5327	9.5728e-02	0.063	1.87
0.65	2708.950	2.5967	9.9185e-02	0.039	2.46
0.70	2556.588	2.5184	9.5138e-02	0.029	2.47
0.75	2390.376	2.6993	9.6735e-02	0.025	2.475

Per 0.80 si ottiene uno spacing violation, in pratica il tool Encounter non riesce a piazzare tutti gli elementi del circuito (dato che il 20% è rappresentato dai terminali di I/O).Dopo aver rieseguito nuovamente il place&route con 0.75 di percentuale di occupazione, attraverso l'apposito menù di encounter (Display => Physical view => Min/Max paths, dopo aver attivato nuovamente il clock tree browser) abbiamo evidenziato i due registri per i quali il ritardo sul clock assume il valore minimo e massimo.Prendiamo quindi nota della posizione fisica del morsetto di clock di questi due registri: questa informazione ci verrà utile in seguito quando effettueremo una simulazione circuitale della distribuzione del clock, al fine di controllare il valore dello skew:

X	Y
26.2	8.2
32.6	13.8

Sesta Esercitazione: Simulazione a livello transistor

In questa esercitazione effettueremo una simulazione che modelli i singoli transistor, in modo da avere una valutazione più accurata dei ritardi. Per la simulazione dei MOSFET useremo una caratterizzazione basata sul modello BSIM versione 4, abbastanza compatto ma in grado di considerare tutti i fenomeni dei MOSFET con gate sub-micrometrica.

Nella tabella che segue sono riportati i parametri fondamentali del MOSFET a canale n che utilizzeremo e che è stata completata nelle parti mancanti estraendo le informazioni dai file di modellazione.

Dispositivo	Processo	Low-field mobility (u0) [m2/Vs]	Electrical gate equivalent oxide thickness (TOXE) [m]	Silicon Oxide Dielectric Constant (3.9*ε_0) [F/m]	Long-channel threshold voltage (VTH0) [V]	k' [A/V2]	$I_{D,SAT}$ [A] VDD=VGS=1.1v L=45nm W=90nm
NMOS_VTL	Nominale	0.04359	1.75e-09	34e-12	0.471V	0.423e-3	0.335e-3
PMOS_VTL	Nominale	0.00432	1.85e-09	34e-12	-0.423V	0.0397e-3	0.0364e-3

$$K'_N = \frac{1}{2}\mu_N C_{OX} = \frac{1}{2}\mu_N \frac{\varepsilon_{OX}}{T_{OX_N}} = 0.423 * 10^{-3}$$
$$K'_P = \frac{1}{2}\mu_P \frac{\varepsilon_{OX}}{T_{OX_P}} = 0.0397 * 10^{-3}$$

Per calcolare la corrente di drain di pinch-off abbiamo trascurato l'effetto di modulazione di canale e abbiamo usato $V_{GS} = V_{DD} = 1.1v$, L=45nm e W=90nm

$$I_{D,SAT} = K' \frac{W}{L}(V_{GS} - V_T)^2(1 + \lambda V_{DS}) \cong K' \frac{W}{L}(V_{GS} - V_T)^2 = \frac{1}{2}\mu \frac{\varepsilon_{OX}}{T_{OX}} \frac{W}{L}(V_{GS} - V_T)^2$$

Per le simulazioni successive useremo il programma *spectre* e per la visualizzazione dei risultati il programma *wavescan.*

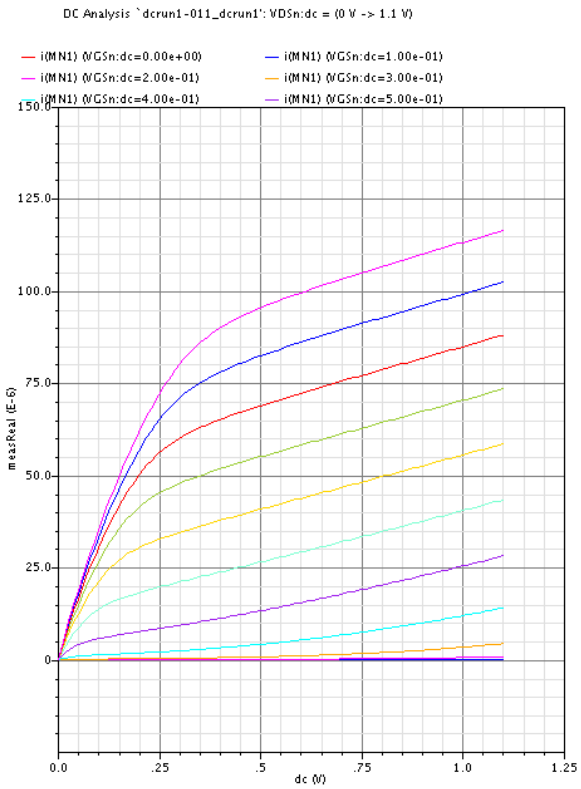
Analisi delle caratteristiche in DC dei MOSFET

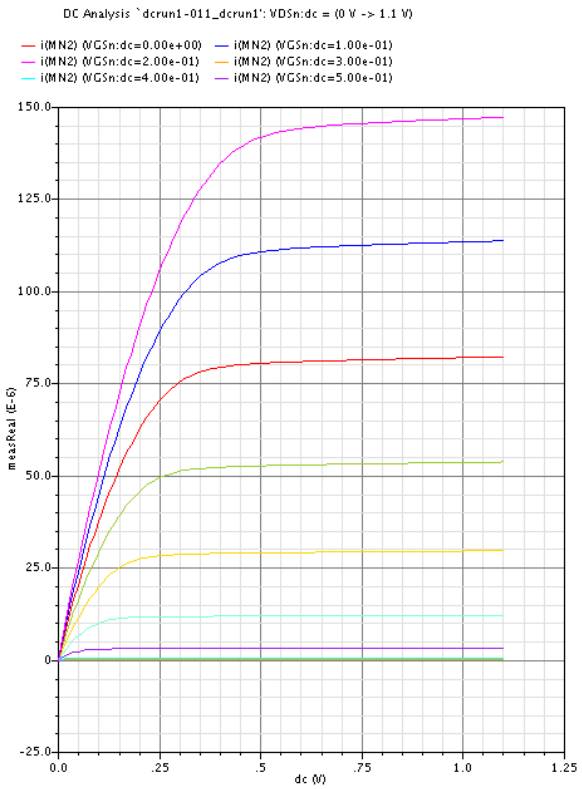
Simulazione del MOSFET in DC con variazione sia della tensione Vds che della tensione Vgs del MOS da 0 fino a 1.1V, con un passo di 0.1V.

Confrontiamo l'analisi effettuata da spectre con quella manuale. Come parametri per il MOSFET abbiamo usato K=160uA/V^2; λ=0.5 V^{-1}; vsat=0.35V; V_T=0.25V;

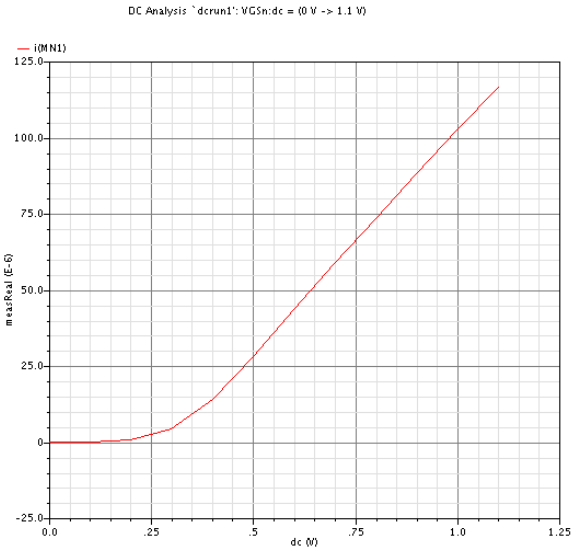
Dispositivo L=45nm, W=90nm	Spectre	Calcolo manuale
Vgs=1v, Vds=1v (pinch-off)	Id=0.1000e-3 (100uA)	Id=0.135e-3 (135uA)
Vgs=1v, Vds=0.5v (triodo)	Id=0.0825e-3 (82.5uA)	Id=0.100e-3 (100uA)

Aggiungiamo al listato un nuovo MOSFET con L=500nm e W=1000nm e effettuiamo nuovamente la simulazione confrontando i due MOS. Anche se il rapporto W/L è lo stesso le caratteristiche non sono uguali.





Ora valutiamo la transcaratteristica del dispositivo, con V_{DS}=1.1v e V_{GS} variabile fra 0 e 1.1v. La transcaratteristica varia secondo una legge esponenziale e poi lineare.



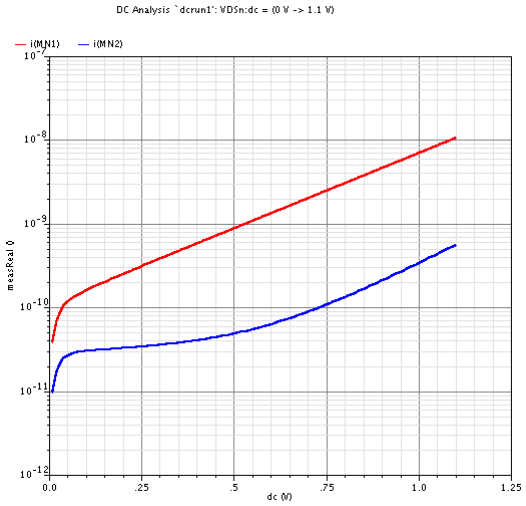
Confrontiamo ora un PMOS con un NMOS. Eseguiamo una simulazione con un PMOS e un NMOS di pari dimensione (L=45nm, W=90nm) e alla stessa polarizzazione (V_{DS}=1.1v, V_{GS}=1.1v). Osserviamo che la corrente che circola, come ci aspettavamo, è inferiore nel PMOS.

I_{NMOS}=116e-6 A, I_{PMOS}=48e-6 A, $R = \frac{I_{NMOS}}{I_{PMOS}} = 2.42$

Alessandro Arrichiello M63/43
Flavio Battimo M63/38

Leakage

Poiché i dispositivi a canale corto sono affetti da una significativa corrente di leakage, che scorre quando il dispositivo è nominalmente in interdizione, simuliamo un dispositivo a canale N con L=45nm, W=90nm e un dispositivo, sempre a canale N, con L=90nm e W=90nm, in cui la tensione VDS varia fra 0 ed 1.1V mentre VGS=0.



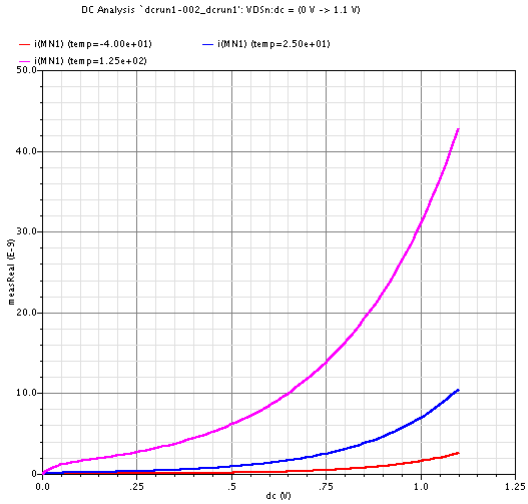
Osserviamo che la corrente di leakage nel dispositivo con L=45nm (in rosso) è circa 20 volte maggiore rispetto a quello con lunghezza di canale doppia, L=90nm (in blu). Di seguito riportiamo la corrente di leakage con V_{DS}=1.1v

$$I_{L=45nm} = 1.041 * 10^{-8} A = 10.41nA$$

$$I_{L=90nm} = 5.531 * 10^{-10} A = 0.5531nA$$

Ripetiamo la simulazione variando la temperatura modificando il parametro temp

.temp -40 25 125



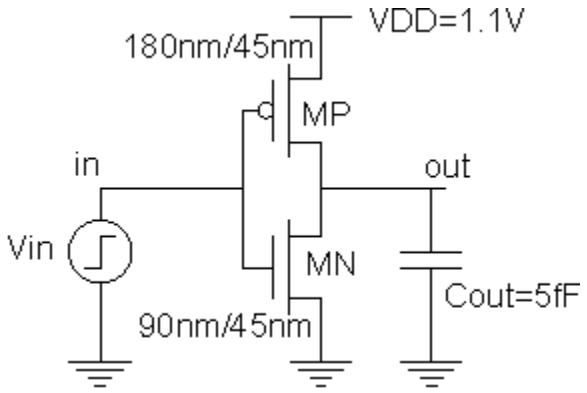
Osserviamo che con l'aumento della temperatura anche la corrente di leakage aumenta.

Se simuliamo il dispositivo L=45nm, W=90nm con il modello "fast" il valore della corrente di leakage cambia passando da 10.41nA a 67.01nA. Si verifica un aumento di circa 7 volte.

Alessandro Arrichiello M63/43
Flavio Battimo M63/38

Analisi dinamica

Effettueremo delle simulazioni usando un listato spice che modella un invertitore per valutare i ritardi del circuito. Come segnali di ingresso useremo un generatore di tensione ad onda quadra. L'onda quadra ha un tempo di salita e di discesa pari a 10ps. La tensione di 1.1V è mantenuta alta per 500ps e la forma d'onda ha periodo pari a 1ns.



Calcoliamo i ritardi dell'invertitore calcolando il tempo di propagazione da quando V_{IN} supera $V_{DD}/2$ in salita fino a quando V_{OUT} supera V_{DD} / 2 in discesa. Ripetiamo il procedimento anche per l'altro verso.

$$IN\ 0 \rightarrow 1;\ OUT\ 1 \rightarrow 0 \rightarrow t_{P_{HL}} = 29.53ps$$

$$IN\ 1 \rightarrow 0;\ OUT\ 0 \rightarrow 1 \rightarrow t_{P_{HL}} = 36.00ps$$

Calcoliamo ora le resistenze equivalenti dei dispositivi con il modello switch-level.

R_{eq} dell'NMOS usato come pull down (fase HL); $R_{eq} = \frac{\tau}{0.69C} = \frac{29.53ps}{0.69*5fF} = 8.559K\Omega$

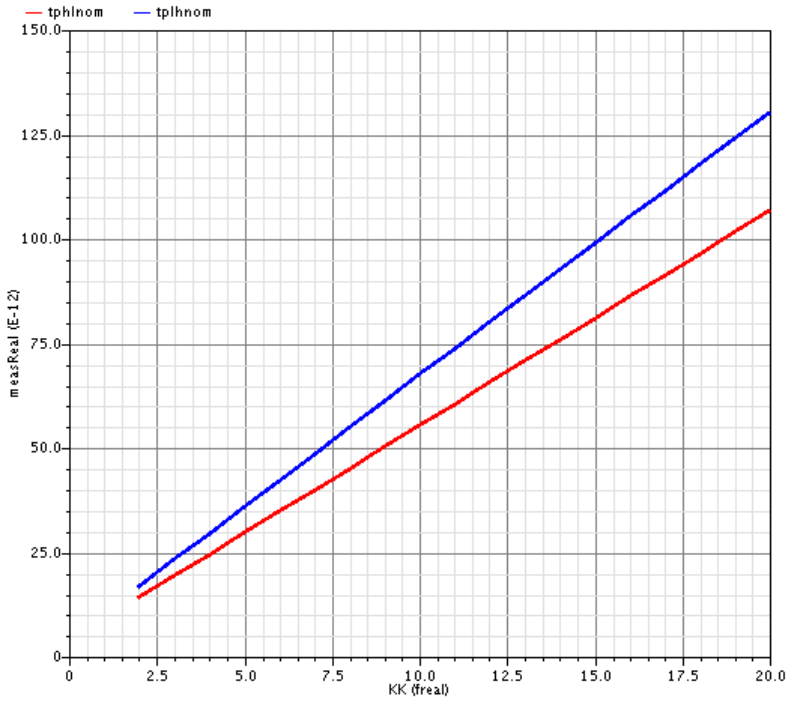
R_{eq} del PMOS usato come pull up (fase LH); $R_{eq} = \frac{\tau}{0.69C} = \frac{36ps}{0.69*5fF} = 10.434K\Omega$

Osserviamo che viene rispettata la condizione $R_{eq_{PMOS}} > R_{eq_{NMOS}}$

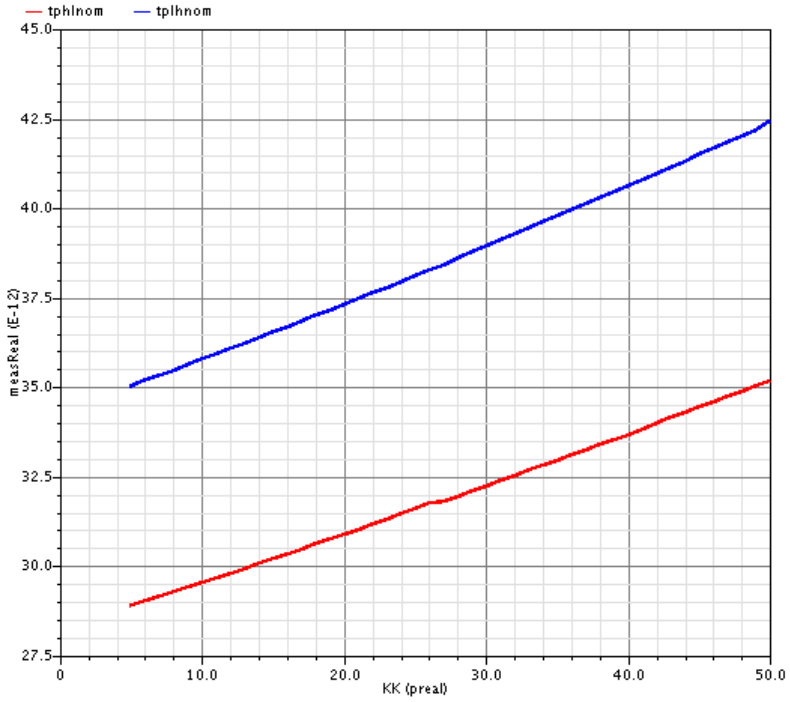
Alessandro Arrichiello M63/43
Flavio Battimo M63/38

Modifichiamo la capacità di carico in modo dinamico da 2fF a 20fF con passo di 1fF.

Osserviamo che l'aumento del tempo di propagazione aumenta in modo lineare con l'aumento della capacità di carico.



Di seguito viene riportato il grafico dei tempi di propagazione $t_{p_{HL}}$ e $t_{p_{LH}}$ al variare dei tempi di salita discesa del segnale in ingresso variabili tra 5ps e 50ps.



Alessandro Arrichiello M63/43
Flavio Battimo M63/38

Netlist gerarchiche

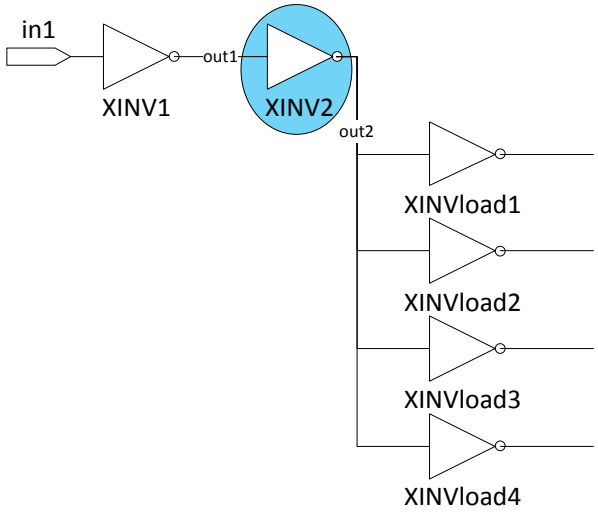
Di seguito viene riportato lo schema della netlist test_inv.spice. L'invertitore sotto esame è XINV2 (evidenziato in blu). Come segnale in ingresso utilizziamo il segnale prodotto da un generatore di tensione lineare a tratti definito dalla seguente istruzione:

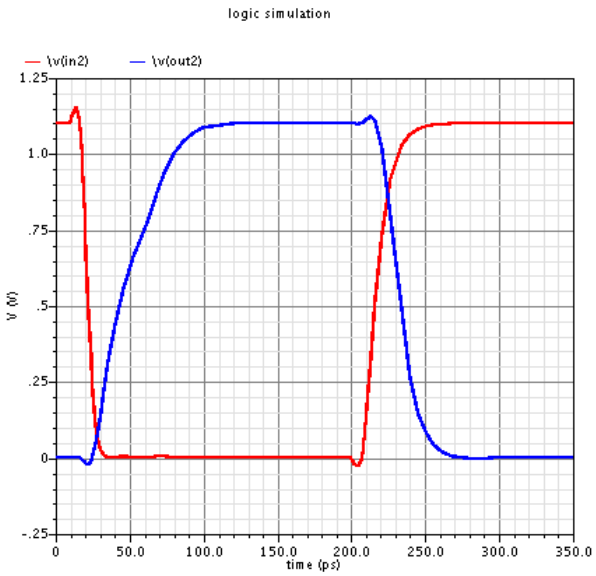
Vin in1 0 pwl 0 0 10p 0 20p 1.1 200p 1.1 220p 0

I mosfet presenti in ogni invertitore hanno le seguenti caratteristiche:

L=50nm e W_N=90nm per gli NMOS; L=50nm e W_P=135nm per i PMOS

$in2: 0 \rightarrow 1; out2: 1 \rightarrow 0; t_{p_{HL}} = 15ps$ $\qquad$ $in2: 1 \rightarrow 0; out2: 0 \rightarrow 1; t_{p_{LH}} = 23.54ps$





Per calcolare le capacità usiamo i valori $C_{OX_N} = 19.42 * 10^{-3}$e $C_{OX_P} = 18.37 * 10^{-3}$

$$C_N = W * L * C_{OX} = 0.0874fF \qquad C_P = W * L * C_{OX} = 0.1240fF$$

$$C_{TOT} = 4 * (C_N + C_P) = 0.8456fF$$

Per le resistenze equivalenti utilizziamo quelle del modello switch level calcolate precedentemente per l'NMOS (L=45nm e W=90nm) $R_{eq} = \frac{\tau}{0.69C} = \frac{29.53ps}{0.69*5fF} = 8.559K\Omega$ e per il PMOS (L=45nm e W=180nm) $R_{eq} = \frac{\tau}{0.69C} = \frac{36ps}{0.69*5fF} = 10.434K\Omega$ ma opportunamente scalate.

$$R'_N = R_{eq_N} * \frac{50nm}{45nm} * \frac{90nm}{90nm} = 9.51K\Omega \rightarrow \tau_{HL} = 0.69 * R'_N * C_{TOT} = 5.549ps$$

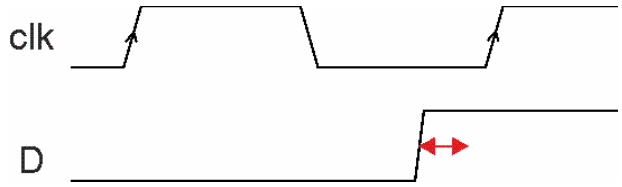
$$R'_P = R_{eq_P} * \frac{50nm}{45nm} * \frac{180nm}{135nm} = 15.458K\Omega \rightarrow \tau_{LH} = 0.69 * R'_P * C_{TOT} = 9.019ps$$

Mettendo a confronto i risultati della simulazione con quelli calcolati analiticamente abbiamo notato una notevole discrepanza, molto probabilmente dovuta all'approssimazione fatta per calcolare le resistenze equivalenti usando le resistenze proporzionali a quelle calcolate precedentemente.

Alessandro Arrichiello M63/43
Flavio Battimo M63/38

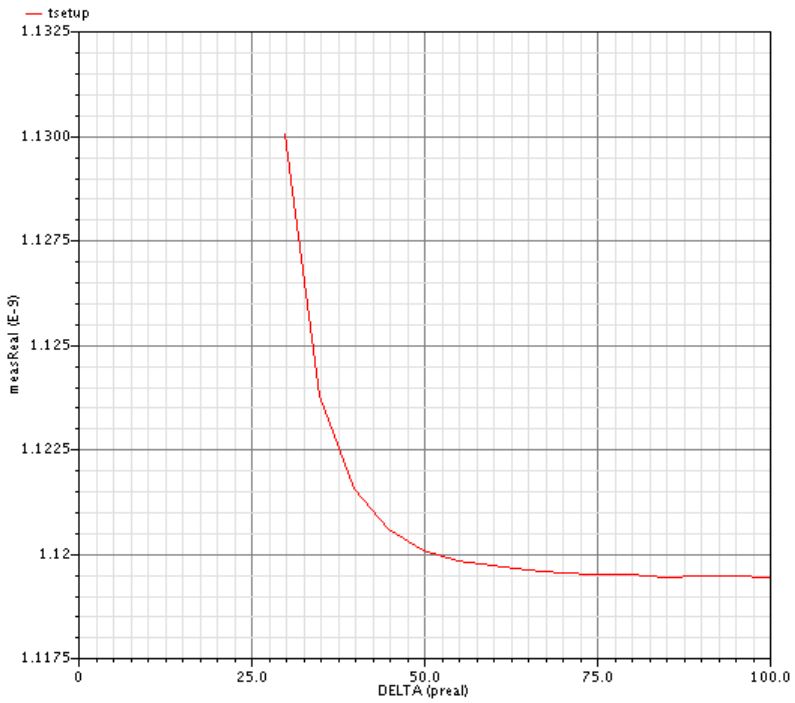
Tempi di setup e di hold di un registro

Utilizziamo spectre per valutare i tempi di setup e di hold di un flip-flop. Applichiamo i seguenti segnali per valutare il tempo di set-up del flip-flop.



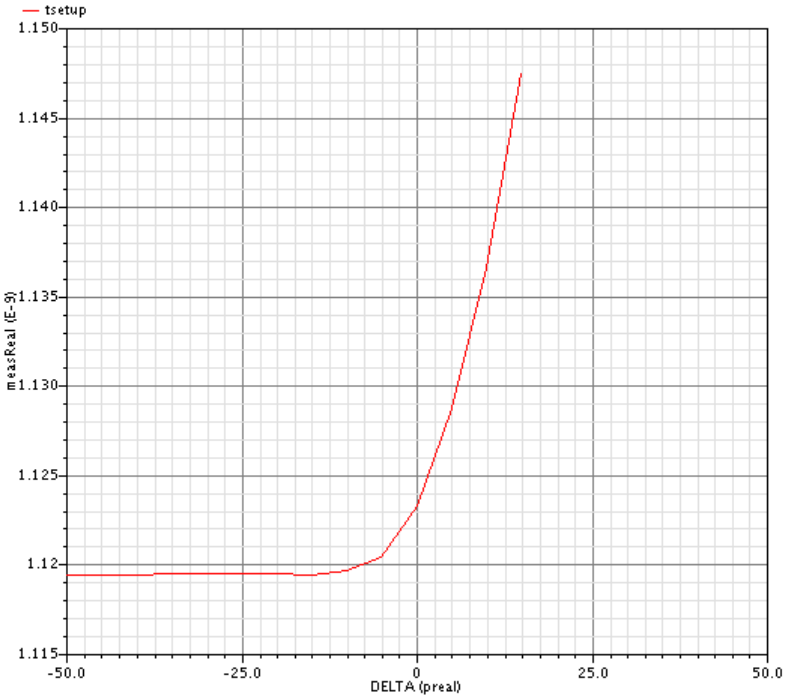
Il clock ha un periodo di 1ns e tempi di salita/discesa di 10ps. Il dato D ha anch'esso tempi di salita/discesa di 10ps mentre ha un fronte di salita opportunamente parametrizzato facendo in modo che il fronte del segnale D preceda il fronte del clock con un tempo variabile da 0ps a 100ps.

Osserviamo che al ridursi del tempo di set-up il ritardo del flip-flop aumenta sempre più fino a non commutare affatto. Sotto i 50ps di delta il tempo T_{C2Q} inizia ad aumentare in modo esponenziale.



Alessandro Arrichiello M63/43
Flavio Battimo M63/38

Osserviamo ora il comportamento del flip-flop variando l'istante in cui il segnale D cambia dopo il fronte di salita del clock. Il fronte di D varia da -50ps a +50ps da quando varia il fronte del clock.



Alessandro Arrichiello M63/43
Flavio Battimo M63/38

Simulazione con ultrasim

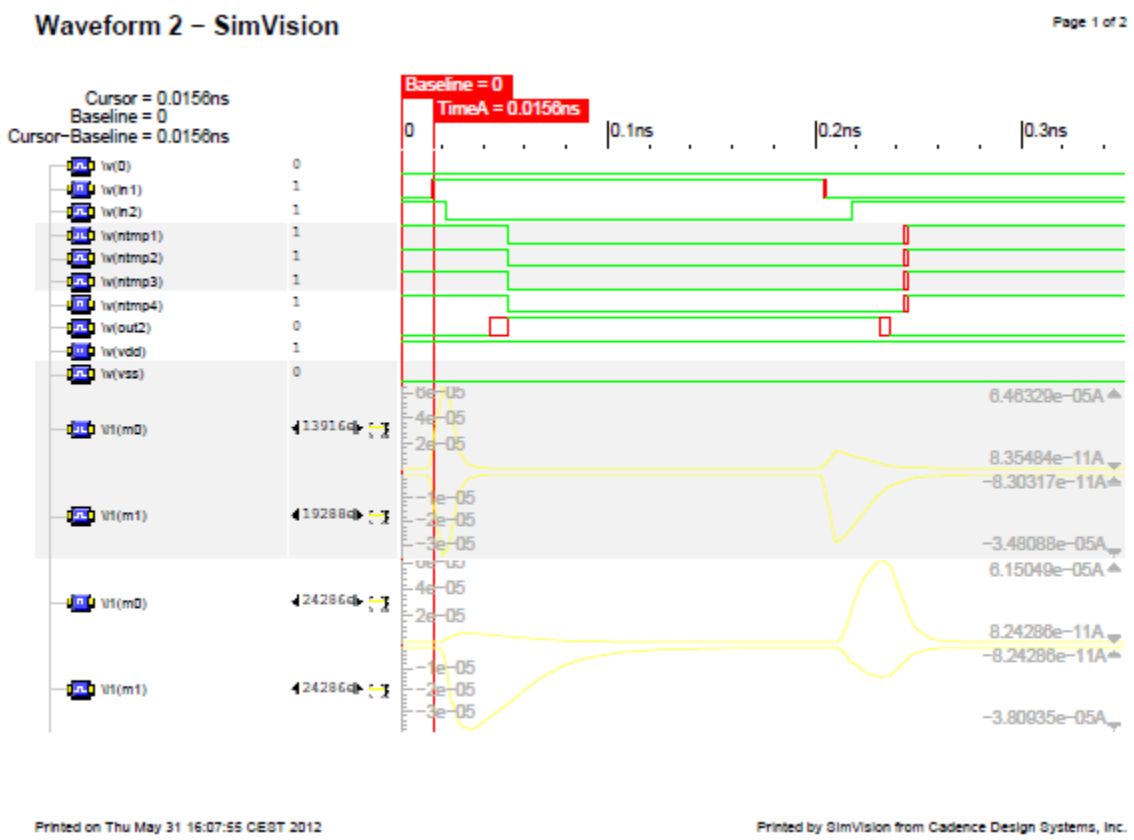
Utilizziamo Ultrasim per la simulazione dei ritardi usando le varie modalità messe a disposizione dal programma. Di seguito sono riportati i tempi di propagazione simulati. Per la simulazione useremo il circuito generato precedentemente composto dai due invertitori in serie che comandano quattro invertitori connessi in parallelo.

Simulazione	Tphl [ps]	Tplh[ps]
spectre	15	23.54
Ultrasim – spice	15.38	23.707
Ultrasim – mixed-signal	14.53	23.11
Ultrasim – digital accurate	14.36	23.554
Ultrasim – digital fast	14.541	21.635

Nel file test_inv.spice abbiamo decommentato la linea:

```
.lprobe tran low=0.50 high=0.60 v(*)
```

In questo modo vengono salvati sul file di uscita non solo i valori "analogici" delle tensioni sui vari nodi, ma anche una versione in cui ai valori di tensione sono associati i valori logici '0' ed '1'. Le tensioni di valore compreso fra 0.5V e 0.6V saranno marcate come 'indefinite'.



Alessandro Arrichiello M63/43
Flavio Battimo M63/38

Settima Esercitazione: Dal livello layout alla simulazione transistor

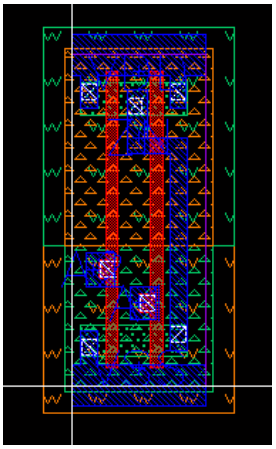
In questa esercitazione abbiamo visto come verificare la correttezza del layout di un circuito e come estrarre, da esso, una netlist a livello transistor grazie all'interfaccia grafica di Cadence Design System, un ambiente integrato di tool per il CAD di sistemi VLSI.

Attraverso il tool Library Manager abbiamo esplorato le librerie esistenti al fine di visualizzare diverse rappresentazioni per ogni componente contenuto (schematic, layout).

La libreria **std_cells** contiene ad esempio il layout di tutte le celle della libreria standard.

Maschere per la tecnologia CMOS

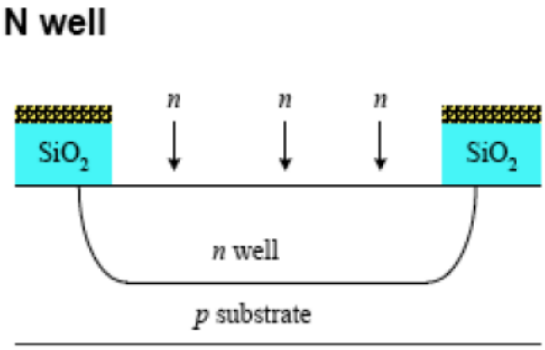
Abbiamo quindi iniziato ad esplorare il layout della cella NAND2_X1:



Cliccando due volte sulla rappresentazione di tipo layout, il tool ci mette a disposizione due finestre: una del layout editor Virtuoso ed una Layer Selection Window (LSW).

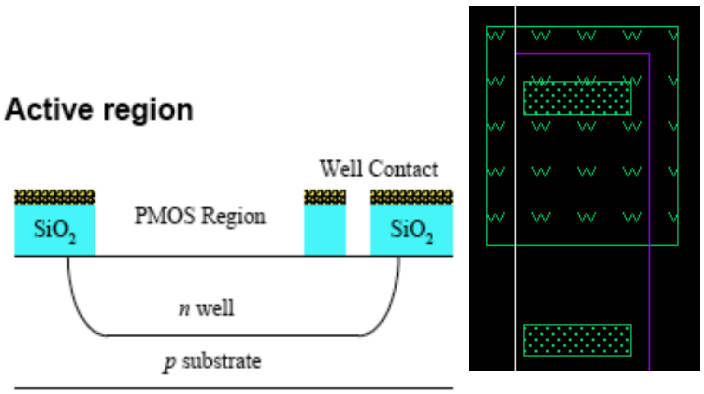
La finestra LSW mostra la lista delle maschere definite nella tecnologia a 45nm che abbiamo utilizzato nelle esercitazioni precedenti ed il colore che è utilizzato in Virtuoso per rappresentare ciascuna di esse.

Abbiamo utilizzato questo semplice layout per ricordare quali sono i passi tecnologici (e quindi le maschere) necessarie per realizzare un circuito CMOS. Focalizziamo la nostra attenzione sulla parte superiore del layout, in cui sono allocati i dispositivi PMOS.



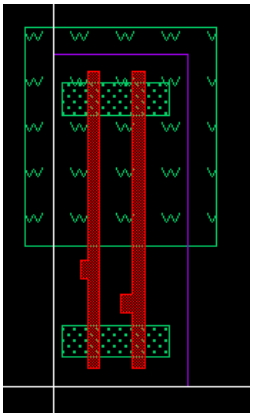
Il primo passo consiste nel definire la n-well. Per evidenziare la sola maschera della n-well, abbiamo selezionato il layer "nwell drw" nella finestra LSW e premendo sul tasto "NV" abbiamo disattivato gli altri livelli.

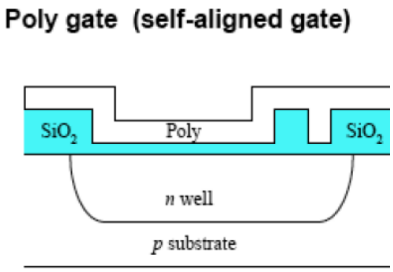
In seguito si definisce la regione attiva, in cui verranno realizzati i MOS, attivando il layer "active drw":



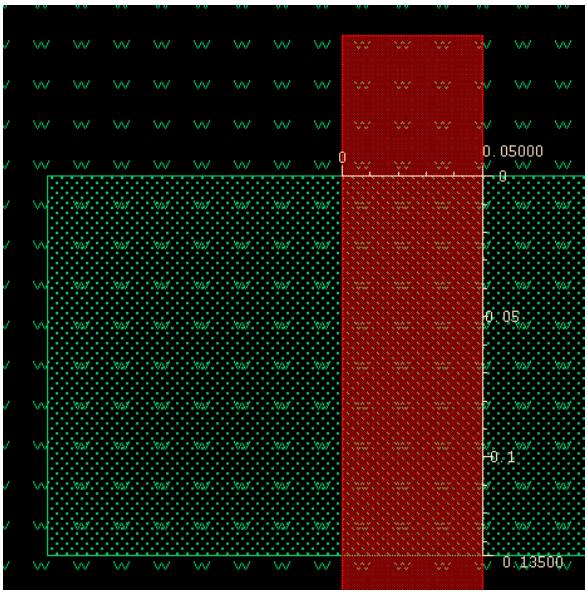
Alessandro Arrichiello M63/43
Flavio Battimo M63/38

Abbiamo quindi evidenziato il layer del polisilicio “poly drw”, che identifica le regioni di gate dei MOS e le relative interconnessioni. L’incrocio tra il polisilicio e le regioni attive definisce invece i MOSFET.

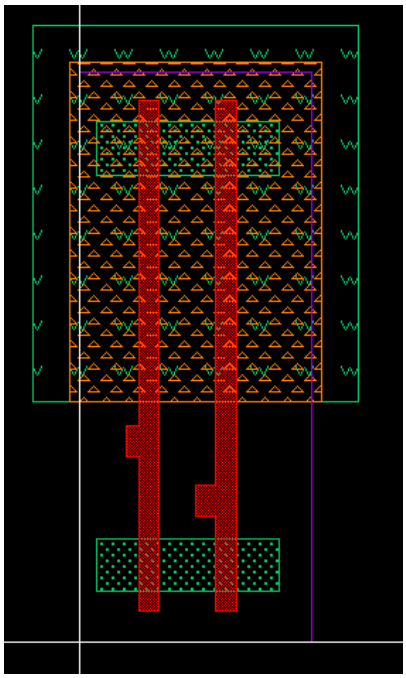


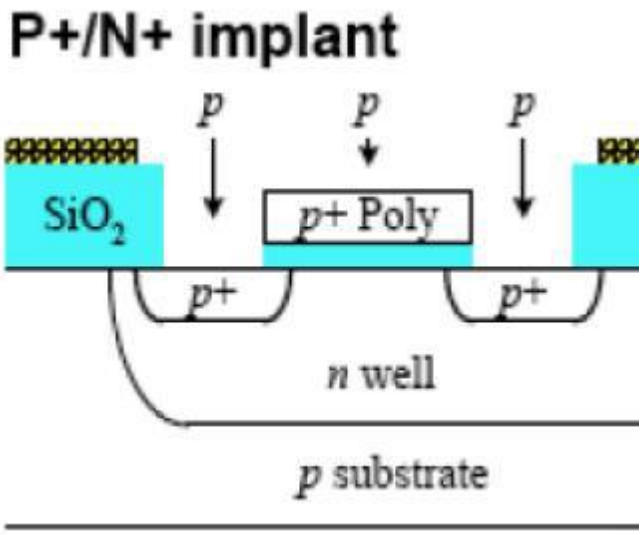


Attraverso il tool integrato abbiamo anche misurato la L(=0.05) e la W(=0.135):



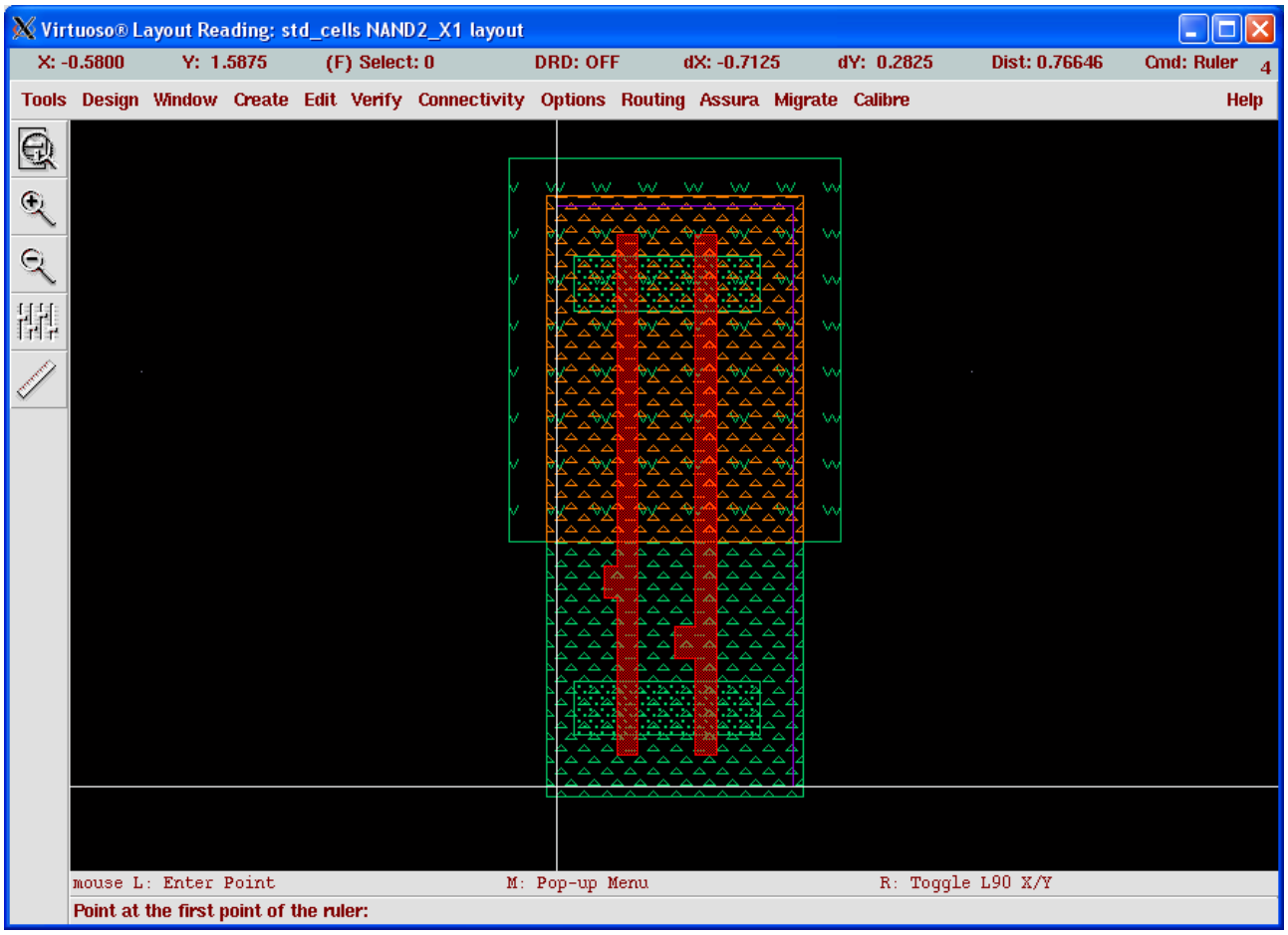
Il passo successivo consiste nella definizione delle regioni P, a tal fine si utilizza il layer “pimplant”:

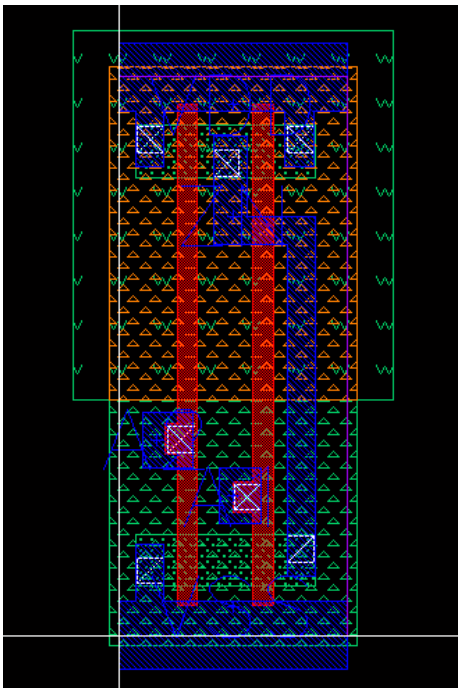




Alessandro Arrichiello M63/43
Flavio Battimo M63/38

Attivando anche il layer nimplant si può notare che le due impiantazioni definiscono una zona più ampia di quella della cella. Le regioni di source e drain dei dispositivi sono in effetti delimitate dalla maschera della regione attiva; la maschera delle impiantazioni serve solo a distinguere dove è necessario realizzare la diffusione N invece della P.





Abbiamo quindi attivato la visualizzazione dei layer successivi "contact" e "metal1" per poter vedere i collegamenti tra i drain ed i source dei MOSFET della cella nonché le label definite sui due nodi di ingresso e sul nodo di uscita. Nel layout della cella sono facilmente riconoscibili le due linee in metal1 collegate alle alimentazioni (VDD in alto e VSS in basso) nonché i due MOSFET a canale n posti in serie ed i due MOSFET a canale p messi in parallelo.

Le linee di alimentazione superano il bordo della cella. Questo succede perché si presuppone che sopra e sotto ogni cella sia piazzata un'altra cella ribaltata verticalmente che condivida le stesse linee di alimentazione e massa.

I layers metal2, metal3, … metal10 rappresentano i rimanenti 9 livelli di metal presenti nella tecnologia che stiamo utilizzando, mentre i layers via1, via2, … via9 sono utilizzati per collegare tra di loro i livelli di metal. Il processo tecnologico che stiamo utilizzando

prevede l'utilizzo di una doppia well.

Struttura delle celle standard

Le celle della libreria standard sono progettate per poter essere affiancate l'una all'altra. Per questo motivo:

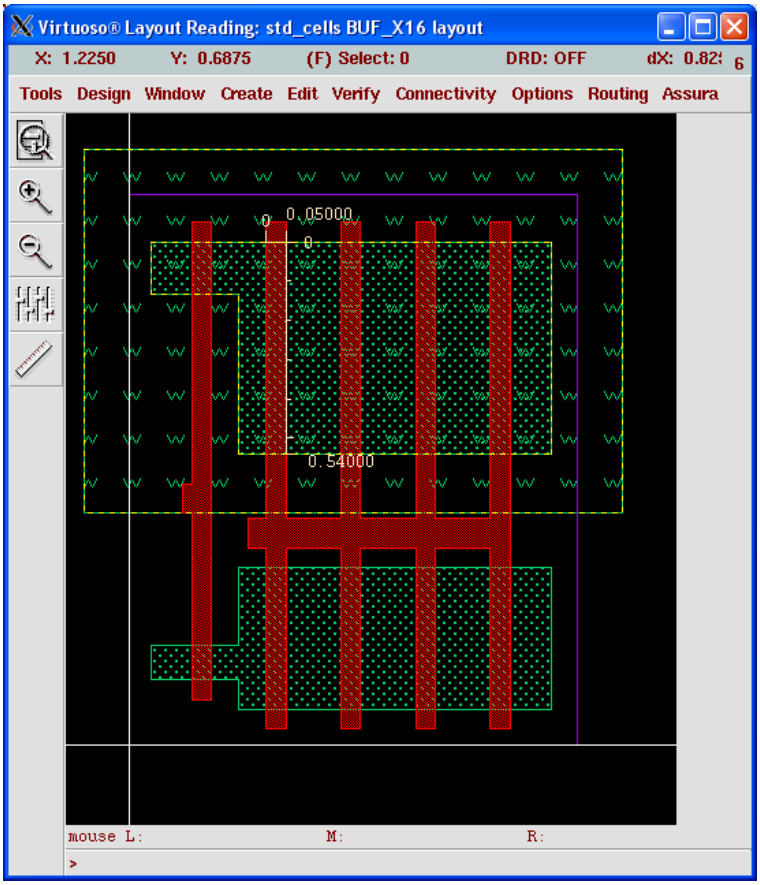
1) tutte le celle hanno la stessa altezza

2) il posizionamento delle linee collegate a VDD ed a VSS di tutte le celle è lo stesso

3) l'ampiezza delle regioni di well per tutte le celle è la stessa

I vincoli precedenti pongono un limite alla massima W utilizzabile per realizzare il singolo MOSFET. Per realizzare dispositivi con larghezza superiore si possono mettere in parallelo diversi MOSFET.

Per vedere un esempio di questa tecnica abbiamo aperto il layout della cella BUF_X16 e calcolato la W del secondo invertitore: W = 0.54 * 4 = 2.16



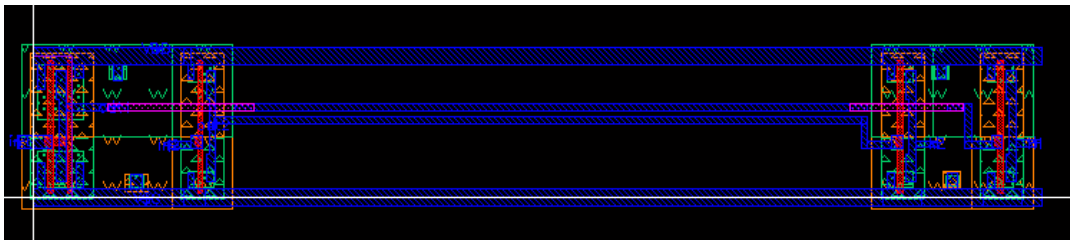
Alessandro Arrichiello M63/43
Flavio Battimo M63/38

Controllo delle regole di progetto

Dopo aver realizzato il layout di un circuito integrato (sia esso una cella standard di libreria o un circuito più complesso come il datapath di pico) esso va verificato, per assicurarsi che rispetti le regole di progetto della tecnologia utilizzata.

Per la verifica delle regole di progetto (e per l'estrazione circuitale) abbiamo usato Calibre, sviluppato dalla Mentor.

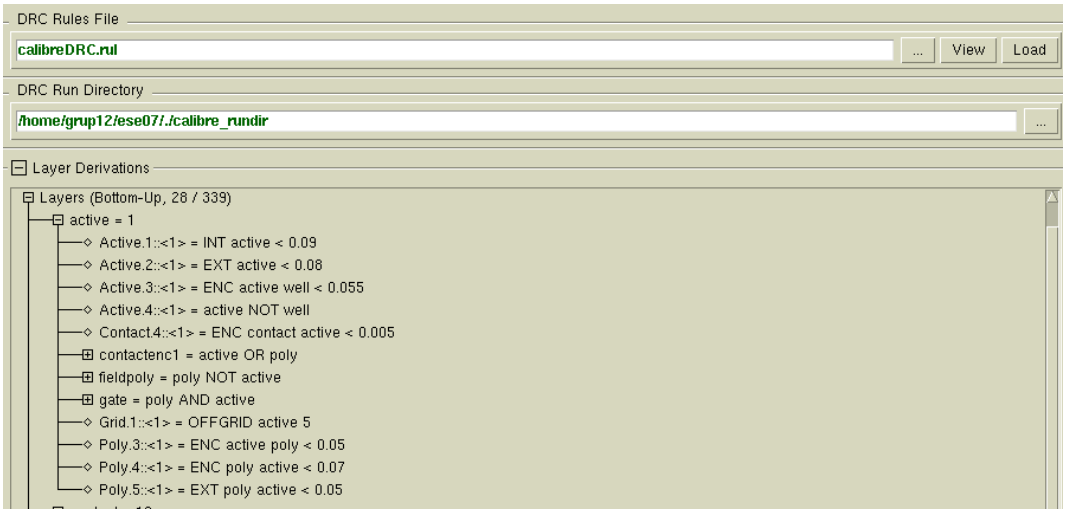
La cella che abbiamo inizialmente analizzato è "buffers_m1_m1" della libreria ctalk, di cui si mostra il layout di seguito:

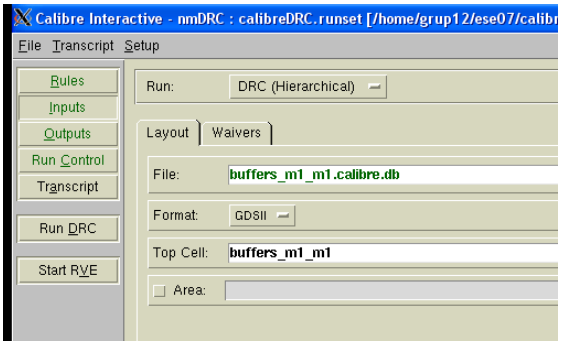


La cella presenta nella parte sinistra due invertitori; a destra vi è il fanout, costituito anch'esso da una coppia di invertitori. Il collegamento fra drivers e fanout è realizzato mediante due linee in metal1, che viaggiano affiancate per una lunghezza di circa 6um.

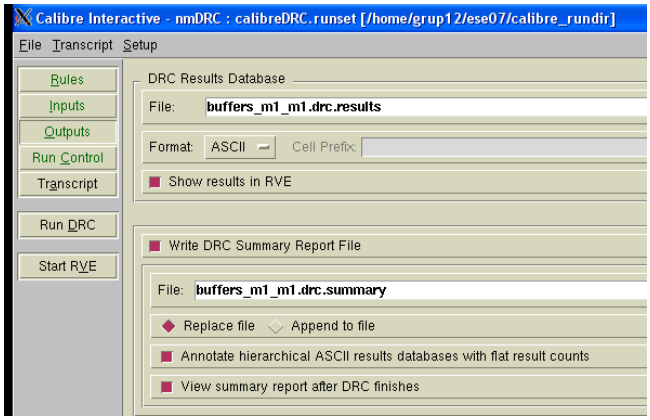
Abbiamo quindi eseguito la verifica delle regole di progetto selezionando la voce di menù "Calibre -> Run DRC", attraverso l'uso del runset file. Tale file fissa i parametri dell'analisi evitando di doverli inserire a mano.

Si può notare come a partire dalle geometrie del layout, vengano riconosciuti i dispositivi della netlist. Ad esempio espandendo la voce "active = 1": si nota che la gate dei MOS si ottiene mettendo in AND la geometria del polisilicio e quella del layer active, mentre sottraendo dal polisilicio la geometria del layer active si ottiene il layer "fieldpoly" che rappresenta le interconnessioni in polisilicio.





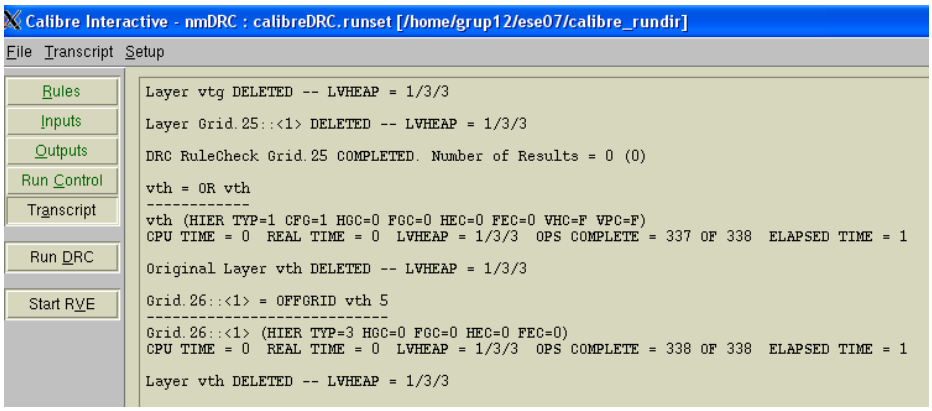
Nella sezione “inputs” sono definiti i dati di ingresso della verifica. Calibre è settato per eseguire un DRC gerarchico operante su un file GDS ottenuto a partire dal layout della cella. In questa sezione è riportato anche il nome della cella. Inoltre è possibile definire un’area del layout in cui eseguire la verifica.



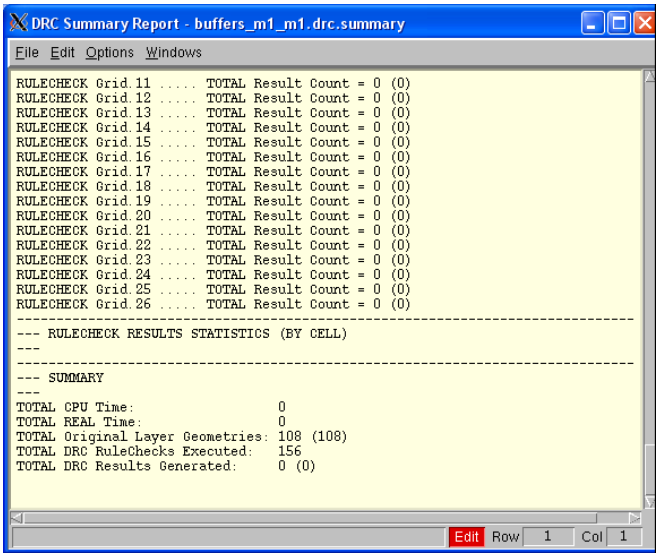
Nella scheda “output” è possibile definire come Calibre fornirà i risultati dell’analisi. In particolare è possibile definire il nome del file di report in cui segnare tutte le violazioni ottenute.

L’analisi è stata quindi avviata attraverso il tasto “Run DRC”.

La finestra Calibre Interactive si porterà nella sezione transcript in cui verrà mostrato l’avanzamento della procedura di verifica, al termine della quale si apriranno due nuove finestre.



Nella finestra DRC Summary Report viene mostrato il contenuto del file di report che elenca tutti i controlli effettuati da Calibre e, per ciascun controllo, il numero di violazioni ritrovate nel layout.

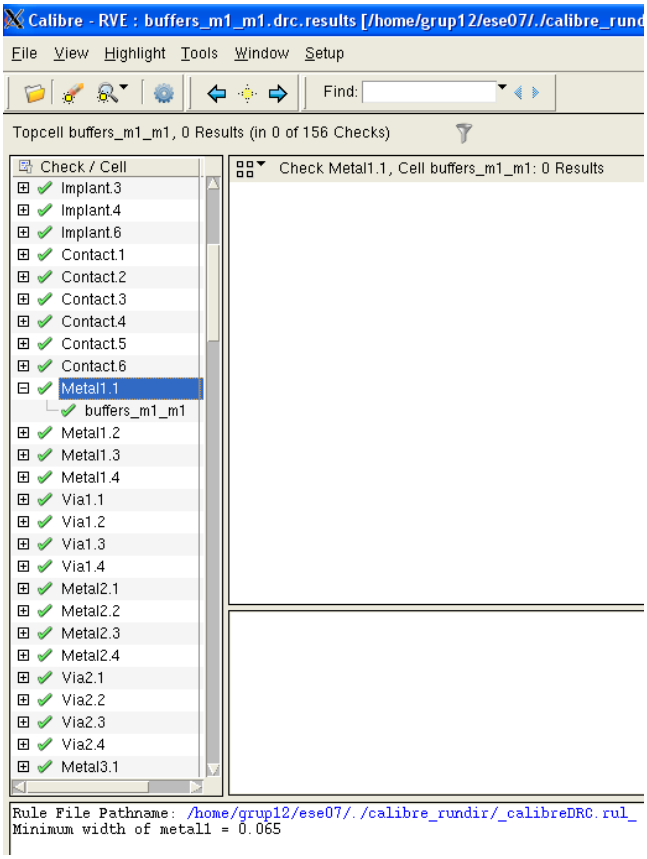


Alessandro Arrichiello M63/43
Flavio Battimo M63/38

La finestra RVE (Results Viewing Environment) rappresenta invece uno strumento interattivo che consente di navigare tra le violazioni rilevate da Calibre e di visualizzarle direttamente nella finestra del layout editor, in modo da semplificarne la correzione.

Nella parte sinistra della finestra RVE si può leggere la lista di tutti i controlli effettuati da Calibre.

Ad esempio con il controllo Metal1.1 Calibre si assicura che le linee in metal1 abbiano uno spessore minimo di 65nm, mentre la regola Metal1.3 impone che le zone in cui il metal1 si collega alla diffusione o al polisilicio tramite il layer contact siano di 35nm più larghe del contatto stesso.



Alessandro Arrichiello M63/43
Flavio Battimo M63/38

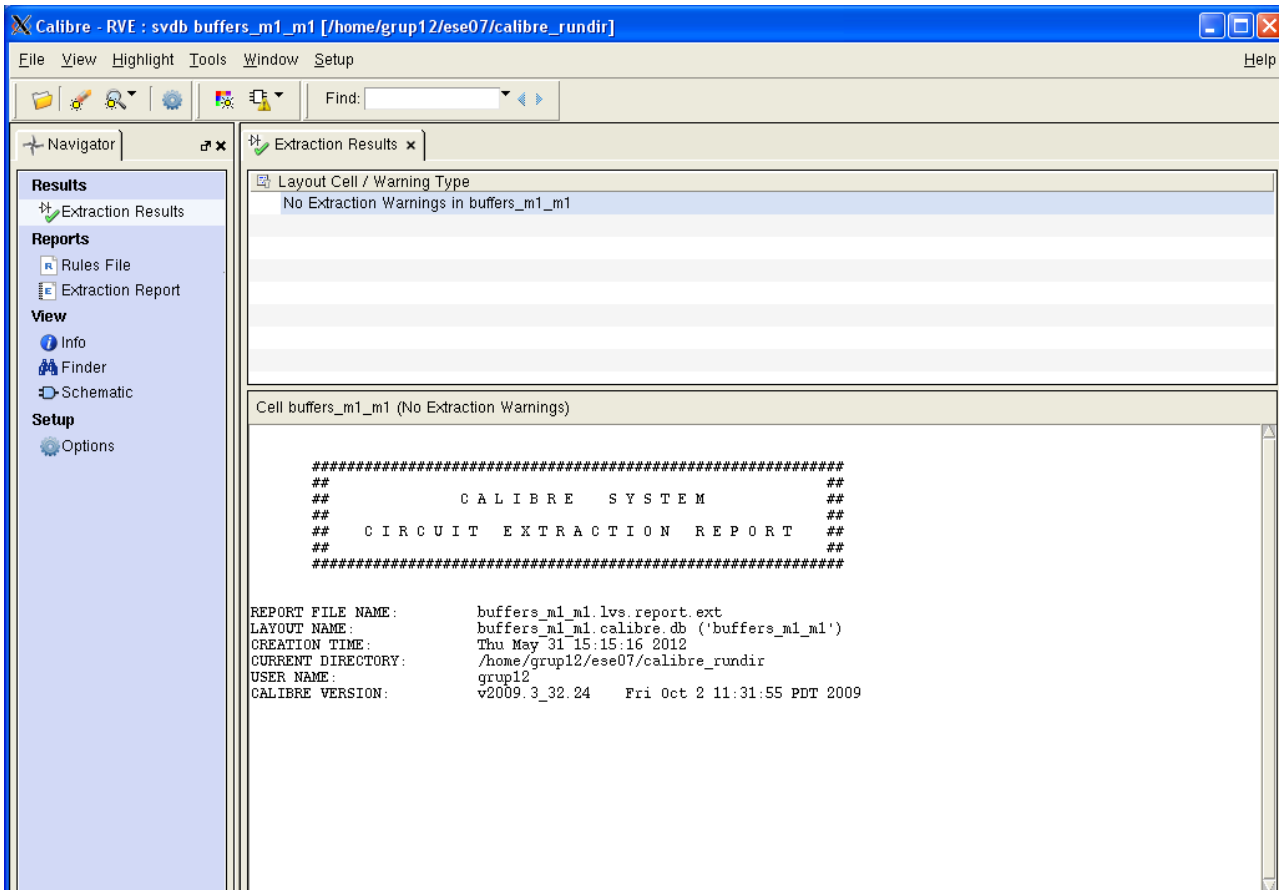
Estrazione circuitale

Dal layout è possibile estrarre una netlist a livello transistor contenente tutti i dispositivi attivi e tutti gli elementi parassiti introdotti dalle interconnessioni, da cui è possibile effettuare una simulazione estremamente accurata del circuito.

Tale estrazione può essere effettuata attraverso il menù “Calibre -> Run LVS”.

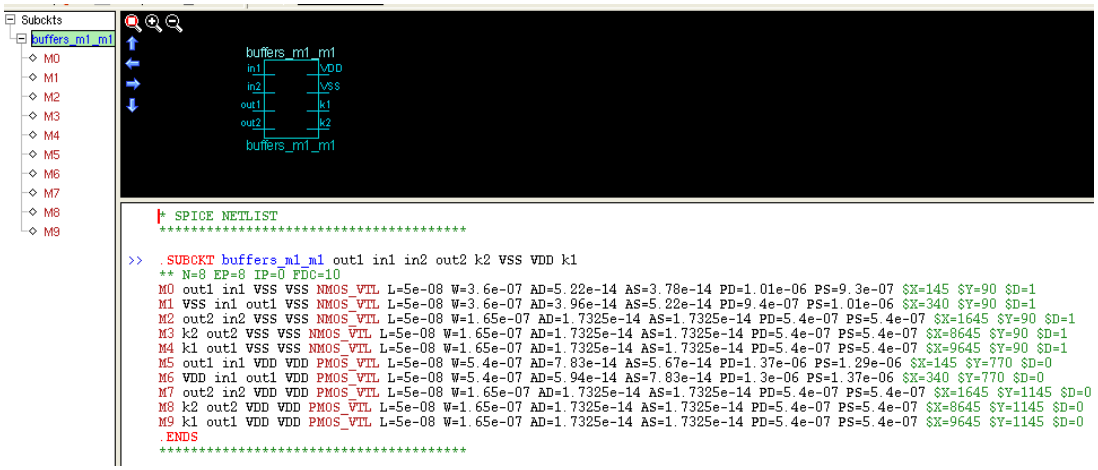
La sigla LVS sta per Layout versus Schematic. L’analisi LVS serve per confrontare un layout con la vista schematica di un circuito. Nel nostro caso non abbiamo una vista schematica del circuito, quindi l’analisi LVS ci permetterà solo di estrarre la netlist del circuito e di visualizzarla.

La finestra RVE in questo caso si presenta in maniera diversa da come appare a valle dell’analisi DRC:

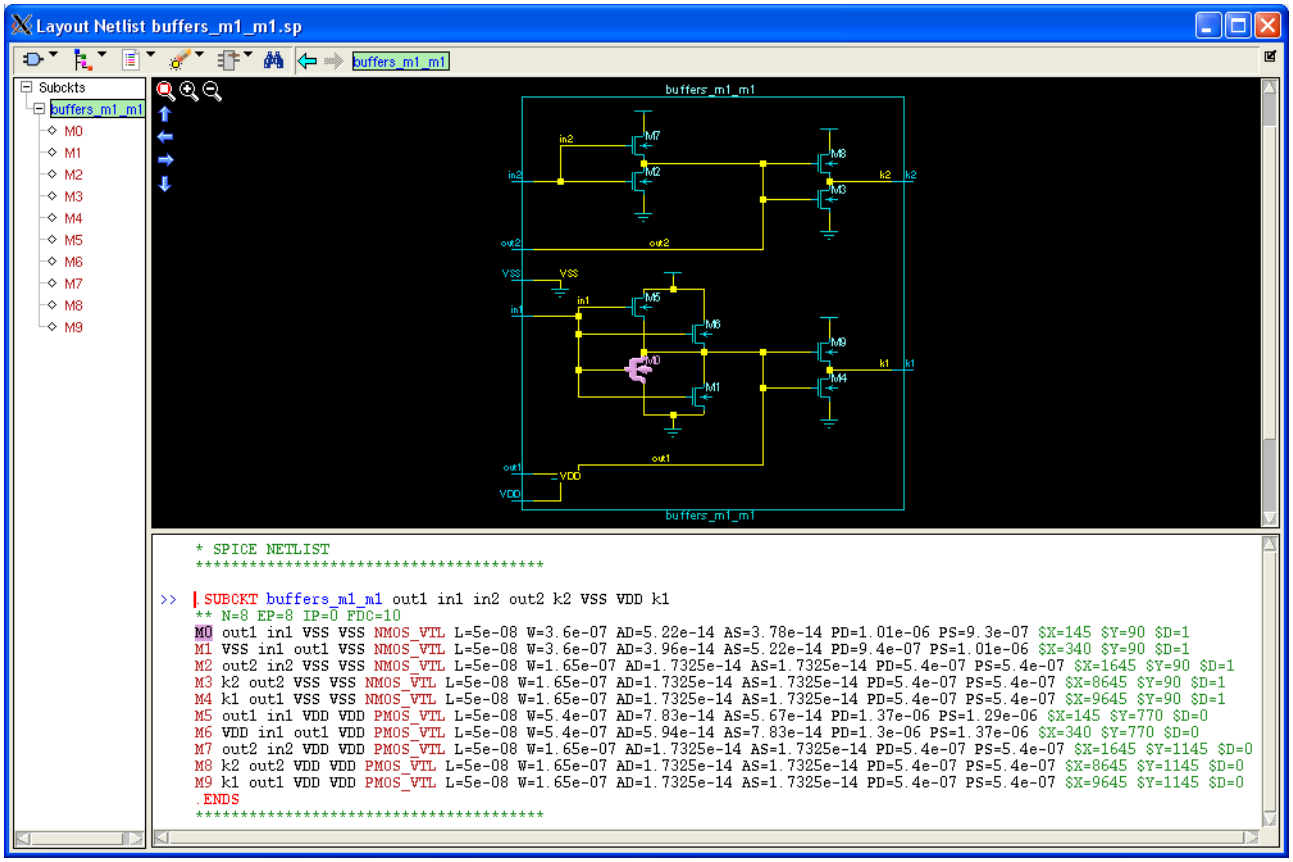


Alessandro Arrichiello M63/43
Flavio Battimo M63/38

Selezionando la voce “Schematic” viene mostrata la netlist estratta, sia in formato testo che grafico:

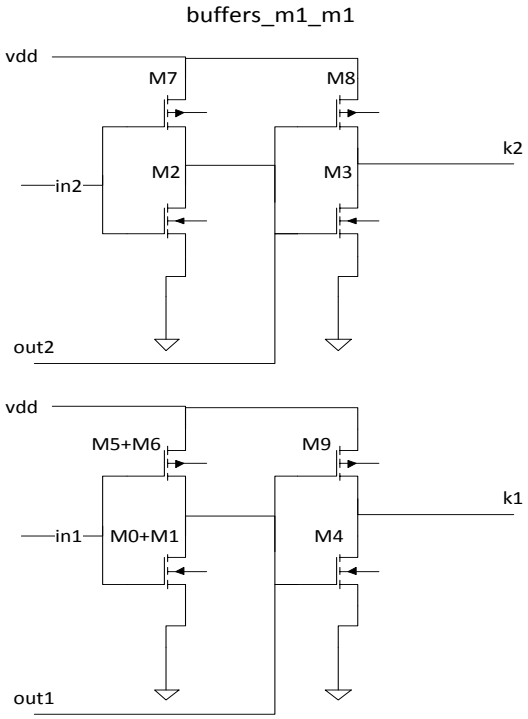


Premendo sul tasto destro del mouse sul nome di un dispositivo e selezionando l’opzione “highlight” il dispositivo stesso verrà evidenziato sia nello schema elettrico che nel layout:



Ecco di seguito un circuito equivalente a quello in figura precedente:

Alessandro Arrichiello M63/43
Flavio Battimo M63/38



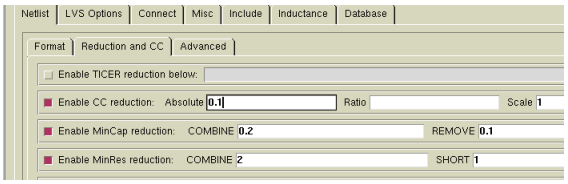
Abbiamo quindi effettuato un'estrazione circuitale considerando anche i dispositivi parassiti, attraverso il menù "Calibre -> Run PEX".

Nella sezione "Outputs" della finestra Calibre interactive abbiamo scelito il tipo di estrazione desiderata: "Transistor level", "R+C+CC" e "No inductance".

Con queste opzioni richiediamo a Calibre di estrarre una netlist a livello transistor in cui siano anche annotati i valori di resistenza parassite (R), capacità verso massa (C) e capacità di cross-coupling (CC). Non estrarremo, invece, le induttanze parassite.

Nella casella Format possiamo poi scegliere il formato in cui scrivere la netlist del circuito. Il formato utilizzato in queste esercitazioni è HSPICE.

Abbiamo poi istruito l'estrattore attraverso le "PEX options" a non includere nella netlist, se possibile, capacità di valore minore ad 1fF e resistenze di valore inferiore ad 1Ω.



Premendo sul tasto "Run PEX" la finestra PEX netlist file mostra il file ottenuto tramite l'estrazione: buffers_m1_m1.pex.netlist

Alessandro Arrichiello M63/43
Flavio Battimo M63/38

PEX Netlist File - buffers_m1_m1.pex.netlist

File Edit Options Windows

```
* File: buffers_m1_m1.pex.netlist
* Created: Thu May 31 15:41:53 2012
* Program "Calibre xRC"
* Version "v2009.3_32.24"
*
.include "buffers_m1_m1.pex.netlist.pex"
.subckt BUFFERS_M1_M1  OUT1 IN1 IN2 OUT2 K2 VSS VDD K1
*
* K1 K1
* VDD VDD
* VSS VSS
* K2 K2
* OUT2 OUT2
* IN2 IN2
* IN1 IN1
* OUT1 OUT1
M0 N_OUT1_M0_d N_IN1_M0_g N_VSS_M0_s N_VSS_M0_b NMOS_VTL L=5e-08 W=3.6e-07
+ AD=5.22e-14 AS=3.78e-14 PD=1.01e-06 PS=9.3e-07
M1 N_VSS_M1_d N_IN1_M1_g N_OUT1_M0_d N_VSS_M0_b NMOS_VTL L=5e-08 W=3.6e-07
+ AD=3.96e-14 AS=5.22e-14 PD=9.4e-07 PS=1.01e-06
M2 N_OUT2_M2_d N_IN2_M2_g N_VSS_M2_s N_VSS_M0_b NMOS_VTL L=5e-08 W=1.65e-07
+ AD=1.7325e-14 AS=1.7325e-14 PD=5.4e-07 PS=5.4e-07
M3 N_K2_M3_d N_OUT2_M3_g N_VSS_M3_s N_VSS_M3_b NMOS_VTL L=5e-08 W=1.65e-07
+ AD=1.7325e-14 AS=1.7325e-14 PD=5.4e-07 PS=5.4e-07
M4 N_K1_M4_d N_OUT1_M4_g N_VSS_M4_s N_VSS_M3_b NMOS_VTL L=5e-08 W=1.65e-07
+ AD=1.7325e-14 AS=1.7325e-14 PD=5.4e-07 PS=5.4e-07
M5 N_OUT1_M5_d N_IN1_M5_g N_VDD_M5_s N_VDD_M5_b PMOS_VTL L=5e-08 W=5.4e-07
+ AD=7.83e-14 AS=5.67e-14 PD=1.37e-06 PS=1.29e-06
M6 N_VDD_M6_d N_IN1_M6_g N_OUT1_M5_d N_VDD_M5_b PMOS_VTL L=5e-08 W=5.4e-07
+ AD=5.94e-14 AS=7.83e-14 PD=1.3e-06 PS=1.37e-06
M7 N_OUT2_M7_d N_IN2_M7_g N_VDD_M7_s N_VDD_M5_b PMOS_VTL L=5e-08 W=1.65e-07
+ AD=1.7325e-14 AS=1.7325e-14 PD=5.4e-07 PS=5.4e-07
M8 N_K2_M8_d N_OUT2_M8_g N_VDD_M8_s N_VDD_M8_b PMOS_VTL L=5e-08 W=1.65e-07
+ AD=1.7325e-14 AS=1.7325e-14 PD=5.4e-07 PS=5.4e-07
M9 N_K1_M9_d N_OUT1_M9_g N_VDD_M9_s N_VDD_M8_b PMOS_VTL L=5e-08 W=1.65e-07
+ AD=1.7325e-14 AS=1.7325e-14 PD=5.4e-07 PS=5.4e-07
*
.include "buffers_m1_m1.pex.netlist.BUFFERS_M1_M1.pxi"
*
.ends
*
```

Edit Row 1 Col 1

Rispetto alla netlist vista durante la fase di analisi LVS, in questo file viene utilizzata la clausola .include per includere il contenuto di 2 ulteriori file (anch'essi prodotti da Calibre):

- buffers_m1_m1.pex.netlist.BUFFERS_M1_M1.pxi
- buffers_m1_m1.pex.netlist.pex

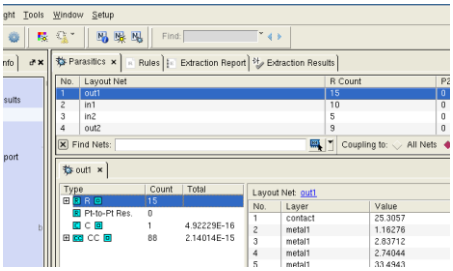
Il file "buffers_m1_m1.pex.netlist.pex" riporta un elenco di subcircuits, ognuno dei quali rappresenta un elemento di interconnessione, costituito da resistenze e capacità.

```
* File: buffers_m1_m1.pex.netlist.pex
* Created: Thu May 31 15:41:53 2012
* Program "Calibre xRC"
* Version "v2009.3_32.24"
* Nominal Temperature: 27C
* Circuit Temperature: 27C
*
.subckt PM_BUFFERS_M1_M1%OUT1 2 6 16 19 54 59
c2 2 0 0.71945f
r3 46 48 9.46974
r4 43 46 3.39286
r5 43 59 5.79882
r6 40 59 33.4943
r7 38 40 5.79882
r8 35 38 4.44643
r9 35 54 6.06482
r10 31 52 4.8207
```

```
r11 31 51 4.7167
r12 31 48 25.3057
r13 27 54 2.74044
r14 19 27 2.83712
r15 16 27 1.16276
r16 6 52 100.152
r17 2 51 52.728
.ends

°°°
```

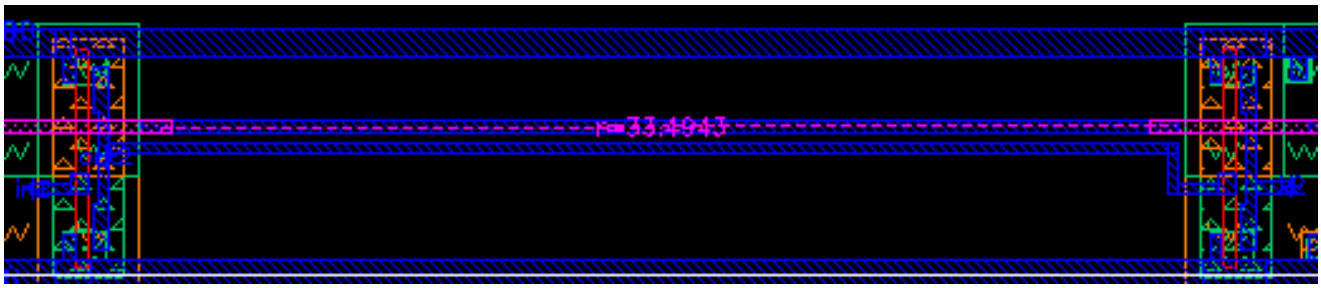
L'altro file "buffers_m1_m1.pex.netlist.BUFFERS_M1_M1.pxi" istanzia i sottocircuiti che rappresentano le interconnessioni ed inoltre include le capacità di cross-coupling (i cui nomi iniziano con le lettere "cc").

```
* File: buffers_m1_m1.pex.netlist.BUFFERS_M1_M1.pxi
* Created: Thu May 31 15:41:53 2012
*
x_PM_BUFFERS_M1_M1%OUT1 N_OUT1_M4_g N_OUT1_M9_g N_OUT1_M5_d N_OUT1_M0_d OUT1
+ N_OUT1_c_1_p PM_BUFFERS_M1_M1%OUT1
x_PM_BUFFERS_M1_M1%IN1 N_IN1_M0_g N_IN1_M5_g N_IN1_M1_g N_IN1_M6_g IN1
+ PM_BUFFERS_M1_M1%IN1
x_PM_BUFFERS_M1_M1%IN2 N_IN2_M2_g N_IN2_M7_g IN2 PM_BUFFERS_M1_M1%IN2
x_PM_BUFFERS_M1_M1%OUT2 N_OUT2_M3_g N_OUT2_M8_g N_OUT2_M2_d N_OUT2_M7_d OUT2
+ N_OUT2_c_3_n PM_BUFFERS_M1_M1%OUT2
x_PM_BUFFERS_M1_M1%K2 N_K2_M3_d N_K2_M8_d K2 PM_BUFFERS_M1_M1%K2
x_PM_BUFFERS_M1_M1%VSS N_VSS_M0_s N_VSS_M1_d N_VSS_M0_b VSS N_VSS_M2_s
+ N_VSS_M3_s N_VSS_M3_b N_VSS_M4_s PM_BUFFERS_M1_M1%VSS
x_PM_BUFFERS_M1_M1%VDD N_VDD_M5_s N_VDD_M6_d N_VDD_M5_b VDD N_VDD_M7_s
+ N_VDD_M8_s N_VDD_M8_b N_VDD_M9_s N_VDD_c_4_n PM_BUFFERS_M1_M1%VDD
x_PM_BUFFERS_M1_M1%K1 N_K1_M4_d N_K1_M9_d K1 PM_BUFFERS_M1_M1%K1
cc_1 N_OUT1_c_1_p N_OUT2_c_3_n 1.95094f
cc_2 N_OUT1_M5_d N_VDD_c_4_n 0.189198f
```



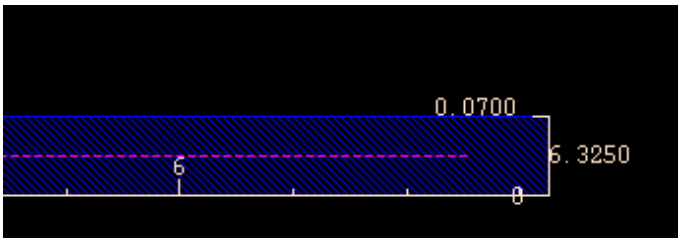
Per evidenziare come sono stati estratti i valori dei parametri parassiti, abbiamo usato il tasto Start RVE. Nel tab Parasitics dopo aver selezionato la net "out1" (l'uscita del primo invertitore) abbiamo selezionato la voce "Show Detailed Parasitic". Nella parte bassa della finestra sono stati riportati i dettagli riguardanti le resistenze e le capacità parassite ricavate dall'estrattore. Selezionando la voce R per vedere la lista di tutte le resistenze collegate alla net "out1", abbiamo notato che la resistenza distribuita sulla net si compone di ben 15 elementi.

Selezionando la resistenza dovuta al metal1 di valore maggiore e selezionando la voce "Highlight Selected Parasitic", abbiamo individuato nel layout editor il percorso al quale l'estrattore ha associato il valore di questa resistenza; esso corrisponde ad una delle linee di interconnessione tra i due circuiti, come si può notare di seguito:

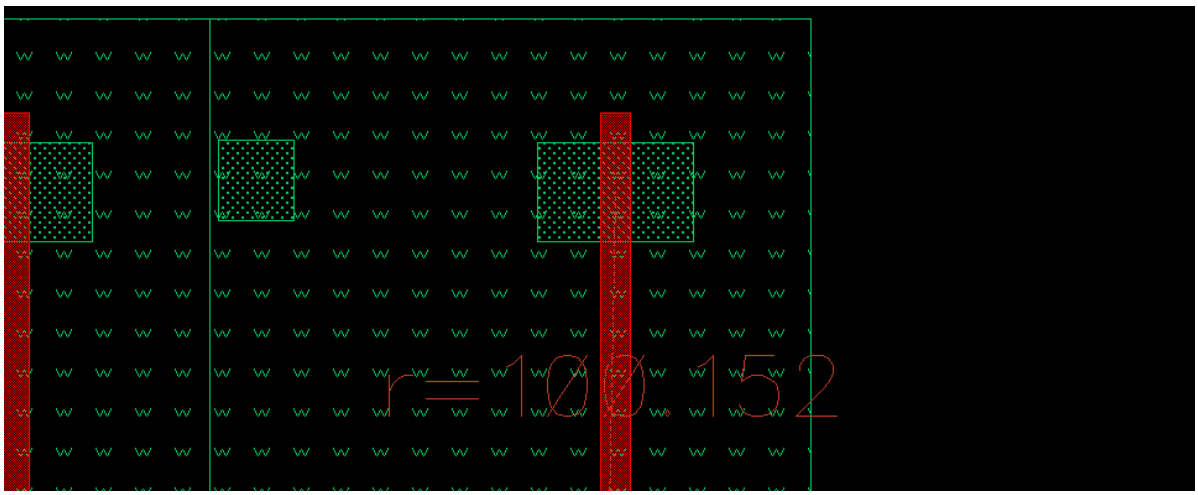


Alessandro Arrichiello M63/43
Flavio Battimo M63/38

Calcolando manualmente il valore atteso di questa resistenza, considerando che nella tecnologia che stiamo utilizzando il metal1 è caratterizzato da una resistenza di strato pari a 0.38Ω, abbiamo ottenuto una resistenza pari a R = 0.38 * L/W = 0.38 * 6.325 / 0.07 = 34.33 che si avvicina molto al valore calcolato: 33.49. Abbiamo in questo caso considerato le misure di W ed L preventivamente calcolate:



Si può notare inoltre sempre dalla finestra RVE che l'elemento che introduce la resistenza di valore maggiore è proprio il POLY.



Oltre la resistenza, l'estrattore ha calcolato anche due tipologie di capacità per ogni nodo: la capacità verso massa e le capacità di accoppiamento con gli altri nodi del circuito.

Selezionando la voce CC per vedere le capacità di cross-coupling collegate alla net "out1" abbiamo potuto verificare che le capacità riportate sono 37 riferite quasi tutte a VDD, come mostrato di seguito:

CC	37	2.14014E-15
out2	14	1.95094E-15
VDD	23	1.89198E-16

26	VDD	0.00000
27	VDD	0.00000
28	VDD	0.00000
29	VDD	0.00000
30	VDD	0.00000
31	VDD	1.89198E-16
32	VDD	0.00000
33	VDD	0.00000
34	VDD	0.00000
35	VDD	0.00000
36	VDD	0.00000
37	VDD	0.00000

L'estrazione a resistenze e capacità distribuite permette di ottenere la più accurata descrizione dei dispositivi passivi presenti nel circuito. D'altra parte la simulazione della netlist ottenuta in questo modo può richiedere elevati tempi di calcolo. E' possibile effettuare un'estrazione dei parassiti senza resistenze e con capacità localizzate (lumped).

Abbiamo quindi modificato la voce Extraction Type in modo da effettuare un'estrazione di tipo C+CC. Abbiamo ripetuto l'estrazione con queste nuove opzioni e la netlist ottenuta si è semplificata di molto. Essa già contiene tutte le capacità verso massa.

Inoltre include soltanto il file "buffers_m1_m1.pex.netlist.BUFFERS_M1_M1.pxi" che contiene l'elenco delle capacità di crosstalk.

```
* File: buffers_m1_m1.pex.netlist
* Created: Thu May 31 16:03:45 2012
* Program "Calibre xRC"
* Version "v2009.3_32.24"
*
.subckt BUFFERS_M1_M1  OUT1 IN1 IN2 OUT2 K2 VSS VDD K1
*
M0 OUT1 IN1 VSS VSS NMOS_VTL L=5e-08 W=3.6e-07 AD=5.22e-14 AS=3.78e-14
+ PD=1.01e-06 PS=9.3e-07
M1 VSS IN1 OUT1 VSS NMOS_VTL L=5e-08 W=3.6e-07 AD=3.96e-14 AS=5.22e-14
+ PD=9.4e-07 PS=1.01e-06
M2 OUT2 IN2 VSS VSS NMOS_VTL L=5e-08 W=1.65e-07 AD=1.7325e-14 AS=1.7325e-14
+ PD=5.4e-07 PS=5.4e-07
M3 K2 OUT2 VSS VSS NMOS_VTL L=5e-08 W=1.65e-07 AD=1.7325e-14 AS=1.7325e-14
+ PD=5.4e-07 PS=5.4e-07
M4 K1 OUT1 VSS VSS NMOS_VTL L=5e-08 W=1.65e-07 AD=1.7325e-14 AS=1.7325e-14
+ PD=5.4e-07 PS=5.4e-07
M5 OUT1 IN1 VDD VDD PMOS_VTL L=5e-08 W=5.4e-07 AD=7.83e-14 AS=5.67e-14
+ PD=1.37e-06 PS=1.29e-06
M6 VDD IN1 OUT1 VDD PMOS_VTL L=5e-08 W=5.4e-07 AD=5.94e-14 AS=7.83e-14
+ PD=1.3e-06 PS=1.37e-06
M7 OUT2 IN2 VDD VDD PMOS_VTL L=5e-08 W=1.65e-07 AD=1.7325e-14 AS=1.7325e-14
+ PD=5.4e-07 PS=5.4e-07
M8 K2 OUT2 VDD VDD PMOS_VTL L=5e-08 W=1.65e-07 AD=1.7325e-14 AS=1.7325e-14
+ PD=5.4e-07 PS=5.4e-07
M9 K1 OUT1 VDD VDD PMOS_VTL L=5e-08 W=1.65e-07 AD=1.7325e-14 AS=1.7325e-14
+ PD=5.4e-07 PS=5.4e-07
c_3 OUT1 0 0.722027f
c_4 IN1 0 0.272459f
c_5 IN2 0 0.192668f
c_7 OUT2 0 0.632078f
c_8 K2 0 0.204549f
c_9 VSS 0 0.90975f
c_11 VDD 0 0.747635f
c_12 K1 0 0.182231f
*
.include "buffers_m1_m1.pex.netlist.BUFFERS_M1_M1.pxi"
.ends
```

```
* File: buffers_m1_m1.pex.netlist.BUFFERS_M1_M1.pxi
* Created: Thu May 31 16:03:45 2012
*
cc_1 OUT1 OUT2 1.95185f
cc_2 OUT1 VDD 0.189436f
```

Simulazione del circuito estratto dal layout

Abbiamo successivamente avviato la simulazione attraverso il comando: "make_simulation", al termine della procedura è stato generato il file "tb_buffers_m1_m1.sp" che rappresenta un testbench in formato spice.

Dapprima richiama il file container.net che istanzia il circuito estratto con calibre, richiama i modelli dei MOSFET da utilizzare, e definisce i potenziali delle alimentazioni. Nel file sono poi riportati tre esempi che ricordano la sintassi dei generatori di tensione costante, di tipo pulse e di tipo pwl.

```
**** Title: tb_circuit.sp
```

```
*************************************************************
* INCLUSIONE CIRCUITO
*************************************************************
.include container.net
*************************************************************
* INCLUSIONE MODELLI TRANSISTORS
*************************************************************
.include
/home/caduser/ NCSU-FreePDK45-1.3/FreePDK45/ncsu_basekit/models/hspice/hspice_nom.include
*************************************************************
* COLLEGAMENTO ALIMENTAZIONI
*************************************************************
VVSS VSS 0 0
VVDD VDD 0 1.1
*************************************************************
* ESEMPI DI GENERATORI DI TENSIONE PER GLI INGRESSI:
*************************************************************
*segnali costanti
*VNOME NOME 0 1.1
VP2 IN2 0 0
*
*segnali PWL
*VP1 P1 0 pwl 0 0 100p 0 101p 1.1 200p 1.1 201p 0
VP1 IN1 0 pwl 0 0 100p 0 101p 1.1 200p 1.1 201p 0
*************************************************************
* PARAMETRI DELLA SIMULAZIONE
* cambiare la durata della simulazione
*************************************************************
.tran 1p 400p
.temp 25
*************************************************************
* PROBES:
*************************************************************
.probe tran I(VVDD) v(*) depth=1

.end
```

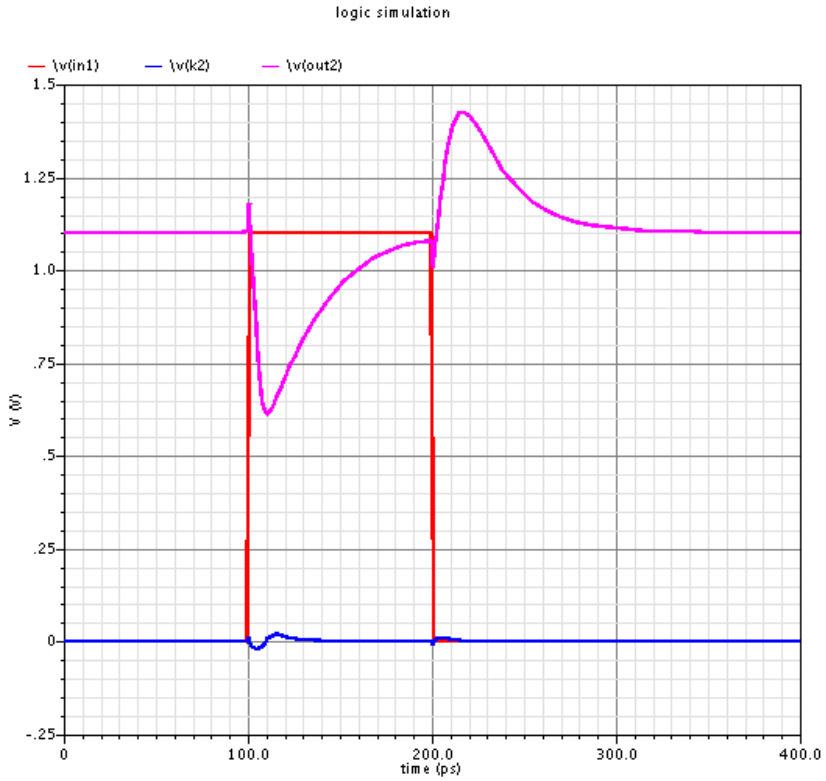
Abbiamo quindi definito l'andamento dei segnali di ingresso.

Abbiamo definito un generatore PWL in modo da effettuare una transizione 0→1→0 sull'ingresso in1, lasciando l'ingresso in2 costante, pari a zero.

Nella sezione parametri della simulazione abbiamo fissato la durata della simulazione a 400ps, la temperatura a 25 e la modalità di simulazione analog.

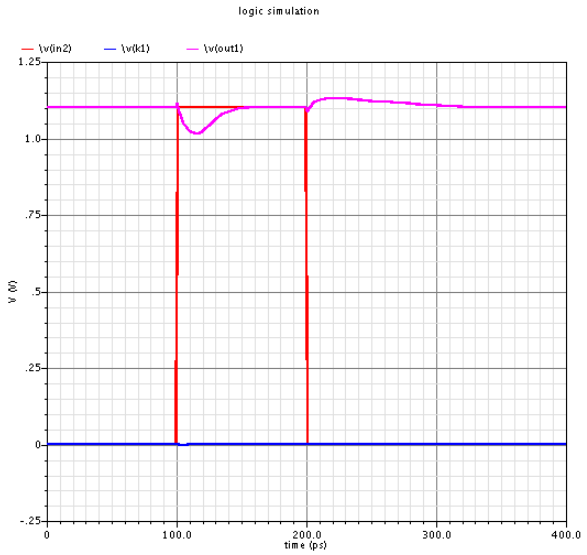
Infine abbiamo lanciato la simulazione: ultrasim +log buffers_m1_m1 tb_buffers_m1_m1.sp

Ed analizzato le forme d'onda con wavescan:



Come si può notare i segnali out2 e k2 sono in parte influenzati dalle transizioni del segnale in1.

Modificando i segnali di ingresso, in modo da effettuare la commutazione su in2, con in1 costante, si ottiene:



Che in questo caso solo out1 viene influenzato dalle transizioni di in2, mentre k1 resta invariato.

Alessandro Arrichiello M63/43
Flavio Battimo M63/38

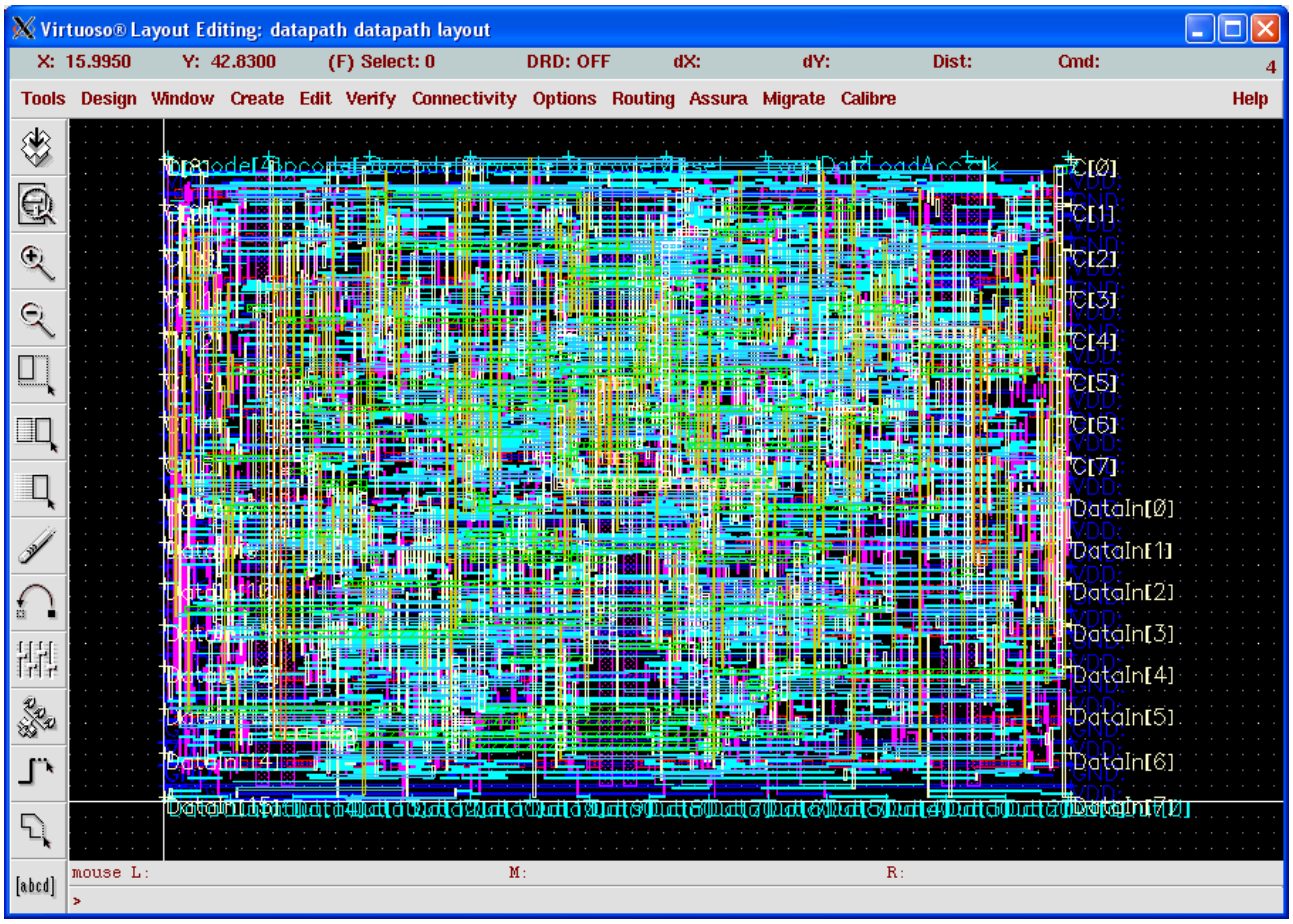
Estrazione del datapath di pico

E' possibile applicare le procedure viste in precedenza per avere la netlist a livello transistor del datapath di pico e per controllare che esso rispetti le regole di progetto. Per fare ciò innanzitutto occorre creare una libreria nella quale caricheremo i file ottenuti durante la fase di place & route.

Abbiamo quindi creato una nuova libreria chiamata "datapath" con technology file delle standard cells.

Abbiamo successivamente importato il file prodotto in fase di place & route dalla finestra icfb, attraverso la voce di menù "File -> import -> stream": datapath_final.gds.

Come si può notare adesso la libreria contiene diverse celle. Le celle il cui nome comincia con "via" sono collegamenti, realizzati dal router, tra differenti livelli di metallo e riutilizzati più volte dal router all'interno del layout del datapath. La cella datapath contiene invece l'intero layout realizzato con Encounter.



Il layout editor mostra il circuito in maniera gerarchica. Sono visibili tutte le wire e le label appartenenti al circuito più esterno della gerarchia, ma sono nascosti i layout delle sottocelle.

Controllo delle regole di progetto

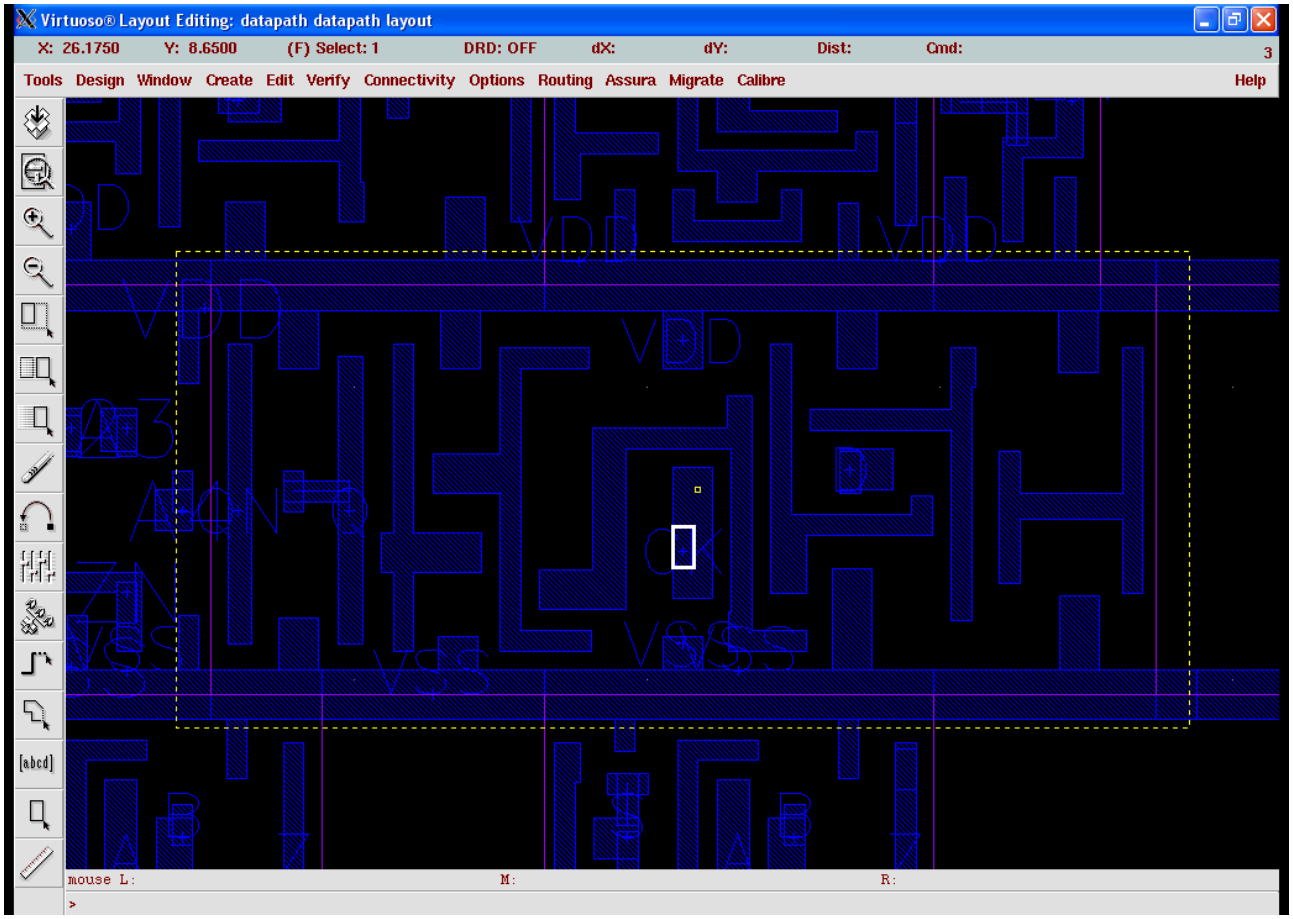
In questa fase abbiamo effettuato il DRC del datapath, attraverso l'apposito menu: "Calibre -> Run DRC" scegliendo il runset file "ese07/calibre_rundir/calibreDRC.runset". Dopo aver verificato l'assenza di errori siamo passati al passo successivo dell'esercitazione.

Alessandro Arrichiello M63/43
Flavio Battimo M63/38

Estrazione circuitale

Dobbiamo adesso estrarre il circuito a livello transistor ed effettuare una simulazine spice, al fine di valutare accuratamente il clock skew e la potenza dissipata a causa della commutazione del clock. A tal fine è stato necessario apporre delle etichette (labels) sui punti in cui volevamo avere accesso in fase di simulazione.

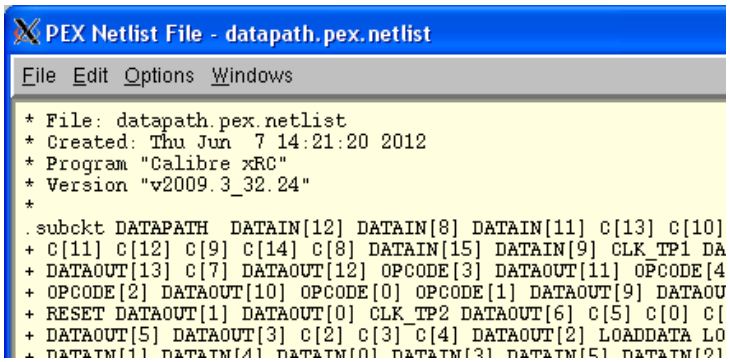
Nel nostro caso, abbiamo usato come "test points" i pin di clock dei due registri individuati nella esercitazione 5 come quelli per i quali si evidenzia il minimo ed il massimo ritardo sul clock.



Abbiamo quindi applicato le LABEL: "clk_tp1" e "clk_tp2" ai due punti individuati in precedenza, al fine (come già detto) di rendere più semplice la misurazione dei ritardi con ultrasim.

Nel layout editor quindi abbiamo selezionato la voce di menù "Calibre -> Run PEX" ed utilizzato il file di configurazione "ese07/calibre_rundir/calibrexRC.runset".

Nella sezione "Outputs" di calibre interactive abbiamo selezionato un'estrazione di tipo C+CC e attivato, come visto in precedenza, le opzioni dell'estrattore in modo da non includere nella netlist capacità di valore minore ad 1fF.



Al termine dell'estrazione abbiamo verificato che i testpoints clk_tp1 e clk_tp2 risultino fra i terminali del subcircuit estratto, come si può verificare a lato.

Alessandro Arrichiello M63/43
Flavio Battimo M63/38

Dopo aver lanciato il comando make_simulation al fine di creare l'albero delle directory necessario alla simulazione con il tool ultrasim, abbiamo modificato il file di testbench (tb_datapath.sp) al fine di effettuare un test appropriato della netlist estratta.

Nella parte relativa ai segnali di ingresso, abbiamo aggiunto la definizione del segnale di clock e richiamato il file datapath.vec che pone a zero tutti gli altri segnali d'ingresso del circuito:

```
VCLK CLK 0 pulse 0 1.1 0 0.02n 0.02n 1.25n 2.5n
.vec datapath.vec
```

Abbiamo quindi modificato i comandi di misura, al fine di valutare la potenza dissipata a causa della commutazione del clock ed i ritardi sui due test points:

```
.measure tran potenza avg I(VVDD)*V(VVDD) from = 0ns  to = 2.5ns
.measure tran delay1_rise trig v(clk) val=0.55 td=0 rise=1 targ v(clk_tp1) val=0.55 td=0 rise=1
.measure tran delay2_rise trig v(clk) val=0.55 td=0 rise=1 targ v(clk_tp2) val=0.55 td=0 rise=1
.measure tran delay1_fall trig v(clk) val=0.55 td=0 fall=1 targ v(clk_tp1) val=0.55 td=0 fall=1
.measure tran delay2_fall trig v(clk) val=0.55 td=0 fall=1 targ v(clk_tp2) val=0.55 td=0 fall=1
```

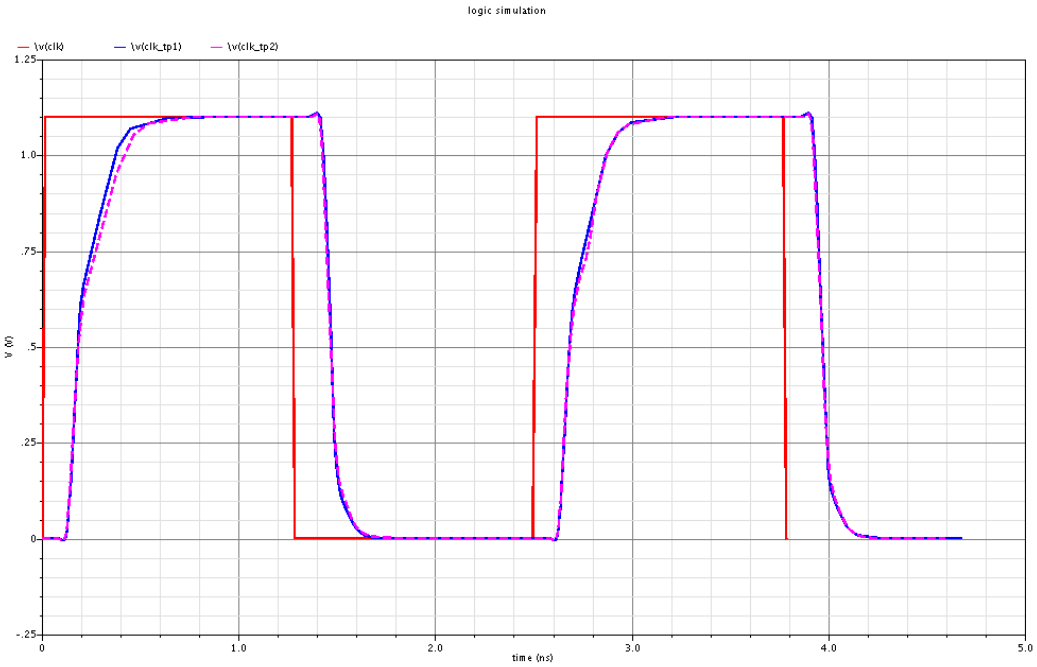
Nella sezione parametri della simulazione, abbiamo poi fissato la durata della simulazione a 5ns e la modalità di simulazione digital accurate:

```
.tran 1p 5n
.temp 25
.usim_opt sim_mode=da
```

Abbiamo quindi lanciato la simulazione con il comando:

```
ultrasim +log datapath.log tb_datapath.sp
```

Abbiamo quindi utilizzato wavescan per analizzare le forme d'onda di CLK, clk_tp1 e clk_tp2:



Come si nota pur essendoci un lieve delay tra CLK e clk_tp1/2, i segnali clk_tp1 e clk_tp2 sono quasi coincidenti.